

FPGA-Based Edge AI Acceleration for Vision Transformers

Knowledge Background, System Architecture, and HW/SW Co-Design

Capstone Project Research Manual

Ngày 14 tháng 9 năm 2026

Mục lục

Hướng dẫn đọc và câu hỏi kiến thức	v
1 Tổng quan kiến trúc Vision Transformer và nút thắt phần cứng Edge AI	1
1.1 Từ CNN đến Vision Transformer	1
1.1.1 Ví dụ mở đầu: bốn token tạo mười sáu quan hệ	2
1.2 Biểu thức toán học cốt lõi của ViT Encoder	2
1.2.1 Quy ước kích thước và patch embedding	2
1.2.2 Multi-head self-attention và chuẩn hóa trước nhánh	3
1.2.3 MLP và đường residual	4
1.3 Linear Unit, hàm kích hoạt và activation tensor	4
1.4 Loại bỏ Dropout và DropPath khi suy luận	5
1.5 Ba nút thắt phần cứng trên Edge FPGA	5
1.5.1 Chi phí GEMM: phân biệt thành phần tuyến tính và bậc hai	5
1.5.2 Memory bandwidth wall	6
1.5.3 Nút thắt phi tuyến và phạm vi Batch 1	7
2 Lượng tử hóa INT8 và xấp xỉ thuật toán phi tuyến	8
2.1 Mô hình lượng tử hóa uniform symmetric INT8	8
2.1.1 Scale, zero-point và miền biểu diễn	8
2.1.2 INT8 không đồng nghĩa với một định dạng fixed-point cố định	9
2.1.3 Ví dụ: cùng mã nguyên, khác scale	9
2.2 Dyadic rescaling và độ rộng trung gian	9
2.2.1 Tích lũy ma trận và bias	10
2.2.2 Biểu diễn hệ số bằng số nguyên và phép dịch	10
2.2.3 Quy tắc làm tròn phải là một phần giao diện	11
2.2.4 Ví dụ: dịch phải số âm không phải cắt về zero	11
2.3 ShiftGELU: xấp xỉ số nguyên có quy định rõ ràng về scale	12
2.3.1 Ví dụ xuyên suốt: mã 35 đi qua năm tầng	12
2.3.2 Từ GELU đến sigmoid và shift-add	13
2.3.3 Chứng minh hệ số và sai số do từng phép dịch	13
2.3.4 ShiftExp trên miền không dương	14
2.3.5 Sigmoid chín bit và tích đầu ra	15
2.3.6 Bất biến bộ chia và tích shift-add	15
2.3.7 Pipeline và phạm vi đánh giá	16
2.4 I-LayerNorm: thống kê, căn nguyên và affine	16
2.4.1 Thống kê theo token	16
2.4.2 Căn bậc hai bằng Newton nguyên	17

2.4.3	Ví dụ: từ hai đặc trưng đến ba bước Newton	18
2.4.4	Chứng minh hội tụ trên miền unsigned 32-bit	18
2.4.5	Chuẩn hóa và tham số affine bằng dyadic scaling	19
2.5	Residual connection và requantization	19
2.5.1	Hai scale đầu vào khác nhau	19
2.5.2	QAT scale-locking	20
2.5.3	Quy định dữ liệu giữa các tầng và các yêu cầu kiểm chứng còn lại	21
3	Thiết kế datapath và tối ưu hóa tài nguyên FPGA	22
3.1	Đơn vị nhân–tích lũy INT8	22
3.2	Cây cộng pipelined và quản lý bit-growth	23
3.2.1	Ví dụ: cây cộng ba phần tử và một bubble	23
3.2.2	Quy luật độ rộng và lịch thanh ghi	24
3.3	DSP packing trên DSP48E2	25
3.3.1	Ví dụ: đóng gói 12 nhân với 5 và âm 8	25
3.3.2	Đóng gói hai phép nhân dùng chung một toán hạng	26
3.3.3	Bù dấu khi tách trường cao	26
3.3.4	Hai tích mỗi chu kỳ khác hai MAC độc lập	27
3.4	Unified GEMM Engine và lịch systolic	27
3.5	Bộ đệm nội chip và residual	28
3.5.1	Half-layer mapping và giữ token gốc	28
3.5.2	Single-load policy và double S-buffer	28
3.6	Static Timing Analysis và timing closure	29
3.6.1	Setup và hold với quy ước skew rõ ràng	29
3.6.2	Phân biệt cách sửa setup và hold	29
4	Phân cấp bộ nhớ và tích hợp SoC thời gian thực	31
4.1	Bốn chặng dữ liệu từ lưu trữ đến datapath	31
4.1.1	Ẩn dụ nhà kho, khu tập kết và băng chuyền	31
4.1.2	Đối chiếu với hệ thống thực	32
4.1.3	Băng thông cực đại và băng thông hữu dụng	32
4.1.4	Cache coherency và vòng đời buffer	32
4.2	Matrix tiling theo đủ ba chiều	33
4.2.1	Dung lượng và reuse	33
4.2.2	Ví dụ: ma trận nhỏ vẫn cần đếm ba chiều	34
4.2.3	Ping-pong là giao thức sở hữu bank	34
4.2.4	Ví dụ: khi nào hai bank thực sự che được tải vào?	35
4.2.5	Ví dụ quyền sở hữu buffer trong một phiên loopback	35
4.3	Giao diện AXI-Stream và AXI4-Lite	36
4.3.1	Handshake, byte lane và biên packet	36
4.3.2	AXI4-Lite có năm kênh độc lập	37
4.4	Quy định giao diện và hoạt động của top-level Batch 2	37
4.5	Luồng webcam HIL và lộ trình suy luận thực	38
4.5.1	Phân hoạch mục tiêu khi có mô hình đã huấn luyện	38
4.5.2	Demo thực thi được mô tả trong Phụ lục D.1	38
4.5.3	Quy trình năm bước PS–PL và điều kiện thực hiện	38
4.5.4	Latency, FPS và các giới hạn báo cáo	39

5	Đồng kiểm thử, tích hợp SoC và đánh giá thực nghiệm	41
5.1	Phạm vi, phiên bản và trạng thái chứng minh	41
5.2	Scoreboard theo giao dịch, không theo vòng phát dữ liệu	42
5.2.1	Ví dụ: hai phiếu giao nhận và ba lỗi chủ động	42
5.2.2	Miền, tiền điều kiện và hậu điều kiện	43
5.2.3	Bất biến đếm và reset	43
5.2.4	Tránh race và đợi vô hạn	43
5.3	Độ chính xác số học và định dạng vector	44
5.3.1	Bit-exact khác sai số xấp xỉ	44
5.3.2	Chứng minh mã hex bù hai	44
5.4	STA, đóng gói IP và triển khai PYNQ	44
5.5	Đo hiệu năng và bảng trạng thái bằng chứng	45
	Kết luận và hướng phát triển	47
A	Bộ mã nguồn SystemVerilog RTL	48
A.1	ShiftGELU năm tầng: <code>shift_gelu_pipeline.sv</code>	48
A.1.1	Quy định đầu vào, đầu ra và phân tầng	48
A.1.2	Kiểm tra số học và giới hạn xác nhận	52
A.2	Residual dyadic: <code>requant_residual_add.sv</code>	52
A.3	Cây cộng pipelined: <code>pipelined_adder_tree.sv</code>	54
A.4	PE DSP packing: <code>pe_slice_dsp_packing.sv</code>	57
A.5	Khung top-level: <code>attention_core_top.sv</code>	59
B	Python lượng tử hóa và thu thập activation	66
B.1	Forward hooks: <code>quant_hooks.py</code>	66
B.1.1	Phạm vi và quy định về dữ liệu	66
B.1.2	Sử dụng và giới hạn	73
B.1.3	Kết quả kiểm tra hữu hạn khi refactoring	73
B.2	Xuất vector có manifest: <code>export_vectors.py</code>	74
B.3	Golden unit số nguyên: <code>golden_model_sim.py</code>	76
C	Suite Cocotb và cấu hình simulator	80
C.1	Scoreboard adder tree và wrapper phẳng	80
C.2	AXI-Stream và AXI4-Lite của shell loopback	83
C.3	Makefile cho hai simulator	86
D	Trình điều khiển PYNQ và webcam HIL	88
D.1	Host webcam: <code>realtime_vit_inference.py</code>	88
D.1.1	Phạm vi thực thi và cấu hình bắt buộc	88
D.1.2	Tiền/hậu điều kiện và quyền sở hữu tài nguyên	88
D.1.3	Mã nguồn hoàn chỉnh	89
D.1.4	An toàn buffer và giới hạn timeout	95
D.1.5	Trạng thái kiểm tra của bản biên soạn	95
D.1.6	Kiểm tra của lần rigor refactoring	96
D.2	Driver CMA/DMA tái sử dụng	96

E Vivado build, ràng buộc timing và nghiệm thu	102
E.1 Build có tiền điều kiện: <code>build_bitstream.tcl</code>	102
E.2 Ngân sách timing OOC: <code>timing_constraints.xdc</code>	106
E.3 Hồ sơ nghiệm thu bắt buộc	106
Tài liệu tham khảo	107

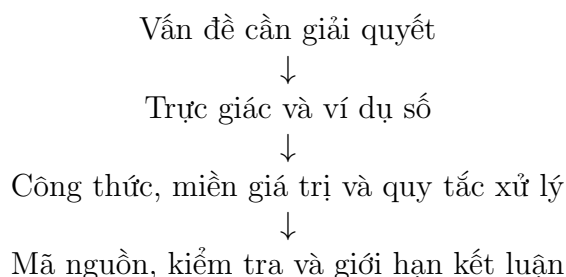
Hướng dẫn đọc và cầu nối kiến thức

Đối tượng và cách sử dụng tài liệu

Tài liệu dành cho sinh viên đã học giải tích, đại số tuyến tính và xác suất thống kê. Không cần biết trước toàn bộ thuật ngữ FPGA để bắt đầu: hãy đọc bản đồ hệ thống, tra bảng cầu nối khi gặp phép tính mới và chỉ tra danh mục thuật ngữ khi cần. Phần mở đầu là công cụ định hướng, không phải danh sách phải học thuộc trước Chương 1.

Ngoài toán đại cương, người học cần bổ sung ba ý niệm về mạch số: mạch tổ hợp tính từ đầu vào hiện tại; thanh ghi giữ trạng thái giữa các cạnh clock; giao thức quy định khi nào một giao dịch được nhận. Các ví dụ trong Chương 2–5 gắn ba ý niệm này với số cụ thể và lịch dữ liệu.

Mỗi nội dung được đọc theo thứ tự:



Có thể đọc theo ba lượt: lượt đầu theo dõi động cơ và ví dụ trong năm chương; lượt thứ hai đối chiếu công thức với các listing phụ lục; lượt thứ ba kiểm tra nghĩa vụ nghiệm thu. Không cần đọc mọi dòng RTL ngay lần đầu. Mỗi chương có mục tiêu và câu hỏi tự kiểm tra kèm tiêu chí đối chiếu.

Bản đồ hệ thống năm tầng

Bản đồ thể hiện lộ trình học và tích hợp, không khẳng định mọi khối đã chạy trên bo mạch. Công thức trên số thực lý tưởng phải được phân biệt với chương trình tham chiếu FP32 có làm tròn. Đọc các tầng theo thứ tự $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$; khi có sai lệch, quay lại kiểm tra các quy định về dữ liệu và giao tiếp giữa hai tầng.

Tầng	Câu hỏi của người học	Vị trí và trạng thái
1. Mô hình toán học	Tensor có ý nghĩa và kích thước nào?	Chương 1–2: mô hình ViT, reference và các quy định biểu diễn, tính toán; không đồng nhất số thực với FP32.
2. Chương trình PyTorch	Lấy activation và hệ số từ đâu?	Phụ lục B: công cụ thu thập/lượng tử hóa; người dùng phải cung cấp mô hình, checkpoint và dữ liệu calibration.
3. Số học INT8 dyadic	Mã, scale, round và saturation được quy định thế nào?	Chương 2 và Phụ lục B: reference số nguyên cho các unit; INT8 không tự xác định scale.
4. Datapath RTL	Dữ liệu nằm trong thanh ghi nào, có bao nhiêu bit và valid khi nào?	Chương 3 và Phụ lục A: các module độc lập và shell giao tiếp; chưa tích hợp đầy đủ attention.
5. SoC và HIL	PS, DMA và PL phối hợp và xử lý lỗi ra sao?	Chương 4–5, Phụ lục C–E: driver, kiểm thử và build có tiền điều kiện; chưa nghiệm thu inference trên KV260.

Bảng cầu nối toán học, ViT và FPGA

Đây là các liên hệ để định hướng đọc, không phải ánh xạ phần cứng duy nhất. Cơ sở mô hình và số học tham chiếu ViT, LayerNorm và I-ViT [1, 2, 3]; các đặc điểm DSP và DMA tham chiếu tài liệu phần cứng [4, 5].

Toán đại cương	Ứng dụng trong ViT	Liên hệ với FPGA PL
Nhân ma trận, tích vô hướng	Chiều Q, K, V; điểm attention; MLP.	Phân rã thành nhân và cộng; tổ chức bằng MAC hoặc mảng systolic tùy lịch và tài nguyên.
Chia ma trận thành khối	Tính một phần của ma trận lớn.	Tiling giới hạn dữ liệu giữ gần PE; phải tích lũy đủ theo chiều reduction trước requantization theo quy tắc đã chọn.
Trung bình, phương sai	LayerNorm theo đặc trưng của một token.	Bộ tích lũy và buffer giữ dữ liệu trong lúc chờ thống kê; phải chặn miền tổng.

Tiếp ở trang sau

Toán đại cương	Ứng dụng trong ViT	Liên hệ với FPGA PL
Căn và phương pháp lặp	Đại lượng chuẩn hóa trong LayerNorm.	Reference căn nguyên có khởi tạo và hậu điều kiện; số vòng lặp không tự bằng số clock.
Hàm mũ, chuẩn hóa theo tổng	Softmax trên các điểm attention.	Có thể nghiên cứu LUT hoặc xấp xỉ nguyên; shell hiện tại chưa có toàn khối này.
Hàm phi tuyến, xấp xỉ	GELU trong MLP.	ShiftGELU có quy định riêng về scale; sai số và tài nguyên sau tổng hợp phải đánh giá riêng.
Nhân hệ số, làm tròn	Đổi scale lượng tử hóa.	Dyadic rescaling giữ tích đủ rộng, dịch theo quy tắc rồi saturation.
Chia lấy sàn cho lũy thừa hai	Một quy tắc requantization.	Dịch phải số học trên toán hạng signed có thể hiện thực quy tắc đó; signedness không được bỏ qua.
Modulo và bù hai	Mã âm, hex và trường bit.	Tách trường phải giải mã signed; packed multiplication trong dự án cần bù trường cao khi tích thấp âm.
Tổng và bất đẳng thức chặn	Giới hạn tích lũy và sai số.	Chọn bit-width trước phép tính; saturation cuối không sửa được overflow trung gian.
Logic và bất biến	Sự kiện nhận/phát dữ liệu.	Handshake, valid/ready và quyền sở hữu buffer được kiểm tra theo cạnh clock.
Sai số bình phương, tỷ số năng lượng	So sánh đầu ra với reference.	MSE/RMSE/SNR dùng cùng đơn vị sau giải lượng tử; không thay kiểm tra bit-exact.

Quy tắc thuật ngữ song ngữ

Lần đầu trong phần diễn giải, dùng *tên tiếng Việt* (*tên tiếng Anh*; viết tắt nếu có). Sau đó chọn một tên nhất quán; khi đọc riêng một chương, có thể tra lại danh mục thay vì lặp mọi định nghĩa. Tên riêng PyTorch, PYNQ, Cocotb và SystemVerilog giữ nguyên. Không dịch identifier, tên cổng, thanh ghi hoặc tên tệp.

- Dùng “lượng tử hóa” cho quantization; không dùng lẫn “định lượng”.
- Dùng “đường tới hạn” cho critical path; “độ rộng bit” cho bit-width.
- Dùng “khoảng cách khởi phát” cho initiation interval (II). Phải nói rõ một công việc là mẫu, vector, beat hay tile.
- Dùng “độ trễ” cho latency, kèm đơn vị và quy ước: số tầng, khoảng cách cạnh clock hoặc thời gian thực.
- Dùng “bit đệm giữa các trường” cho guard bits; tên gọi không bảo đảm loại bỏ mọi carry hay borrow.

- Phân biệt mục tiêu thiết kế, kết quả reference, kết quả mô phỏng HDL và số đo bo mạch. “Lập luận theo tinh thần Lean” không đồng nghĩa có chứng thư Lean kernel.

Danh mục từ viết tắt và thuật ngữ

Các định nghĩa ngắn giúp định vị khái niệm. Miền giá trị, điều kiện áp dụng và bằng chứng nằm ở các chương tương ứng. Các nhóm giao tiếp và kiểm thử đối chiếu tài liệu AXI, Vivado, PYNQ và Cocotb [6, 7, 8, 9, 10].

Thuật ngữ	Tên gốc	Giải thích và giới hạn
Mô hình và số học		
ViT	Vision Transformer	Xử lý ảnh bằng chuỗi token biểu diễn patch và các khối Transformer.
CNN	Convolutional Neural Network	Mạng tích chập; cửa sổ của một phép tích chập là cục bộ nhưng trường tiếp nhận có thể tăng qua nhiều lớp.
Token	Token	Phần tử của chuỗi mô hình; không phải một byte hay một beat AXI.
Patch embedding	Patch Embedding	Chiều patch ảnh thành vector đặc trưng có chiều ẩn đã chọn.
MSA	Multi-Head Self-Attention	Tự chú ý nhiều đầu; kết hợp thông tin theo các phép chiếu và lịch toán học của mô hình.
Q, K, V	Query, Key, Value	Tensor tạo điểm tương tác và kết hợp giá trị; không tự xác định vị trí lưu trữ vật lý.
MLP	Multi-Layer Perceptron	Khối biến đổi affine và kích hoạt; cấu hình cụ thể phụ thuộc mô hình.
GEMM	General Matrix Multiplication	Phép toán nhân ma trận tổng quát, dùng tổ chức nhiều lớp tuyến tính.
MAC	Multiply-Accumulate	Phép nhân rồi cộng vào tổng; phân biệt phép toán với bộ MAC vật lý.
FP32	32-bit Floating Point	Số dấu phẩy động 32 bit có độ chính xác hữu hạn; không phải số thực lý tưởng.
INT8	Signed 8-bit Integer	Mã có dấu tám bit trong $[-128, 127]$; ý nghĩa số thực cần thêm scale.
Scale	Quantization Scale	Hệ số liên hệ mã nguyên với giá trị thực; không suy được chỉ từ INT8.
Zero-point	Quantization Zero Point	Mã biểu diễn số thực không; quy ước lượng tử hóa symmetric của dự án dùng mã zero.
QAT	Quantization-Aware Training	Huấn luyện có mô hình hóa lượng tử hóa; một observer chưa phải toàn quy trình QAT.
RNE	Round to Nearest, Ties to Even	Làm tròn đến số nguyên gần nhất; đúng giữa hai số thì chọn số chẵn.

Tiếp ở trang sau

Thuật ngữ	Tên gốc	Giải thích và giới hạn
Dyadic rescaling	Dyadic Rescaling	Đổi scale bằng hệ số nguyên và lũy thừa âm của hai, kèm round và saturation đã chốt.
GELU	Gaussian Error Linear Unit	Hàm kích hoạt phi tuyến; phân biệt reference toán học với xấp xỉ phần cứng.
LayerNorm	Layer Normalization	Chuẩn hóa nhóm đặc trưng đã chọn rồi áp dụng affine; không tự đổi thành chuẩn hóa cả batch.
Saturation	Saturation	Kẹp giá trị vào miền biểu diễn; không đồng nghĩa lấy các bit thấp.

Datapath và tài nguyên FPGA

FPGA	Field-Programmable Gate Array	Vì mạch có tài nguyên logic và kết nối cấu hình được để hiện thực mạch số.
HDL	Hardware Description Language	Ngôn ngữ mô tả phần cứng; simulator và công cụ tổng hợp diễn giải theo các mục đích khác nhau.
RTL	Register-Transfer Level	Mô tả thao tác dữ liệu và chuyển trạng thái giữa thanh ghi.
PE	Processing Element	Phần tử xử lý có chức năng/giao diện phải được định nghĩa cho từng kiến trúc.
LUT	Look-Up Table	Tài nguyên hiện thực hàm logic; phân biệt LUT vật lý với bảng tra thuật toán tùy kích thước.
FF	Flip-Flop	Phần tử giữ một bit trạng thái; nhiều FF tạo thanh ghi dữ liệu.
DSP	Digital Signal Processing Slice	Khối số học chuyên dụng của FPGA, không phải tên một chương trình xử lý tín hiệu.
BRAM	Block RAM	Bộ nhớ khối tích hợp; đủ byte chưa bảo đảm đủ cổng hoặc băng thông.
LUTRAM	Distributed RAM Using LUT Resources	Bộ nhớ dùng LUT có hỗ trợ chức năng này, tùy kiến trúc FPGA.
Systolic array	Systolic Array	Mảng PE phối hợp theo lịch, truyền toán hạng hoặc kết quả giữa các phần tử.
Pipeline	Pipeline	Chia xử lý thành tầng; các mẫu khác nhau có thể nằm ở các tầng khác nhau.
Bubble	Pipeline Bubble	Vị trí không mang giao dịch hợp lệ; dữ liệu thanh ghi vẫn có thể thay đổi khi valid bằng zero.
II	Initiation Interval	Khoảng cách khởi phát công việc mới; II bằng một không có nghĩa một clock là xong kết quả.
Latency	Latency	Độ trễ giữa sự kiện đầu vào và đầu ra đã định nghĩa, với đơn vị đo rõ ràng.

Tiếp ở trang sau

Thuật ngữ	Tên gốc	Giải thích và giới hạn
Bit-growth	Bit-Width Growth	Độ rộng cần tăng để giữ kết quả trong miền đang chứng minh.
Guard bits	Guard Bits	Bit đệm giữa trường; vẫn phải kiểm tra carry/borrow khi xử lý số có dấu.
Retiming	Register Retiming	Phân bố lại thanh ghi để tối ưu timing dưới điều kiện bảo toàn hành vi; cần kiểm tra reset/giao diện.
FSM	Finite-State Machine	Máy trạng thái hữu hạn điều khiển trình tự và điều kiện chuyển trạng thái.
Bộ nhớ và tích hợp		
SoC	System on Chip	Hệ thống tích hợp nhiều khối chức năng trên chip.
PS	Processing System	Phần hệ thống xử lý, gồm bộ xử lý và thành phần liên quan trong nền tảng đang xét.
PL	Programmable Logic	Phần logic lập trình được của nền tảng SoC.
DDR	Double Data Rate Memory	Bộ nhớ ngoài; băng thông danh định không tự là băng thông hữu dụng.
CMA	Contiguous Memory Allocator	Cơ chế cấp phát liên tục vật lý; cấp phát không thay thế đồng bộ cache.
DMA	Direct Memory Access	Bộ máy/cơ chế chuyển dữ liệu mà không yêu cầu CPU sao chép từng phần tử.
AXI	Advanced eXtensible Interface	Họ giao thức; cần chỉ rõ biến thể thay vì dùng AXI như một bộ quy tắc tín hiệu duy nhất.
AXI4-Stream	AXI4-Stream	Truyền luồng; nhận một beat khi valid và ready cùng được khẳng định tại cạnh clock.
AXI4-Lite	AXI4-Lite	Giao diện memory-mapped cho truy cập thanh ghi; địa chỉ và dữ liệu ghi có kênh độc lập.
Beat	Stream Beat	Một đơn vị dữ liệu truyền trong một lần handshake; số byte hữu dụng còn phụ thuộc TKEEP.
MMIO	Memory-Mapped Input/Output	CPU truy cập thanh ghi thiết bị qua địa chỉ ánh xạ, khác với sao chép payload bằng DMA.
MM2S	Memory-Mapped to Stream	Hướng từ bộ nhớ ra stream của AXI DMA.
S2MM	Stream to Memory-Mapped	Hướng từ stream vào bộ nhớ của AXI DMA.
Backpressure	Backpressure	Bên nhận trì hoãn nhận; nguồn phải giữ dữ liệu ổn định khi chưa được nhận.

Tiếp ở trang sau

Thuật ngữ	Tên gốc	Giải thích và giới hạn
Ping-pong buffer	Ping-Pong Buffer	Hai bank luân phiên theo quyền sở hữu; không tự bảo đảm luôn chồng lấp truyền/tính.
PYNQ	PYNQ	Hệ sinh thái điều khiển nền tảng logic lập trình được từ Python; không tự cung cấp thuật toán còn thiếu.
HIL	Hardware-in-the-Loop	Phần cứng thật tham gia vòng thực thi/kiểm tra; mock Python không phải phép thử HIL.

Kiểm thử và đánh giá

DUT	Design Under Test	Thiết kế đang kiểm thử; phải chốt phạm vi trước khi diễn giải kết quả.
Cocotb	Cocotb	Framework testbench Python dựa trên coroutine, cần simulator để thực thi HDL.
Epoch kiểm thử	Verification Epoch	Khoảng kiểm tra giữa các lần reset; giao dịch bị hủy bởi reset không chuyển thành đáp án chờ của epoch mới.
Golden model	Golden Reference Model	Mô hình tham chiếu cùng quy định tính toán và cách bố trí dữ liệu với chức năng đang kiểm tra.
Scoreboard	Scoreboard	Lưu đáp án và đối chiếu giao dịch về giá trị, số lượng, thứ tự.
Bit-exact	Bit-Exact Agreement	Khớp chính xác mã bit theo quy định biểu diễn và xử lý, không tự chứng minh accuracy phân loại.
STA	Static Timing Analysis	Phân tích ràng buộc thời gian mà không duyệt mọi vector dữ liệu đầu vào.
WNS	Worst Negative Slack	Chỉ tiêu slack setup xấu nhất trong phạm vi báo cáo; cần đọc cùng constraints và đường chưa ràng buộc.
WHS	Worst Hold Slack	Chỉ tiêu slack hold xấu nhất; không đủ để nghiệm thu toàn hệ thống khi đứng riêng.
Critical path	Critical Path	Đường tới hạn theo tiêu chí timing đang xét, không chỉ đường có nhiều phép toán trong source.
CDC	Clock-Domain Crossing	Truyền tín hiệu giữa miền clock, cần cơ chế và kiểm tra phù hợp từng loại tín hiệu.
MSE	Mean Squared Error	Trung bình bình phương sai số giữa các đại lượng có cùng cách diễn giải và đơn vị.
RMSE	Root Mean Squared Error	Căn của MSE, có cùng đơn vị với đầu ra được đo.

Tiếp ở trang sau

Thuật ngữ	Tên gốc	Giải thích và giới hạn
SNR	Signal-to-Noise Ratio	Tỷ số năng lượng tín hiệu/sai số theo quy ước; xử lý riêng trường hợp năng lượng bằng zero.
FPS	Frames Per Second	Số frame hoàn tất mỗi giây; phải ghi rõ loopback, một khối hay toàn inference.
GOPS/W	Giga Operations per Second per Watt	Thông lượng phép toán chia công suất trong cùng phạm vi đo; không suy từ FPS loopback.

Tự kiểm tra trước khi đọc sâu

Bạn đã sẵn sàng đọc một module khi có thể nêu ý nghĩa đầu vào, tính một ví dụ nhỏ, chỉ ra điều kiện biên và xác định loại bằng chứng cần kiểm tra. Nếu chưa làm được, quay lại bảng câu hỏi và ví dụ của chương thay vì đoán từ tên thuật toán. Các phụ lục là mã triển khai để đối chiếu, không thay phần giải thích tại Chương 1 đến Chương 5.

Chương 1

Tổng quan kiến trúc Vision Transformer và nút thắt phần cứng Edge AI

Mục tiêu học tập

- Đi từ ảnh, patch đến token và kiểm tra kích thước phép nhân ma trận.
- Giải thích vì sao tương tác giữa mọi cặp token tạo một bảng bậc hai, nhưng không nhất thiết chiếm đa số MAC của cả block.
- Phân biệt mô hình mong muốn với những module đã được viết và những phép đo chưa có.

Bạn đang ở đâu? Đây là tầng mô hình trong bản đồ năm tầng, trang vi. Hãy tưởng tượng mỗi patch là một phiếu thông tin: phép chiếu tạo vector cho phiếu, còn tự chú ý (Self-Attention) xác định cách kết hợp thông tin giữa các phiếu. Hình ảnh này chỉ giải thích vai trò dữ liệu; trọng số attention vẫn phải được tính từ Q, K theo công thức, không do người đọc gán cảm tính.

Trạng thái chứng minh (proof state). Chương này chứng minh các mệnh đề về kích thước tensor, số phép toán và ánh xạ đồng nhất trên số thực. Kết quả có điều kiện được ghi cùng giả thiết; không coi chúng là chứng nhận Lean kernel, mô phỏng RTL hoặc báo cáo STA. Mỗi kiểm tra gồm: tiền điều kiện, biến đổi và hậu điều kiện. Chưa có tệp Lean hay chứng thư hình thức được thực thi cho tài liệu này.

1.1 Từ CNN đến Vision Transformer

Mạng nơ-ron tích chập (Convolutional Neural Network; CNN) xây dựng đặc trưng bằng các phép tích chập có trọng số dùng chung trên những vùng lân cận. Thiên kiến quy nạp về tính cục bộ và tính tương đương theo phép tịnh tiến giúp tận dụng cấu trúc ảnh. Trường tiếp nhận có thể mở rộng qua nhiều tầng, vì vậy không nên diễn giải rằng CNN hoàn toàn không mô hình hóa được quan hệ khoảng cách xa.

Mô hình Transformer thị giác (Vision Transformer; ViT) biểu diễn ảnh thành một chuỗi patch và dùng self-attention để kết nối các vị trí trong chuỗi. Trong kiến trúc ViT gốc, LayerNorm đứng trước mỗi nhánh attention/MLP và residual nối sau nhánh đó [1]. Khả năng tương tác toàn cục không có nghĩa mô hình không còn bất kỳ thiên kiến cấu

trúc nào: cách chia patch, embedding vị trí và các phép chiếu vẫn xác định tổ chức dữ liệu.

Trong báo cáo này, đối tượng thiết kế là *suy luận* một mô hình đã huấn luyện, không phải tăng tốc backpropagation. Mục tiêu là chuyển một số phép toán sang datapath số nguyên, đồng thời bảo đảm phần mềm và phần cứng dùng cùng quy định về scale, thứ tự tensor và độ trễ. Mọi số liệu tài nguyên, tần số và độ chính xác phân loại chỉ được coi là kết quả thực nghiệm khi có phép đo tương ứng; chúng không suy ra trực tiếp từ tên thuật toán.

1.1.1 Ví dụ mở đầu: bốn token tạo mười sáu quan hệ

Xét riêng một đầu attention với $T = 4$ token và $d_k = 2$ thành phần mỗi vector. Đây là chuỗi minh họa đã cố định, không suy ra rằng mô hình ViT đầy đủ bỏ class token. Cho $Q, K \in \mathbb{R}^{4 \times 2}$:

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 0 \end{bmatrix}, \quad QK^T = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 2 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \quad (1.1)$$

Hàng thứ ba hỏi mức tương tác của token 3 với từng token. Chẳng hạn ô $(3, 2)$ là $1 \cdot 0 + 1 \cdot 1 = 1$, còn ô $(3, 3)$ là $1 \cdot 1 + 1 \cdot 1 = 2$. Mỗi ô cần hai phép nhân và một phép cộng nếu không tính cộng vào tổng khởi tạo zero. Có $4 \cdot 4 = 16$ ô, do đó có 32 phép nhân và 16 phép cộng theo cách đếm này.

Trước Softmax, từng ô được chia cho $\sqrt{2}$; sau Softmax theo hàng, mới nhân với tensor V. Ma trận trong (1.1) là điểm chưa chuẩn hóa, không phải xác suất. Hàng điểm toàn zero không có nghĩa output attention bằng zero: Softmax của bốn điểm zero cho bốn trọng số $1/4$.

Nếu tăng lên $T = 8$ mà giữ $d_k = 2$, bảng có $8 \cdot 8 = 64$ ô: $64/16 = 4$ lần số ô ban đầu. Một cửa sổ tích chập cố định chỉ kết nối các vị trí lân cận trong cửa sổ của một lớp; dense attention lập quan hệ cho mọi cặp vị trí. So sánh này giải thích tính bậc hai của bảng điểm, không phủ nhận việc nhiều lớp CNN mở rộng trường tiếp nhận [1].

1.2 Biểu thức toán học cốt lõi của ViT Encoder

1.2.1 Quy ước kích thước và patch embedding

Để tránh dùng cùng ký hiệu cho chiều cao ảnh và số head, đặt $H_{\text{img}}, W_{\text{img}}$ là kích thước ảnh, C là số kênh, P là cạnh patch, h là số head, D là chiều embedding và L là số encoder block. Miền kích thước là $H_{\text{img}}, W_{\text{img}}, C, P, h, D, L \in \mathbb{Z}_{>0}$, với $P \mid H_{\text{img}}, P \mid W_{\text{img}}$ và $h \mid D$. Chỉ số block $\ell \in \{1, \dots, L\}$; chỉ số head $a \in \{1, \dots, h\}$. Các phép toán trong chương dùng số thực hữu hạn đầu vào, không dùng ngữ nghĩa overflow FP32. Khi đó:

$$X \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times C}, \quad N_p = \frac{H_{\text{img}} W_{\text{img}}}{P^2}, \quad T = N_p + 1. \quad (1.2)$$

N_p đếm patch, còn T bao gồm token phân loại [CLS]. Nếu mô hình không dùng token này, thay T bằng N_p . Với patch đã làm phẳng $x_p^{(j)} \in \mathbb{R}^{1 \times P^2 C}$, $j \in \{1, \dots, N_p\}$:

$$x_{\text{cls}} \in \mathbb{R}^{1 \times D}, \quad E \in \mathbb{R}^{P^2 C \times D}, \quad (1.3)$$

$$E_{\text{pos}} \in \mathbb{R}^{T \times D}, \quad (1.4)$$

$$z_0 = [x_{\text{cls}}; x_p^{(1)} E; \dots; x_p^{(N_p)} E] + E_{\text{pos}}, \quad z_0 \in \mathbb{R}^{T \times D}. \quad (1.5)$$

Kiểm tra shape. Vì $(H_{\text{img}}/P)(W_{\text{img}}/P) \in \mathbb{Z}_{>0}$, N_p là số nguyên dương. Tích $(1 \times P^2 C)(P^2 C \times D)$ tạo một hàng $1 \times D$; ghép một hàng CLS với N_p hàng patch tạo $(N_p + 1) \times D = T \times D$. Phép cộng với E_{pos} do đó xác định theo từng phần tử và trả $z_0 \in \mathbb{R}^{T \times D}$.

Mỗi hàng tensor tương ứng một token; định nghĩa thứ tự patch và thứ tự kênh phải được giữ nguyên khi xuất vector phần cứng.

1.2.2 Multi-head self-attention và chuẩn hóa trước nhánh

Với block thứ ℓ , đặt $U_\ell = \text{LayerNorm}(z_{\ell-1})$. Các phép chiếu có thể tính riêng hoặc ghép thành một GEMM với đầu ra rộng $3D$:

$$Q = U_\ell W_Q + b_Q, \quad K = U_\ell W_K + b_K, \quad V = U_\ell W_V + b_V, \quad (1.6)$$

$$W_Q, W_K, W_V \in \mathbb{R}^{D \times D}, \quad Q, K, V \in \mathbb{R}^{T \times D}. \quad (1.7)$$

Các miền còn lại là $U_\ell, z_{\ell-1} \in \mathbb{R}^{T \times D}$, $b_Q, b_K, b_V, b_O \in \mathbb{R}^{1 \times D}$ và $W_O \in \mathbb{R}^{D \times D}$. Bias được broadcast theo token. Khi D chia hết cho h , đặt $d_k = D/h$ và chia từng tensor thành h head. Với head a :

$$Q_a, K_a, V_a \in \mathbb{R}^{T \times d_k}, \quad (1.8)$$

$$A_a = \frac{Q_a K_a^\top}{\sqrt{d_k}}, \quad A_a \in \mathbb{R}^{T \times T}, \quad (1.9)$$

$$P_{a,ij} = \frac{\exp(A_{a,ij} - m_{a,i})}{\sum_{j'=1}^T \exp(A_{a,ij'} - m_{a,i})}, \quad m_{a,i} = \max_{1 \leq j \leq T} A_{a,ij}, \quad (1.10)$$

$$O_a = P_a V_a, \quad O_a \in \mathbb{R}^{T \times d_k}, \quad (1.11)$$

$$\text{MSA}(U_\ell) = \text{Concat}(O_1, \dots, O_h) W_O + b_O. \quad (1.12)$$

Tiền điều kiện và hậu điều kiện Softmax. Với $i, j \in \{1, \dots, T\}$, đặt tổng chuẩn hóa dương:

$$Z_{a,i} = \sum_{j=1}^T \exp(A_{a,ij}) > 0. \quad (1.13)$$

Tính chất $\exp(v - w) = \exp(v) \exp(-w)$ cho:

$$\sum_{j=1}^T \exp(A_{a,ij} - m_{a,i}) = \exp(-m_{a,i}) Z_{a,i} > 0, \quad (1.14)$$

$$P_{a,ij} = \frac{\exp(A_{a,ij}) \exp(-m_{a,i})}{\exp(-m_{a,i}) Z_{a,i}} = \frac{\exp(A_{a,ij})}{Z_{a,i}}, \quad (1.15)$$

$$\sum_{j=1}^T P_{a,ij} = Z_{a,i} / Z_{a,i} = 1, \quad 0 < P_{a,ij} \leq 1. \quad (1.16)$$

Như vậy $P_a \in \mathbb{R}^{T \times T}$; nhân với $V_a \in \mathbb{R}^{T \times d_k}$ trả $O_a \in \mathbb{R}^{T \times d_k}$. Ghép h đầu cho chiều $hd_k = D$, rồi nhân W_O giữ shape $T \times D$. Đây là chứng minh trên số thực; một hiện thực hữu hạn cần quy định riêng về sai số và xử lý underflow.

Softmax thực hiện *theo từng hàng*, không theo toàn bộ ma trận. Trừ giá trị cực đại giữ nguyên tỷ số số thực và tránh tính hàm mũ tại đầu vào dương lớn. Trong hiện thực số nguyên, sai số làm tròn vẫn cần được đo; biến đổi ổn định số không tự bảo đảm tương đương bit với FP32.

1.2.3 MLP và đường residual

Với tỷ lệ mở rộng bằng bốn, MLP thực hiện hai phép biến đổi affine:

$$\text{MLP}(U) = \text{GELU}(UW_1 + b_1)W_2 + b_2, \quad (1.17)$$

$$W_1 \in \mathbb{R}^{D \times 4D}, \quad W_2 \in \mathbb{R}^{4D \times D}. \quad (1.18)$$

Ở đây $U \in \mathbb{R}^{T \times D}$, $b_1 \in \mathbb{R}^{1 \times 4D}$, $b_2 \in \mathbb{R}^{1 \times D}$. Phép nhân thứ nhất trả $T \times 4D$, GELU tác động từng phần tử giữ shape, phép nhân thứ hai trả $T \times D$. Hai nhánh của một block pre-normalization được viết đầy đủ:

$$\hat{z}_\ell = z_{\ell-1} + \text{MSA}(\text{LayerNorm}(z_{\ell-1})), \quad (1.19)$$

$$z_\ell = \hat{z}_\ell + \text{MLP}(\text{LayerNorm}(\hat{z}_\ell)), \quad 1 \leq \ell \leq L. \quad (1.20)$$

Đường shortcut phải giữ dữ liệu đến khi nhánh tính toán tương ứng hoàn tất. Trên luồng có backpressure, độ sâu FIFO không thể chỉ suy ra từ số thanh ghi của nhánh: việc ghép hai toán hạng phải dựa trên cùng token và cùng giao dịch hợp lệ.

1.3 Linear Unit, hàm kích hoạt và activation tensor

Trong tài liệu này, *linear layer* chỉ phép $XW + b$; có bias thì đó là ánh xạ affine. Tên gọi “Linear Unit” trong ReLU/GELU không có nghĩa toàn bộ hàm kích hoạt là tuyến tính. Các định nghĩa cần phân biệt là:

$$\text{ReLU}(x) = \max(0, x), \quad (1.21)$$

$$\text{SiLU}(x) = x\sigma(x), \quad \sigma(x) = \frac{1}{1 + e^{-x}}, \quad (1.22)$$

$$\text{GELU}(x) = x\Phi(x), \quad \Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-t^2/2} dt. \quad (1.23)$$

GELU là hàm có trọng số theo phân phối chuẩn [11]. Vùng dương lớn gần x không đồng nghĩa hàm loại bỏ hoàn toàn vanishing gradient. Thay GELU bằng ReLU cũng là thay đổi mô hình, không phải phép tối ưu đại số luôn giữ nguyên kết quả.

Activation tensor là dữ liệu trung gian, chẳng hạn Q , đầu ra MLP hoặc đầu ra LayerNorm; tensor này không nhất thiết vừa đi qua một activation function. Phân biệt hai nghĩa giúp xác định đúng nơi đặt observer lượng tử hóa: scale của đầu ra linear trước GELU khác scale của đầu ra sau GELU.

1.4 Loại bỏ Dropout và DropPath khi suy luận

Với $x \in \mathbb{R}^{T \times D}$, mặt nạ $m \in \{0, 1\}^{T \times D}$ và xác suất loại bỏ $p \in [0, 1)$, biểu thức ở chế độ huấn luyện là:

$$y = \frac{m \odot x}{1 - p}, \quad m_i \sim \text{Bernoulli}(1 - p). \quad (1.24)$$

Ở chế độ đánh giá, Dropout của PyTorch là ánh xạ đồng nhất [12]:

$$y_{\text{eval}} = x. \quad (1.25)$$

Không cần sửa thuộc tính p thành không; phải chuyển mô hình sang `eval()` và kiểm tra đồ thị xuất. Với DropPath, thiết kế giả định cách triển khai cũng trả về nhánh nguyên vẹn ở chế độ đánh giá; cần xác nhận điều này cho lớp cụ thể của mô hình nguồn.

Biến đổi và điều kiện loại bỏ RNG. Tại $p = 0$, mặt nạ suy biến thành $m_{ij} = 1$ với xác suất một, nên

$$y_{ij} = \frac{m_{ij}x_{ij}}{1 - p} = \frac{1 \cdot x_{ij}}{1 - 0} = x_{ij}. \quad (1.26)$$

Với ngữ nghĩa eval đồng nhất, kết quả không phụ thuộc trạng thái RNG. Có thể bỏ một LFSR *chỉ nếu* mọi consumer của nó cũng đã bị bỏ hoặc đầu ra của nó không còn quan sát được. Phép chứng minh này không khẳng định graph gốc có LFSR, không cho phép xóa RNG được dùng ở nơi khác và không chứng minh số LUT/FF sau synthesis.

Bất biến giao dịch cục bộ. Xét wrapper dây nối: valid và payload đầu ra bằng đầu vào, ready đầu vào bằng ready đầu ra. Tại mỗi cạnh $k \in \mathbb{Z}_{\geq 0}$ không reset, hai phía có cùng điều kiện $\text{fire}[k] = \text{valid}[k] \wedge \text{ready}[k]$, nên số giao dịch nhận và phát bằng nhau, payload và thứ tự được bảo toàn. Nếu nguồn giữ payload khi stall, đích cũng nhận payload ổn định khi stall. Với nguồn valid liên tục, đích ready liên tục và timing đạt, fire bằng một mỗi cạnh: wrapper không thêm giới hạn khởi phát ngoài một beat/chu kỳ. Không có suy diễn từ mệnh đề cục bộ này sang $II = 1$ hoặc WNS/WHS không âm của toàn accelerator.

Sau khi cố định chế độ suy luận, phép đồng nhất có thể được loại khỏi đồ thị RTL. Khi đó không cần tạo mặt nạ ngẫu nhiên cho phép toán đã bỏ. Tuy nhiên, không thể suy ra rằng toàn bộ accelerator tiết kiệm một tỷ lệ LUT/FF cố định hay tự đạt $II = 1$. Khoảng cách khởi phát còn phụ thuộc bộ chia, băng thông, lịch truy cập bộ nhớ và giao thức điều khiển. Suy luận có chủ ý dùng Monte Carlo dropout nằm ngoài giả định của báo cáo này.

1.5 Ba nút thắt phần cứng trên Edge FPGA

1.5.1 Chi phí GEMM: phân biệt thành phần tuyến tính và bậc hai

Đếm phép toán theo phần tử. Với $Q_a, K_a \in \mathbb{R}^{T \times d_k}$:

$$(Q_a K_a^T)_{ij} = \sum_{r=1}^{d_k} Q_{a,ir} K_{a,jr}. \quad (1.27)$$

Có T^2 cặp (i, j) ; mỗi cặp có d_k phép nhân và $d_k - 1$ phép cộng nếu cộng các tích mà không tính phép cộng vào accumulator zero. Do đó:

$$C_{\text{mul}} = T^2 d_k, \quad C_{\text{add}} = T^2 (d_k - 1), \quad (1.28)$$

$$C_{\text{scalar}} = T^2 (2d_k - 1), \quad C_{2/\text{MAC}} = 2T^2 d_k. \quad (1.29)$$

Cột cuối dùng quy ước hai operation cho mỗi MAC, không phải số phép cộng tối thiểu. $P_a V_a$ có $T d_k$ đầu ra với T tích mỗi đầu ra, tức $T^2 d_k$ MAC. Tổng hai GEMM trên h head là $2hT^2 d_k = 2T^2 D$ MAC. Giữ D cố định, tỷ lệ chi phí này khi T tăng gấp đôi là $(2T)^2 / T^2 = 4$; đây là thành phần bậc hai.

Bỏ qua bias, chuẩn hóa, embedding đầu vào và classifier, số MAC cho một encoder block là kết quả đếm trực tiếp từ kích thước các ma trận ở trên:

$$C_{\text{QKV+O}} = 4TD^2, \quad (1.30)$$

$$C_{\text{MLP}} = 8TD^2, \quad (1.31)$$

$$C_{\text{attention}} = 2T^2 D, \quad (1.32)$$

$$C_{\text{block}} = 12TD^2 + 2T^2 D. \quad (1.33)$$

Ba projection QKV cần $3(TD)D = 3TD^2$ MAC; projection đầu ra cần $(TD)D = TD^2$. Hai GEMM MLP lần lượt cần $(T \cdot 4D)D = 4TD^2$ và $(TD)4D = 4TD^2$. Cộng các hạng tạo công thức (1.33). Vì $2TD > 0$, điều kiện attention lớn hơn linear được biến đổi thành $2T^2 D > 12TD^2 \iff T > 6D$.

Vì vậy, $\mathcal{O}(T^2)$ mô tả thành phần attention khi giữ D cố định, không phải khẳng định attention luôn chiếm phần lớn phép tính. Nếu quy ước một MAC bằng hai phép toán, số operation bằng hai lần số MAC; báo cáo GOPS phải công bố quy ước này.

Ví dụ phân tích với $T = 197$, $D = 768$, $h = 12$: phần linear cần 1 394 343 936 MAC, còn hai phép nhân attention cần 59 610 624 MAC trong mỗi block. Đây là số đếm lý thuyết, không phải kết quả benchmark trên KV260. Attention vượt phần linear theo mô hình đếm này khi $T > 6D$.

1.5.2 Memory bandwidth wall

Nếu lưu toàn bộ score của mọi head, số phần tử là hT^2 . Với ví dụ trên, một bản score 8-bit cần 465 708 byte; bản 32-bit cần gấp bốn. Đây chỉ là dung lượng score, chưa bao gồm Q, K, V , buffer residual và trọng số. Tiling có thể giảm lượng dữ liệu phải lưu đồng thời nhưng cần phân tích lại lưu lượng đọc/ghi.

Điều kiện của nút thắt bộ nhớ. Đặt $b_s \in \mathbb{Z}_{>0}$ là số bit lưu mỗi score và $B_{\text{on}} \in \mathbb{Z}_{\geq 0}$ là dung lượng byte dành riêng cho score. Với packing bit liên tục, một bản score cần

$$B_{\text{score}} = \left\lceil \frac{hT^2 b_s}{8} \right\rceil. \quad (1.34)$$

Padding/alignment vật lý có thể cần thêm byte. Nếu $B_{\text{score}} > B_{\text{on}}$, toàn bộ score không thể đồng thời nằm trong vùng nhớ này. Nếu thêm giả thiết lịch thực thì ghi một bản score ra DDR rồi đọc lại đúng một lần, lưu lượng riêng phần này ít nhất $2B_{\text{score}}$. Các lịch tiled/fused không materialize toàn score không chịu giả thiết đó; không được áp dụng lưu lượng này cho chúng.

Đặt B_{ext} là tổng số byte thực sự truyền qua bộ nhớ ngoài cho một lần suy luận, BW_{eff} là băng thông hữu dụng và P_{MAC} là thông lượng tính toán hữu dụng. Miền các đại lượng

là $B_{\text{ext}}, C_{\text{MAC}} \in \mathbb{Z}_{\geq 0}$, $BW_{\text{eff}} > 0$ byte/s, $P_{\text{MAC}} > 0$ MAC/s và các thời gian không âm đo bằng giây. Các cận dưới thời gian là:

$$t_{\text{mem}} \geq \frac{B_{\text{ext}}}{BW_{\text{eff}}}, \quad t_{\text{compute}} \geq \frac{C_{\text{MAC}}}{P_{\text{MAC}}}, \quad (1.35)$$

$$t_{\text{inference}} \geq \max(t_{\text{mem}}, t_{\text{compute}}). \quad (1.36)$$

Từ $BW_{\text{eff}} t_{\text{mem}} \geq B_{\text{ext}}$ và $P_{\text{MAC}} t_{\text{compute}} \geq C_{\text{MAC}}$, chia cho các thông lượng dương cho hai cận đầu. Một lịch hoàn tất phải chứa cả hoạt động truyền và tính, nên thời gian tổng không nhỏ hơn từng thời gian thành phần, do đó không nhỏ hơn maximum. Cận cuối chỉ là điều kiện cần; phụ thuộc dữ liệu, thời gian khởi động DMA và các pha không chồng lấp có thể làm thời gian thực lớn hơn. Nạp lại cùng trọng số cho nhiều tile làm tăng B_{ext} , ngay cả khi tổng kích thước tệp trọng số không đổi.

1.5.3 Nút thắt phi tuyến và phạm vi Batch 1

Softmax cần reduction theo hàng và chuẩn hóa; GELU cần xấp xỉ phi tuyến theo phần tử; LayerNorm cần thống kê theo chiều đặc trưng. Số MAC nhỏ không bảo đảm độ trễ nhỏ nếu khối phi tuyến có phép chia nối tiếp hoặc buộc chờ đủ một vector.

Chương 2 thiết lập quy định biểu diễn và tính toán cho các khối này. Phụ lục A cung cấp hai module xử lý phần tử, không phải toàn bộ attention core; Phụ lục B cung cấp observer và công cụ lượng tử hóa. Các kết luận về tốc độ hay độ chính xác toàn mạng phải đợi giai đoạn tích hợp và đánh giá phần cứng tiếp theo.

Tự kiểm tra và đối chiếu

1. Với $T = 8, d_k = 2$, bảng điểm có bao nhiêu phần tử và cần bao nhiêu phép nhân? Hãy ghi riêng phép nhân và phép cộng.
2. Vì sao một hàng score bằng zero không phải một hàng trọng số attention bằng zero?
3. Có thể suy ra FPS trên KV260 chỉ từ số MAC của mô hình không?

Đối chiếu. Có 64 phần tử, 128 phép nhân và 64 phép cộng theo quy ước cộng hai tích. Softmax cho $e^0/(4e^0) = 1/4$ trong ví dụ bốn token. Không suy được FPS vì còn lịch thực thi, bộ nhớ, phần phi tuyến và phép đo cần thiết.

Tóm tắt và chuyển sang số học hữu hạn

Ta đã xác định tensor và các phép toán cần thực hiện. Bước tiếp theo không phải chép trực tiếp ma trận số thực thành RTL, mà chọn cách biểu diễn hữu hạn: cùng mã 35 có thể biểu diễn các giá trị thực khác nhau nếu scale khác nhau. Chương 2 sẽ chọn scale, quy tắc làm tròn và độ rộng để biến bài toán đại số thành các quy tắc tính toán số nguyên có thể kiểm tra.

Chương 2

Lượng tử hóa INT8 và xấp xỉ thuật toán phi tuyến

Mục tiêu học tập

- Phân biệt mã INT8 với giá trị thực được tái dựng bằng scale.
- Tính tay một ví dụ lượng tử hóa, ShiftGELU và căn nguyên rồi đối chiếu reference.
- Chỉ ra vị trí làm tròn, saturation và điều kiện ngăn overflow trước khi chọn RTL.

Trực giác trước công thức. Scale giống vạch chia của một thước: mã nguyên đếm số vạch, còn giá trị thực phụ thuộc độ dài mỗi vạch. Thước hữu hạn nên có hai loại sai lệch khác nhau: làm tròn về vạch gần nhất và kẹp khi vượt đầu thước. Vì vậy giảm số bit không chỉ là đổi kiểu dữ liệu; phải quyết định vạch chia, miền đo và quy tắc làm tròn.

Pre-check và trạng thái chứng minh. Các đẳng thức sau dùng số nguyên không tràn hoặc số thực chính xác; mỗi phép xấp xỉ có miền và quy tắc làm tròn riêng. Bốn dịch 0, 1, 3, 4 không tạo hệ số 1.703125; mười vòng Newton không mặc nhiên là mười clock; kiểu INT8 không tự xác định scale. Các mệnh đề số học được chứng minh dưới giả thiết ghi rõ, còn chứng thư Lean, mô phỏng HDL và kết quả synthesis/STA chưa có. Không coi việc chạy một assertion trên một tập mẫu là chứng minh phổ quát ngoài tập đó.

2.1 Mô hình lượng tử hóa uniform symmetric INT8

2.1.1 Scale, zero-point và miền biểu diễn

Báo cáo dùng lượng tử hóa đều với zero-point bằng không. Với các kích thước $n_1, \dots, n_d \in \mathbb{Z}_{>0}$, $d \in \mathbb{Z}_{>0}$, tensor $X \in \mathbb{R}^{n_1 \times \dots \times n_d}$ và $S_X \in \mathbb{R}_{>0}$, mọi phép tính sau tác động theo phần tử. Mã I_X thuộc tensor trên tập $\text{INT8} = \{z \in \mathbb{Z} : -128 \leq z \leq 127\}$. Với $u \in \mathbb{R}$, đặt:

$$I_X = \text{sat}_8(\text{RNE}(X/S_X)), \quad \hat{X} = S_X I_X, \quad (2.1)$$

$$\text{sat}_8(u) = \min(127, \max(-128, u)). \quad (2.2)$$

RNE là làm tròn đến số nguyên gần nhất, trường hợp đúng giữa hai số thì chọn số chẵn. Mã lưu trữ bù hai có miền $[-128, 127]$, không đối xứng tuyệt đối. Với calibration cực đại tuyệt đối a_X và $S_X = a_X/127$, hai cực trị calibration thường ánh xạ về $[-127, 127]$. Nếu yêu cầu tập mã đối xứng nghiêm ngặt, phải kẹp về $[-127, 127]$ trong cả phần mềm lẫn phần cứng; không được trộn hai quy ước.

Trong miền không clipping, sai số lượng tử hóa theo quy tắc trên bị chặn bởi:

$$|X - \hat{X}| \leq \frac{S_X}{2}. \quad (2.3)$$

Để chứng minh, đặt $v = X/S_X$ cho một phần tử và $q = \text{RNE}(v)$. Tiền điều kiện không clipping là $q \in \text{INT8}$. Theo định nghĩa chọn số nguyên gần nhất, $q - 1/2 \leq v \leq q + 1/2$, do đó

$$|X - \hat{X}| = |S_X v - S_X q| = S_X |v - q| \leq S_X/2. \quad (2.4)$$

Đây là cận trên số thực chính xác. Khi script chuyển input/scale sang FP32 rồi chia, cần cộng sai số chuyển kiểu và chia; cận này không tự trở thành chứng nhận bit-exact cho tính toán FP32. Ngoài miền biểu diễn, clipping có thể gây sai số lớn hơn. Scale của activation nên được đo trên nhiều mẫu đại diện rồi đóng băng cho suy luận. Scale động theo từng frame là một giao thức khác: phần cứng phải nhận đúng hệ số tương ứng với frame đó.

Với tensor toàn không, Phụ lục B.1 dùng một scale dương dự phòng. Tensor rỗng, NaN, vô cực và scale không hợp lệ bị từ chối thay vì âm thầm tạo vector không xác định. Script này là công cụ thu thập/calibration, chưa phải quy trình QAT hoàn chỉnh.

2.1.2 INT8 không đồng nghĩa với một định dạng fixed-point cố định

Với số dấu phẩy cố định (Fixed-Point), quy ước $Q_{m.n}$ của báo cáo là: m bao gồm bit dấu và $m + n$ là tổng số bit. Ví dụ $Q_{4.4}$ có tám bit, bốn bit phần lẻ:

$$S = 2^{-4} = 1/16, \quad x = I_x/16, \quad -8 \leq x \leq 7.9375. \quad (2.5)$$

INT8 với scale calibration bất kỳ không nhất thiết là $Q_{4.4}$. Module ShiftGELU trong Batch 1 cố định cả scale vào và ra bằng $1/16$. Trước khi dùng module, phải chuyển tensor về đúng scale và định dạng này và đánh giá clipping; không thể truyền nguyên mã từ observer có scale khác.

2.1.3 Ví dụ: cùng mã nguyên, khác scale

Cho $x = 5/32 = 0.15625 \in \mathbb{R}$ và $S = 1/16$. Chia cho scale cho $x/S = 5/2 = 2.5$. Hai số gần nhất là 2 và 3; quy tắc RNE chọn số chẵn 2, vì vậy:

$$I_x = 2, \quad \hat{x} = 2/16 = 0.125, \quad |x - \hat{x}| = 1/32 = S/2. \quad (2.6)$$

Mã 2 với scale $1/8$ lại biểu diễn $1/4$, không phải $1/8$. Với đầu vào $x = 9$ và scale $1/16$, mã trước kẹp là 144, sau kẹp là 127; sai số $9 - 127/16 = 17/16$ lớn hơn $S/2$. Cận làm tròn không được dùng cho trường hợp clipping này.

2.2 Dyadic rescaling và độ rộng trung gian

Đổi scale là đổi đơn vị của cùng một đại lượng, giống đổi số đo từ vạch lớn sang vạch nhỏ. Hệ số dyadic chọn một số nguyên và một số bit dịch để gần đúng tỷ lệ đơn vị. Nó bỏ nhu cầu nhân số thực trong bước này, nhưng vẫn cần giữ đủ độ rộng tích và nêu rõ cách làm tròn.

2.2.1 Tích lũy ma trận và bias

Tiền điều kiện GEMM: $M, N, K_{\text{red}} \in \mathbb{Z}_{>0}$, $I_X \in \text{INT8}^{M \times K_{\text{red}}}$, $I_W \in \text{INT8}^{K_{\text{red}} \times N}$, $b \in \mathbb{R}^N$, $S_X, S_{W,j}, S_{Y,j} > 0$, $1 \leq i \leq M$, $1 \leq j \leq N$. Bias nguyên $I_{b,j} \in \mathbb{Z}$ phải được kiểm tra miền trước khi nạp. Với chiều reduction K_{red} , định nghĩa tích lũy nguyên:

$$A_{ij} = \sum_{r=1}^{K_{\text{red}}} I_{X,ir} I_{W,rj} + I_{b,j}, \quad (2.7)$$

$$I_{b,j} = \text{RNE}\left(\frac{b_j}{S_X S_{W,j}}\right), \quad (2.8)$$

$$\hat{Y}_{ij} \approx S_X S_{W,j} A_{ij}. \quad (2.9)$$

Scale trọng số có thể theo từng kênh đầu ra j , nhưng khi đó bộ requantization cũng phải chọn đúng hệ số cho kênh. Đây là sự mở rộng so với observer per-tensor của Batch 1.

Một tích INT8 cần 16 bit có dấu. Vì tích dương lớn nhất là $(-128)(-128) = 16384$, điều kiện bảo thủ để tích lũy không bias nằm trong INT32 là:

$$K_{\text{red}} 16384 \leq 2^{31} - 1. \quad (2.10)$$

Miền tích chính xác là $[-16256, 16384]$: các cực trị đạt tại $(-128)127$ và $(-128)(-128)$. Với mọi tiền tố $k \in \{0, \dots, K_{\text{red}}\}$, bất đẳng thức tam giác cho $|\sum_{r=1}^k I_{X,ir} I_{W,rj} + I_{b,j}| \leq 16384k + |I_{b,j}|$. Vì vậy $16384K_{\text{red}} + |I_{b,j}| \leq 2^{31} - 1$ bảo đảm mọi tiền tố có bias đều nằm trong INT32. Không bias, điều kiện này cho $K_{\text{red}} \leq 131071$. Bias và các giá trị trung gian khác làm giảm phần headroom này. “Tích lũy INT32” phải là quyết định sau khi chặn miền giá trị, không phải giả định an toàn cho mọi kích thước.

2.2.2 Biểu diễn hệ số bằng số nguyên và phép dịch

Biến đổi từ giá trị tái dựng. Đặt $\hat{X}_{ir} = S_X I_{X,ir}$, $\hat{W}_{rj} = S_{W,j} I_{W,rj}$, $\hat{b}_j = S_X S_{W,j} I_{b,j}$. Khi các scale trọng số không thay theo chiều reduction:

$$Y_{ij}^{\text{rec}} = \sum_{r=1}^{K_{\text{red}}} (S_X I_{X,ir})(S_{W,j} I_{W,rj}) + \hat{b}_j \quad (2.11)$$

$$= S_X S_{W,j} \left(\sum_{r=1}^{K_{\text{red}}} I_{X,ir} I_{W,rj} + I_{b,j} \right) \quad (2.12)$$

$$= S_X S_{W,j} A_{ij}, \quad (2.13)$$

$$Y_{ij}^{\text{rec}} / S_{Y,j} = (S_X S_{W,j} / S_{Y,j}) A_{ij}. \quad (2.14)$$

Đây là đẳng thức cho giá trị tái dựng, không phải khẳng định tensor FP32 gốc bằng tensor lượng tử hóa.

Chọn hệ số dương $\alpha_j = S_X S_{W,j} / S_{Y,j}$ và số nguyên:

$$\alpha_j \approx M_j 2^{-e_j}, \quad M_j = \text{RNE}(\alpha_j 2^{e_j}), \quad (2.15)$$

$$0 \leq M_j \leq 2^{31} - 1, \quad 0 \leq e_j \leq 31. \quad (2.16)$$

Miền số mũ trên khớp với cổng năm bit trong module residual. Nếu không tìm được cặp đủ chính xác trong miền này, phải đổi scale hoặc mở rộng giao diện. Với e_j đã chọn và không tràn M_j :

$$|\alpha_j - M_j 2^{-e_j}| \leq 2^{-e_j-1}. \quad (2.17)$$

Đặt $\xi_j = \alpha_j 2^{e_j} - M_j$. RNE cho $|\xi_j| \leq 1/2$, nên $\alpha_j - M_j 2^{-e_j} = 2^{-e_j} \xi_j$ và cận trên theo sau bằng phép nhân với $2^{-e_j} > 0$. Cho phép $M_j = 0$ vì một hệ số nhỏ có thể làm tròn về không; nếu mô hình không cho phép triệt tiêu nhánh này, phải thêm tiền điều kiện $M_j \geq 1$.

Dyadic arithmetic loại bỏ hệ số số thực khỏi datapath suy luận, là cơ sở cho các phép toán tuyến tính trong I-ViT [3]. Tuy nhiên, phép nhân INT32 với hệ số INT32 nói chung cần tích 64 bit trước khi dịch; gọi cả khối này là “một phép nhân 32-bit” không có nghĩa đầu ra cũng chỉ cần 32 bit.

2.2.3 Quy tắc làm tròn phải là một phần giao diện

Để mã RTL đơn giản và xác định hoàn toàn, Batch 1 chọn dịch phải đại số với làm tròn xuống:

$$\mathcal{F}_e(u) = \lfloor u/2^e \rfloor, \quad (2.18)$$

$$I_Y = \text{sat}_8(\mathcal{F}_e(MA)). \quad (2.19)$$

Bổ đề dịch có dấu. Với $u \in \mathbb{Z}$, $e \in \mathbb{Z}_{\geq 0}$, phép chia Euclid cho duy nhất $q, r \in \mathbb{Z}$ sao cho $u = 2^e q + r$, $0 \leq r < 2^e$. Chia hai vế cho 2^e cho $u/2^e = q + r/2^e$ với phần lẻ trong $[0, 1)$; do đó $q = \lfloor u/2^e \rfloor$. Bỏ e bit thấp và mở rộng dấu biểu diễn đúng thương này khi toán hạng đã nằm trong miền signed của thanh ghi; không được thu hẹp tích trước khi dịch.

Ví dụ $\mathcal{F}_1(-3) = -2$, khác phép chia nguyên cắt về không và khác RNE trong một số trường hợp. Cổng dữ liệu phải có kiểu **signed** để toán tử $\gg$ mở rộng bit dấu. Làm tròn xuống có độ lệch âm; muốn RNE phải thay cả RTL và golden model. Trước clipping, cận sai số cho phép rescale này là:

$$|\alpha A - \mathcal{F}_e(MA)| < |A| |\alpha - M2^{-e}| + 1. \quad (2.20)$$

Với $\theta = MA/2^e - \lfloor MA/2^e \rfloor \in [0, 1)$:

$$\alpha A - \mathcal{F}_e(MA) = A(\alpha - M2^{-e}) + \theta, \quad (2.21)$$

$$|\alpha A - \mathcal{F}_e(MA)| \leq |A| |\alpha - M2^{-e}| + \theta < |A| |\alpha - M2^{-e}| + 1. \quad (2.22)$$

Cận này chỉ đo rescale trên accumulator đã cố định, chưa cộng sai số lượng tử hóa toán hạng/bias. Clipping phải được tính riêng khi so với giá trị lý tưởng ngoài miền INT8.

2.2.4 Ví dụ: dịch phải số âm không phải cắt về zero

Với accumulator $A = -7$, hệ số $M = 3$ và số mũ $e = 2$, giữ tích chính xác $AM = -21$ trong kiểu signed đủ rộng. Quy tắc của datapath cho:

$$\left\lfloor \frac{AM}{2^e} \right\rfloor = \lfloor -21/4 \rfloor = \lfloor -5.25 \rfloor = -6. \quad (2.23)$$

Cắt về zero sẽ cho -5 , sai một mã so với quy tắc làm tròn đã chọn. Đây là lý do golden và RTL phải cùng quy tắc, ngay cả khi hai cách đều được gọi chung là “chia cho bốn”.

2.3 ShiftGELU: xấp xỉ số nguyên có quy định rõ ràng về scale

Có thể hình dung GELU như một van điều chỉnh mềm: giá trị dương đủ lớn gần được giữ nguyên, còn giá trị âm có độ lớn đủ lớn bị giảm về gần zero. Ẩn dụ này không nói mọi đầu vào âm đều bị xóa. Bài toán phần cứng là tính hệ số điều chỉnh hữu hạn bằng những phép nguyên có miền rõ ràng; các bước dịch để tạo hệ số chỉ là một phần của toàn khối [11, 3].

2.3.1 Ví dụ xuyên suốt: mã 35 đi qua năm tầng

Tiền điều kiện: $I_x = 35 \in \text{INT8}$, $S_x = S_y = 1/16$, nên $x = 35/16 = 2.1875$; nguồn đánh dấu mẫu hợp lệ, bên nhận không stall và không reset trong cửa sổ theo dõi. Các phép chia dưới đây lấy sàn, đúng reference B.3, không làm tròn gần nhất.

Stage 0: tạo mã của đối số sigmoid.

$$p = 35 + \lfloor 35/2 \rfloor + \lfloor 35/8 \rfloor + \lfloor 35/16 \rfloor \quad (2.24)$$

$$= 35 + 17 + 4 + 2 = 58. \quad (2.25)$$

Stage 1: tính ShiftExp cho đối số không dương.

$$\delta = -|58| = -58, \quad (2.26)$$

$$J = -58 + \lfloor -58/2 \rfloor - \lfloor -58/16 \rfloor = -58 - 29 + 4 = -83, \quad (2.27)$$

$$q = \lfloor 83/16 \rfloor = 5, \quad r = -83 + 16 \cdot 5 = -3, \quad (2.28)$$

$$E = \left\lfloor \frac{4096 + 128(-3)}{2^5} \right\rfloor = \lfloor 3712/32 \rfloor = 116. \quad (2.29)$$

Đặc biệt, $\lfloor -58/16 \rfloor = -4$, không phải -3 . Giá trị E thỏa miền unsigned 13-bit đã chọn.

Stage 2: hệ số sigmoid nguyên. Vì $p \geq 0$, chọn $U = 4096$, $V = 4096 + 116 = 4212$:

$$G = \left\lfloor \frac{256 \cdot 4096}{4212} \right\rfloor = 248, \quad 1048576 = 4212 \cdot 248 + 4000, \quad 0 \leq 4000 < 4212. \quad (2.30)$$

G là mã của hệ số $G/256$, chưa phải output activation.

Stage 3–4: nhân, đổi scale và kẹp.

$$P = 35 \cdot 248 = 8680, \quad (2.31)$$

$$I_y = \text{sat}_8(\lfloor 8680/256 \rfloor) = \text{sat}_8(33) = 33, \quad (2.32)$$

$$8680 = 33 \cdot 256 + 232, \quad 0 \leq 232 < 256, \quad (2.33)$$

$$\hat{y} = 33/16 = 2.0625. \quad (2.34)$$

Tích 8680 nằm trong signed 18-bit; kết quả 33 nằm trong INT8 nên saturation không đổi giá trị. $\hat{y}$ là kết quả của xấp xỉ đã chọn, không phải khẳng định bằng GELU toán học chính xác.

Sau cạnh	Thanh ghi theo mẫu đang xét	Valid đi kèm
k	$p = 58$, mẫu gốc 35	$v_0 = 1$
$k + 1$	$E = 116$, dấu không âm, mẫu 35	$v_1 = 1$
$k + 2$	$G = 248$, mẫu 35	$v_2 = 1$
$k + 3$	$P = 8680$	$v_3 = 1$
$k + 4$	$I_y = 33$	$v_4 = \text{out_valid} = 1$

Bảng theo dõi một mẫu, không nói các tầng còn lại đều rỗng. Mẫu khác có thể cùng nằm trong pipeline; bubble phải tiến cùng offset và không làm mẫu 35 nhảy sang một lịch khác. Đây là năm tầng chốt và bốn khoảng clock kể từ cạnh nhận.

2.3.2 Từ GELU đến sigmoid và shift-add

GELU chính xác là $x\Phi(x)$; dạng sigmoid chỉ là xấp xỉ [11]:

$$\text{GELU}(x) \approx x\sigma(1.702x). \quad (2.35)$$

Hệ số nhị phân được sử dụng trong ShiftGELU [3] là:

$$1 + 2^{-1} + 2^{-3} + 2^{-4} = 1.6875 \neq 1.702, \quad (2.36)$$

$$p = I_x + \mathcal{F}_1(I_x) + \mathcal{F}_3(I_x) + \mathcal{F}_4(I_x). \quad (2.37)$$

Các phép dịch riêng biệt còn tạo sai số làm tròn, đặc biệt ở số âm. Vì vậy không thay biểu thức thứ hai bằng phép nhân $1.6875I_x$ rồi làm tròn một lần trong golden model. Với INT8, $-216 \leq p \leq 212$; phải mở rộng dấu trước khi cộng.

2.3.3 Chứng minh hệ số và sai số do từng phép dịch

Đặt $z = I_x \in \text{INT8}$, $a = 851/500 = 1.702$, $b = 27/16$. Quy đồng mẫu số cho:

$$b = \frac{16 + 8 + 2 + 1}{16} = \frac{27}{16} = 1.6875, \quad (2.38)$$

$$|a - b| = \left| \frac{851 \cdot 16 - 27 \cdot 500}{8000} \right| = \frac{29}{2000} = 0.0145, \quad (2.39)$$

$$100 \frac{|a - b|}{a} = \frac{725}{851} \% \approx 0.85194\%. \quad (2.40)$$

Hệ số $109/64 = 1.703125$ cần thêm hạng 2^{-6} , không tồn tại trong RTL hiện tại. Sai số tương đối của hệ số đó là $100(9/8000)/(851/500)\% \approx 0.06610\%$, cũng không nhỏ hơn 0.066% .

Với $k \in \{1, 3, 4\}$, đặt $r_k = z - 2^k \lfloor z/2^k \rfloor$, nên $0 \leq r_k \leq 2^k - 1$. Thay từng thương vào tổng shift-add:

$$p = z + \frac{z - r_1}{2} + \frac{z - r_3}{8} + \frac{z - r_4}{16} \quad (2.41)$$

$$= \frac{27}{16}z - \rho(z), \quad \rho(z) = \frac{r_1}{2} + \frac{r_3}{8} + \frac{r_4}{16}, \quad (2.42)$$

$$0 \leq \rho(z) \leq \frac{1}{2} + \frac{7}{8} + \frac{15}{16} = \frac{37}{16}, \quad (2.43)$$

$$p - az = -\frac{29}{2000}z - \rho(z), \quad (2.44)$$

$$|p - az| \leq \frac{29}{2000}|z| + \frac{37}{16} \leq 4.1685. \quad (2.45)$$

Cận cuối tính bằng đơn vị mã trước sigmoid; ở miền vật lý $Q_{4.4}$ phải chia cho 16. Phản ví dụ $z = 127$ cho $p = 127 + 63 + 15 + 7 = 212$, còn $az = 216.154$: sai lệch 4.154 mã lớn hơn nửa LSB. Do đó không có kết luận “sai số nằm trong làm tròn INT8”. Mỗi hạng trong $p(z)$ không giảm theo z , nên $p(-128) = -128 - 64 - 16 - 8 = -216$ và $p(127) = 212$ chặn toàn miền. Tổng từng bước cũng nằm trong signed 16-bit.

2.3.4 ShiftExp trên miền không dương

Bản RTL ở Phụ lục A.1 là một *biến thể fixed-point theo nguyên lý ShiftGELU*, không phải bản sao bit-exact của cấu hình PyTorch/TVM trong I-ViT. Nó chọn scale cố định, thêm độ chính xác phần lẻ cho hàm mũ và dùng chia nguyên trực tiếp thay phép nhân nghịch đảo của cấu hình tham khảo. Không được chuyển kết quả độ chính xác của bài báo sang module này.

Dùng đẳng thức sigmoid ổn định theo dấu, chỉ cần một hàm mũ có đối số không dương:

$$\delta = -|p|, \quad u \approx e^{\delta/16}, \quad (2.46)$$

$$\sigma(p/16) = \begin{cases} 1/(1+u), & p \geq 0, \\ u/(1+u), & p < 0. \end{cases} \quad (2.47)$$

Đổi cơ số bằng xấp xỉ $\log_2 e \approx 1 + 2^{-1} - 2^{-4} = 1.4375$, rồi tách phần nguyên:

$$J = \delta + \mathcal{F}_1(\delta) - \mathcal{F}_4(\delta), \quad (2.48)$$

$$q = \left\lfloor \frac{-J}{16} \right\rfloor, \quad r = J + 16q, \quad -15 \leq r \leq 0, \quad (2.49)$$

$$2^{J/16} = 2^{-q} 2^{r/16} \approx 2^{-q} (1 + r/32). \quad (2.50)$$

Chọn scale hàm mũ 2^{-12} , định nghĩa phiên bản ShiftExp dùng trong báo cáo:

$$E = \left\lfloor \frac{4096 + 128r}{2^q} \right\rfloor, \quad 0 \leq E \leq 4096. \quad (2.51)$$

Chứng minh miền trung gian. $\delta \in [-216, 0] \cap \mathbb{Z}$. Khi δ tăng một, J tăng $1 + \Delta[\delta/2] - \Delta[\delta/16]$; nếu hạng cuối tăng một thì hạng chia hai cũng tăng một. Vì vậy gia số thuộc $\{1, 2\}$, cho $J(-216) = -216 - 108 + 14 = -310 \leq J \leq J(0) = 0$. Viết $-J = 16q + v$, $0 \leq v \leq 15$; suy ra $0 \leq q \leq 19$ và $r = -v \in [-15, 0]$. Tiếp theo:

$$2176 = 4096 - 128 \cdot 15 \leq 4096 + 128r \leq 4096, \quad 0 \leq E \leq 4096. \quad (2.52)$$

Do $-J < 512$, lấy bit $[8 : 4]$ của độ lớn trong RTL trả đúng q . Mã E cần 13 bit unsigned; signed 16-bit đủ cho δ, J, r , còn phép dịch tạo $128r$ được mở rộng lên signed 32-bit trước khi thực hiện.

Sai số xấp xỉ hàm mũ. Đặt $t = r/16 \in [-15/16, 0]$ và $f(t) = 1 + t/2 - 2^t$. Vì $f''(t) = -(\ln 2)^2 2^t < 0$, $f(-1) = f(0) = 0$, nên $f(t) \geq 0$ trên $[-1, 0]$. Đạo hàm bằng không tại $t_* = -\log_2(2 \ln 2) \in [-1, 0]$, là điểm cực đại duy nhất; đặt

$$c_{\text{exp}} = f(t_*) = 1 - \frac{1}{2} \log_2(2 \ln 2) - \frac{1}{2 \ln 2}. \quad (2.53)$$

Như vậy $0 \leq (1 + r/32) - 2^{r/16} \leq c_{\text{exp}}$. Floor sau scale Q12 tạo sai số nhỏ hơn 2^{-12} , nên

$$|E/4096 - 2^{J/16}| < 2^{-q} c_{\text{exp}} + 2^{-12}. \quad (2.54)$$

Viết $[\delta/2] = \delta/2 - \eta_1$ và $[\delta/16] = \delta/16 - \eta_4$, với $0 \leq \eta_1 \leq 1/2$, $0 \leq \eta_4 \leq 15/16$. Khi đó $J = (23/16)\delta - \eta_1 + \eta_4$, cho

$$|J - (\log_2 e)\delta| \leq |23/16 - \log_2 e| 216 + 15/16. \quad (2.55)$$

Hai số mũ $J/16$ và $(\log_2 e)\delta/16$ đều không dương. Đạo hàm $2^t \ln 2 \leq \ln 2$ trên miền đó và định lý giá trị trung bình cho:

$$|E/4096 - e^{\delta/16}| < 2^{-q} c_{\text{exp}} + 2^{-12} \quad (2.56)$$

$$+ \frac{\ln 2}{16} (|23/16 - \log_2 e| 216 + 15/16). \quad (2.57)$$

Đây là cận cho ShiftExp trên miền đã chốt; không phải sai số tổng GELU hoặc độ chính xác phân loại.

Mã giữ 128r trước khi dịch để tránh mất phần lẻ quá sớm. Khi đầu vào rất âm, underflow của E về không là một quyết định fixed-point đã xác định, không phải tràn số. Không dùng hằng cực đại 216 làm điểm dịch cho mọi mẫu: điểm dịch của sigmoid phải bảo toàn tỷ số của cả tử và mẫu.

2.3.5 Sigmoid chín bit và tích đầu ra

Đặt tử số U và mẫu số V như sau:

$$U = \begin{cases} 4096, & p \geq 0, \\ E, & p < 0, \end{cases} \quad V = 4096 + E, \quad (2.58)$$

$$G = \left\lfloor \frac{256U}{V} \right\rfloor, \quad 0 \leq G \leq 256, \quad (2.59)$$

$$I_y = \text{sats}(\mathcal{F}_8(I_x G)), \quad S_y = S_x = 1/16. \quad (2.60)$$

G cần *chín bit không dấu* để biểu diễn chính xác hệ số một bằng mã 256. Ép 256 vào tám bit sẽ làm nó thành không; kẹp về 255 lại gây suy hao có hệ thống ở miền dương lớn. Tích $I_x G$ giữ trong 18 bit có dấu trước khi requantize.

Bộ chia trong mã dùng chín bước so sánh–trừ tổ hợp; tích cuối dùng tổng dịch có điều kiện theo từng bit của G . Không có toán tử nhân hay chia số thực. Đây là lựa chọn hướng tới ánh xạ LUT/FF, *không phải báo cáo đã đo được 0 DSP*. Chuỗi tổ hợp bộ chia và bộ cộng có thể là critical path; cần synthesis và STA để kết luận tài nguyên hoặc khả năng chạy ở tần số mục tiêu.

2.3.6 Bất biến bộ chia và tích shift-add

Tiền điều kiện. $U \in \{0, \dots, 4096\}$, $V \in \{4096, \dots, 8192\}$ và $U \leq V$. Đặt $R_0 = U$, $Q_0 = 0$. Với $i \in \{0, \dots, 8\}$, thuật toán chọn $d_i = \mathbf{1}[R_i \geq V]$, rồi cập nhật:

$$Q_{i+1} = 2Q_i + d_i, \quad R_{i+1} = 2(R_i - d_i V). \quad (2.61)$$

Q_i là prefix toán học; trong RTL các bit của prefix nằm ở đầu cao của vector quotient chín bit. Chứng minh quy nạp bất biến:

$$2^i U = 2V Q_i + R_i, \quad 0 \leq R_i < 2V. \quad (2.62)$$

Tại $i = 0$, đẳng thức là $U = 0 + U$ và $U \leq V < 2V$. Nếu $R_i < V$ thì $d_i = 0$; nếu $V \leq R_i < 2V$ thì $d_i = 1$. Cả hai cho $0 \leq R_i - d_i V < V$, nên $0 \leq R_{i+1} < 2V$. Nhân đẳng thức cũ với hai và thay cập nhật:

$$2^{i+1} U = 4V Q_i + 2R_i = 2V(2Q_i + d_i) + 2(R_i - d_i V) = 2V Q_{i+1} + R_{i+1}. \quad (2.63)$$

Sau chín bước, R_9 chẵn; chia hai cho $256U = VQ_9 + R_9/2$, $0 \leq R_9/2 < V$. Hậu điều kiện là $Q_9 = \lfloor 256U/V \rfloor = G$. Mọi remainder nhỏ hơn $2V \leq 16384$, nằm trong unsigned 15-bit; quotient cuối $G \leq 256$ nằm trong unsigned 9-bit.

Viết $G = \sum_{j=0}^8 g_j 2^j$, $g_j \in \{0, 1\}$. Với $A_0 = 0$, $A_{i+1} = A_i + g_i(z2^i)$, quy nạp cho $A_i = z \sum_{j=0}^{i-1} g_j 2^j$: cơ sở tổng rỗng bằng không, bước sau thêm đúng hạng $g_i z 2^i$. Prefix không âm và không vượt $G \leq 256$, nên mọi $A_i \in [-32768, 32512]$; mỗi hạng $z2^i$ cũng trong miền này. Signed 18-bit chứa cả toán hạng dịch và tổng. Cuối cùng $A_9 = zG$, floor chia 256 và sat₈ cho đúng (2.60).

Nghĩa vụ còn mở. Các chứng minh trên xác định số học và độ rộng, không chứng minh netlist chỉ dùng LUT6 hay không dùng DSP. Dịch với khoảng cố định có thể chỉ là nổi dây; dịch biến có thể cần mux. Ánh xạ cell và timing phải được xác nhận bằng công cụ FPGA trên cấu hình đích.

2.3.7 Pipeline và phạm vi đánh giá

Năm tầng thanh ghi lần lượt giữ: p và mẫu gốc; E và dấu; G ; tích; kết quả kẹp ngưỡng. Mọi tầng tiến ở mỗi chu kỳ, kể cả bubble. Nếu nhận mẫu tại cạnh k , đầu ra tương ứng xuất hiện sau cạnh $k + 4$; đây là năm tầng tính cả cạnh nhận. `out_valid` đi qua cùng số tầng, không chạy sớm hơn dữ liệu.

Giao diện không có **ready**: nguồn có thể phát tối đa một mẫu mỗi chu kỳ và bên nhận phải luôn nhận được đầu ra hợp lệ. $II = 1$ là khả năng cấu trúc của pipeline này khi timing đạt, không phải cam kết thông lượng toàn hệ thống. Reset xóa mọi valid đang bay.

Cần đánh giá cả sai số vô hướng trên 256 mã và độ chính xác tác vụ sau khi tích hợp. Quy ước biểu diễn $Q_{4.4}$ bao phủ toàn miền mã INT8 nhưng không bao phủ toàn bộ trục số thực. Kết quả kiểm tra số học vô hướng được ghi trong Phụ lục A.1.2; chưa có kết quả ImageNet hay đo đạc FPGA cho biến thể này.

2.4 I-LayerNorm: thống kê, căn nguyên và affine

Trừ trung bình đưa các đặc trưng về quanh một mốc chung; chia cho độ lệch chuẩn đưa độ dao động về một đơn vị tham chiếu. Trên mạch số, khó khăn cần quản lý là bộ đệm chờ thống kê, độ rộng tổng và lịch phép chia/căn. Căn bậc hai không phải phép toán “không thể làm” trên FPGA, và Newton không phải lựa chọn duy nhất; ở đây ta học một thuật toán căn nguyên với điều kiện đầu vào và yêu cầu đầu ra cụ thể.

2.4.1 Thống kê theo token

LayerNorm chuẩn hóa theo D đặc trưng của *mỗi token*, không theo cả batch [2]:

$$\mu = \frac{1}{D} \sum_{j=1}^D x_j, \quad v = \frac{1}{D} \sum_{j=1}^D (x_j - \mu)^2, \quad (2.64)$$

$$y_j = \gamma_j \frac{x_j - \mu}{\sqrt{v + \epsilon}} + \beta_j. \quad (2.65)$$

Dưới đây là một bộ quy tắc tính toán số nguyên minh họa với scale S_X dùng chung trên chiều chuẩn hóa:

$$\mu_I = \text{RNE} \left(\frac{1}{D} \sum_{j=1}^D I_{x,j} \right), \quad c_j = I_{x,j} - \mu_I, \quad (2.66)$$

$$v_I = \left\lfloor \frac{1}{D} \sum_{j=1}^D c_j^2 \right\rfloor, \quad \epsilon_I = \max(1, \lceil \epsilon / S_X^2 \rceil), \quad (2.67)$$

$$V_I = v_I + \epsilon_I. \quad (2.68)$$

Trung bình nguyên và variance bị làm tròn nên biểu thức này là xấp xỉ, không phải đồng nhất với LayerNorm FP32. Ép $\epsilon_I \geq 1$ tránh mẫu số bằng không nhưng có thể làm thay đổi đáng kể vector có variance nhỏ; tăng độ chính xác thống kê là một phương án cần đánh giá. Khi S_X khác nhau giữa các kênh, không được áp dụng phép khử scale này nếu chưa quy đổi các kênh.

Tiền điều kiện thống kê là $D \in \{1, \dots, 32768\}$, $I_{x,j} \in \text{INT8}$, một scale chung $S_X > 0$ và $\epsilon \in \mathbb{R}_{>0}$; thêm $v_I + \epsilon_I \leq 2^{32} - 1$ trước khi gọi căn. Vì trung bình số thực của các mã nằm trong $[-128, 127]$, RNE của nó cũng trong miền đó; do đó $c_j = I_{x,j} - \mu_I \in [-255, 255] \cap \mathbb{Z}$. Mã c_j cần signed 9-bit, $c_j^2 \leq 65025$ cần unsigned 16-bit. Với $|c_j| \leq 255$, tổng bình phương bị chặn bởi $65025D$. Điều kiện $D \leq 32768$ đủ để tổng đó nằm trong INT32 có dấu; cộng epsilon và các định dạng mở rộng vẫn phải kiểm tra riêng. Thu thập $\sum I_x$ và $\sum I_x^2$ có thể thực hiện một lượt, nhưng các mẫu đầu vector thường vẫn cần buffer để chờ thống kê trước khi chuẩn hóa.

2.4.2 Căn bậc hai bằng Newton nguyên

Với $V_I \geq 1$, đặt $b = \lfloor \log_2 V_I \rfloor + 1$ và chọn giá trị khởi tạo trên căn thật:

$$s_0 = 2^{\lceil b/2 \rceil}, \quad (2.69)$$

$$\tilde{s}_{t+1} = \left\lfloor \frac{s_t + \lfloor V_I / s_t \rfloor}{2} \right\rfloor, \quad (2.70)$$

$$s_{t+1} = \min(s_t, \tilde{s}_{t+1}), \quad 0 \leq t < 10. \quad (2.71)$$

Phép chặn bằng giá trị cũ giữ nghiệm khi đã hội tụ, tránh dao động giữa hai số nguyên lân cận ở các số không chính phương. Thiết kế phải kiểm tra bất biến $s_t \geq 1$ và điều kiện nghiệm cuối:

$$s_{10}^2 \leq V_I < (s_{10} + 1)^2. \quad (2.72)$$

Chứng minh ngay sau đây áp dụng riêng cho $1 \leq V_I \leq 2^{32} - 1$, khởi tạo trên và phép chặn min; không áp dụng cho số nguyên có độ rộng tùy ý hoặc thuật toán bỏ min. Golden model còn kiểm tra hậu điều kiện để phát hiện sai lệch triển khai. Trường hợp căn của không, nếu được phép ở giao diện khác, phải có nhánh xử lý riêng.

I-LayerNorm sử dụng lập số nguyên trong I-ViT [3]. Tuy nhiên, *mười vòng lặp không tự tương đương mười chu kỳ clock*. Nếu một vòng cần L_{iter} chu kỳ kể cả bộ chia và thanh ghi, cận lịch thực thi nối tiếp có dạng:

$$L_{\text{LN}} = L_{\text{stats}} + L_{\text{init}} + 10L_{\text{iter}} + L_{\text{normalize}} + L_{\text{affine}}. \quad (2.73)$$

Latency cố định còn yêu cầu bộ chia có lịch cố định. Batch 1 trình bày thuật toán LayerNorm, chưa cung cấp RTL của toàn khối này.

2.4.3 Ví dụ: từ hai đặc trưng đến ba bước Newton

Cho $D = 2$, vector mã $I_x = (-10, 10)$, $S_X = 1/16$ và $\epsilon = 1/256$. Các điều kiện thống kê được thỏa; tính:

$$\mu_I = \text{RNE}((-10 + 10)/2) = 0, \quad c = (-10, 10), \quad (2.74)$$

$$v_I = \lfloor (100 + 100)/2 \rfloor = 100, \quad (2.75)$$

$$\epsilon_I = \max(1, \lceil (1/256)/(1/16)^2 \rceil) = 1, \quad V_I = 101. \quad (2.76)$$

Vì $2^6 \leq 101 < 2^7$, số bit $b = 7$ và $s_0 = 2^{\lceil 7/2 \rceil} = 16$.

t	s_t	$\lfloor 101/s_t \rfloor$	$\tilde{s}_{t+1}$	$s_{t+1} = \min(s_t, \tilde{s}_{t+1})$
0	16	6	$\lfloor (16 + 6)/2 \rfloor = 11$	11
1	11	9	$\lfloor (11 + 9)/2 \rfloor = 10$	10
2	10	10	$\lfloor (10 + 10)/2 \rfloor = 10$	10

Hậu điều kiện là $10^2 = 100 \leq 101 < 121 = (10 + 1)^2$. Từ trạng thái 10, bước tiếp theo vẫn là 10 nên các vòng còn lại của lịch mười vòng giữ nguyên kết quả. Ba dòng là ba bước thuật toán minh họa, không phải ba clock, cũng chưa phải toàn bộ LayerNorm: còn chuẩn hóa và affine ở mục sau.

2.4.4 Chứng minh hội tụ trên miền unsigned 32-bit

Miền và mục tiêu. Đặt $v = V_I \in \{1, \dots, 2^{32} - 1\}$, $u = \sqrt{v} \in \mathbb{R}_{>0}$, $a = \lfloor u \rfloor \in \{1, \dots, 65535\}$. Cần chứng minh $s_{10} = a$. Từ $2^{b-1} \leq v < 2^b$ và $s_0 = 2^{\lceil b/2 \rceil}$:

$$v < s_0^2 = 2^{2\lceil b/2 \rceil} \leq 2^{b+1} \leq 4v, \quad u < s_0 \leq 2u, \quad s_0 \leq 65536. \quad (2.77)$$

Với $s \in \mathbb{Z}_{>0}$, viết $v/s = q + \theta$, $q \in \mathbb{Z}_{\geq 0}$, $0 \leq \theta < 1$. Nếu $s + q = 2m$ với $m \in \mathbb{Z}$, thì $\lfloor m + \theta/2 \rfloor = m$. Nếu $s + q = 2m + 1$, thì $1/2 + \theta/2 \in [1/2, 1)$, nên $\lfloor m + 1/2 + \theta/2 \rfloor = m$. Trong cả hai trường hợp, floor không đổi khi thêm $\theta/2$. Vì vậy

$$T(s) := \left\lfloor \frac{s + \lfloor v/s \rfloor}{2} \right\rfloor = \left\lfloor \frac{s + v/s}{2} \right\rfloor. \quad (2.78)$$

Bất biến và giảm nghiêm ngặt. Với $s \geq a$, dùng $v \geq a^2$:

$$s + v/s \geq s + a^2/s = 2a + (s - a)^2/s \geq 2a, \quad T(s) \geq a. \quad (2.79)$$

Do đó $a \leq \min(s, T(s)) \leq s$. Nếu $s \geq a + 1$ thì $v < (a + 1)^2 \leq s^2$, suy ra $\lfloor v/s \rfloor \leq s - 1$, rồi

$$T(s) \leq \lfloor (2s - 1)/2 \rfloor = s - 1. \quad (2.80)$$

Tại $s = a$, min giữ nguyên a vì $T(a) \geq a$. Min là điều kiện thiết yếu: với $v = 3$, bản không chặn có $s_0 = 2$, $s_1 = 1$, $s_2 = 2$ và lặp lại; mười bước trả 2 không thỏa hậu điều kiện căn nguyên.

Cận số vòng. Đặt $d_t = \max(s_t - u, 0)$, $\eta_t = d_t/u$. Nếu $s_t < u$, tính nguyên và $s_t \geq a$ cho $s_t = a$, nên $d_{t+1} = 0$. Nếu $s_t \geq u$, bỏ floor/min chỉ có thể tăng kết quả, và

$$\frac{s_t + v/s_t}{2} - u = \frac{s_t^2 - 2us_t + u^2}{2s_t} = \frac{(s_t - u)^2}{2s_t}, \quad (2.81)$$

$$\eta_{t+1} \leq \eta_t^2/2. \quad (2.82)$$

Cơ sở $\eta_0 \leq 1 = 2^{-(2^0-1)}$. Nếu $\eta_t \leq 2^{-(2^t-1)}$, bình phương rồi chia hai cho $\eta_{t+1} \leq 2^{-2(2^t-1)-1} = 2^{-(2^{t+1}-1)}$. Tại $t = 5$:

$$d_5 = u\eta_5 \leq u2^{-31} < 2^{16}2^{-31} = 2^{-15} < 1. \quad (2.83)$$

Suy ra $s_5 < u + 1 < a + 2$, nên $s_5 \leq a + 1$. Nếu $s_5 = a + 1$, bước giảm nghiêm ngặt cho $s_6 = a$; nếu $s_5 = a$ thì đã bất động. Vì thế $s_6 = s_7 = s_8 = s_9 = s_{10} = a$, và $a^2 \leq v < (a + 1)^2$ hoàn tất chứng minh bằng tay. Mười vòng đủ với miền đầu vào, cách khởi tạo và phép cập nhật đã quy định; sáu vòng đã đủ theo cận chứng minh, nhưng reference vẫn chạy đủ mười vòng để giữ lịch đã công bố.

Độ rộng và latency. Bất biến cho $1 \leq s_t \leq 65536$ (17-bit unsigned). Vì $v \leq a^2 + 2a$ và $s_t \geq a$, thương $\lfloor v/s_t \rfloor \leq a + 2 \leq 65537$ cũng vừa 17 bit. Tổng trước chia hai không vượt $65536 + 65537 = 131073$, vừa 18 bit. Kiểm tra bình phương nên mở rộng 17-bit thành tích 34-bit; riêng $(a + 1)^2$ có thể bằng 2^{32} , không vừa 32-bit unsigned. $L_{\text{stats}}, L_{\text{init}}, L_{\text{normalize}}, L_{\text{affine}} \in \mathbb{Z}_{\geq 0}$ và $L_{\text{iter}} \in \mathbb{Z}_{> 0}$ chỉ là tham số lịch cho đến khi có RTL. Fixed-count, dữ liệu không tràn và convergence là ba nghĩa vụ tách biệt với timing vật lý.

2.4.5 Chuẩn hóa và tham số affine bằng dyadic scaling

Chọn $F_n \in \mathbb{Z}_{\geq 0}$ bit phần lẻ, $\gamma_j, \beta_j \in \mathbb{R}$, $S_Y > 0$, $M_{\gamma,j}, B_j \in \mathbb{Z}$ và $e_{\gamma,j} \in \mathbb{Z}_{\geq 0}$; không ngần ngại gán các hệ số này vào cổng residual vì affine cho phép số âm. Chọn F_n bit phần lẻ cho giá trị chuẩn hóa, đặt:

$$n_j = \left\lfloor \frac{2^{F_n} c_j}{s_{10}} \right\rfloor, \quad (2.84)$$

$$\frac{\gamma_j}{S_Y 2^{F_n}} \approx M_{\gamma,j} 2^{-e_{\gamma,j}}, \quad B_j = \text{RNE}(\beta_j / S_Y), \quad (2.85)$$

$$I_{y,j} = \text{sat}_8(\mathcal{F}_{e_{\gamma,j}}(M_{\gamma,j} n_j) + B_j). \quad (2.86)$$

γ_j có thể âm, vì vậy hệ số affine cần hỗ trợ có dấu, khác với tỷ số scale dương của residual. Độ rộng của $2^{F_n} c_j$, tích affine và phép cộng bias phải được chặn trước khi chọn thanh ghi. Đặt $C_n = 255 \cdot 2^{F_n}$. Vì $s_{10} \geq 1$, $|n_j| \leq C_n$. Điều kiện đủ cho signed w_n bit là $C_n \leq 2^{w_n-1} - 1$. Để phép nhân/cộng affine không tràn signed w_a bit, dùng $|M_{\gamma,j}| C_n + |B_j| \leq 2^{w_a-1} - 1$ (bảo thủ, trước cả dịch). Chưa chọn F_n, w_n, w_a thì chưa có khẳng định “affine INT32 an toàn”.

Tất cả $S_X, S_Y, \epsilon_I, M_{\gamma,j}, e_{\gamma,j}, B_j$ phải được xuất cùng phiên bản mô hình.

2.5 Residual connection và requantization

2.5.1 Hai scale đầu vào khác nhau

Tiền điều kiện: hai mã thuộc cùng token/kênh, với các miền:

$$I_F, I_X \in \text{INT8}, \quad S_F, S_X, S_{\text{out}} > 0, \quad (2.87)$$

$$M_F, M_X \in \{0, \dots, 2^{31} - 1\}, \quad e_F, e_X \in \{0, \dots, 31\}. \quad (2.88)$$

Các giá trị tái dựng thỏa:

$$\hat{F} + \hat{X} = S_F I_F + S_X I_X, \quad (2.89)$$

$$(\hat{F} + \hat{X}) / S_{\text{out}} = \frac{S_F}{S_{\text{out}}} I_F + \frac{S_X}{S_{\text{out}}} I_X. \quad (2.90)$$

Thay mỗi tỷ số bằng dyadic, floor riêng mỗi nhánh, rồi kẹp là *định nghĩa* datapath, không phải đẳng thức với tổng FP32. Với $Y = F + X$, zero-point bằng không và output scale S_{out} :

$$\frac{S_F}{S_{\text{out}}} \approx M_F 2^{-e_F}, \quad \frac{S_X}{S_{\text{out}}} \approx M_X 2^{-e_X}, \quad (2.91)$$

$$I_{\text{out}} = \text{sat}_8(\mathcal{F}_{e_F}(M_F I_F) + \mathcal{F}_{e_X}(M_X I_X)). \quad (2.92)$$

Đây là làm tròn riêng từng nhánh, có thể khác làm tròn một lần sau khi cộng. Phụ lục A.2 giữ tích 8×32 trong 40 bit, giữ cả 40 bit sau dịch và cộng bằng 41 bit trước khi saturate. Cắt về INT32 trước saturation có thể lật dấu giá trị lớn và làm kẹp về sai biên.

Cận và hậu điều kiện. Đặt $M_{\text{max}} = 2^{31} - 1$. Các miền tích và tổng sau dịch thỏa:

$$M_F I_F, M_X I_X \in [-128M_{\text{max}}, 127M_{\text{max}}] \subset (-2^{38}, 2^{38}), \quad (2.93)$$

$$\mathcal{F}_{e_F}(M_F I_F) + \mathcal{F}_{e_X}(M_X I_X) \in [-256M_{\text{max}}, 254M_{\text{max}}] \subset (-2^{39}, 2^{39}). \quad (2.94)$$

Dịch phải không tăng độ lớn vượt các cận này. Hai tích vừa signed 40-bit và tổng vừa signed 41-bit. Kiểm tra > 127 , < -128 , rồi mới lấy tám bit thấp bảo đảm kết quả sat_8 đúng, không wrap trước khi kẹp.

Đặt $\alpha_F = S_F/S_{\text{out}}$, $\alpha_X = S_X/S_{\text{out}}$ và $R = \mathcal{F}_{e_F}(M_F I_F) + \mathcal{F}_{e_X}(M_X I_X)$. Áp dụng đẳng thức sai số rescale cho hai nhánh, cộng và dùng bất đẳng thức tam giác cho:

$$|R - (\alpha_F I_F + \alpha_X I_X)| < |I_F| |\alpha_F - M_F 2^{-e_F}| + |I_X| |\alpha_X - M_X 2^{-e_X}| + 2. \quad (2.95)$$

Với $I_F = I_X = 1$, hai tỷ số $1/2$ cho $\lfloor 1/2 \rfloor + \lfloor 1/2 \rfloor = 0$, nhưng $\lfloor 1/2 + 1/2 \rfloor = 1$. Vì vậy Python không được dời floor ra sau phép cộng.

Hai số mũ được chốt cùng các tích của cùng giao dịch. Nếu dùng số mũ mới ở chu kỳ sau, hệ số của token kế tiếp sẽ bị áp vào token hiện tại. Module giả định hai nhánh đã được ghép cặp ở đầu vào; nó không tự giải quyết sự lệch token giữa shortcut và nhánh attention.

2.5.2 QAT scale-locking

Nếu quy trình huấn luyện/calibration bảo đảm đúng $S_F = S_X = S_{\text{out}}$, thì:

$$I_{\text{out}} = \text{sat}_8(I_F + I_X). \quad (2.96)$$

Tổng cần chín bit có dấu trước khi kẹp. Tham số elaboration **SCALE_LOCKED=1** trong module chọn đường cộng trực tiếp và bỏ phép nhân khởi mô tả phần cứng được chọn. Chỉ đặt các cổng runtime $M = 1, e = 0$ mà không chuyên biệt hóa thiết kế không bảo đảm công cụ loại được bộ nhân. Scale-locking cũng không loại bỏ clipping: tổng hai tín hiệu vẫn có thể vượt dải INT8.

Với tiền điều kiện scale-locking, $S_F/S_{\text{out}} = S_X/S_{\text{out}} = 1$. Chọn $M_F = M_X = 1$, $e_F = e_X = 0$ cho từng nhánh:

$$\mathcal{F}_0(1 \cdot I_F) = \lfloor I_F/1 \rfloor = I_F, \quad (2.97)$$

$$\mathcal{F}_0(1 \cdot I_X) = \lfloor I_X/1 \rfloor = I_X, \quad (2.98)$$

$$I_{\text{out}} = \text{sat}_8(I_F + I_X), \quad I_F + I_X \in [-256, 254]. \quad (2.99)$$

Signed 9-bit đủ cho tổng chưa kẹp; RTL giữ 41-bit để dùng chung lịch ba tầng. Phép giản lược bảo toàn hàm số nguyên dưới tiền điều kiện scale bằng nhau, không bảo đảm QAT sau khi ép scale giữ nguyên độ chính xác mô hình trước đó.

2.5.3 Quy định dữ liệu giữa các tầng và các yêu cầu kiểm chứng còn lại

Các định nghĩa số nguyên trong B.1 là reference đơn vị cho A.1/A.2 và căn uint32, không phải mô phỏng toàn encoder. Đối chiếu bắt buộc:

Dại lượng	RTL/kiểu	Reference B.1
$S_X = S_Y = 2^{-4}$	INT8 signed	<code>shift_gelu_q44</code>
$p \in [-216, 212]$	signed 16-bit	bốn floor 0, 1, 3, 4
$E \in [0, 4096]$	unsigned 13-bit	Q12, $4096 + 128r$
$G \in [0, 256]$	unsigned 9-bit	floor $256U/V$
M, e	signed 32 / unsigned 5	miền kiểm tra tường minh
Tích/tổng residual	signed 40 / 41	số nguyên không wrap
$v \in [1, 2^{32} - 1]$	chưa có RTL LN	<code>isqrt_u32_fixed</code>

Proof state sau refactoring: shape, cận số học, bất biến thuật toán và convergence căn có dẫn xuất trong tài liệu; kiểm tra hữu hạn được báo riêng ở phụ lục. Còn mở: kiểm tra Lean kernel, ngữ nghĩa HDL bốn trạng thái, scoreboard temporal, ánh xạ DSP/LUT, STA và chất lượng suy luận toàn mạng. $WNS \geq 0$ và $WHS \geq 0$ là tiêu chí nghiệm thu triển khai, không phải bất biến số học có thể suy ra từ mã nguồn.

Tự kiểm tra và đối chiếu

1. Tại sao $I_x = 35$ không tự xác định một giá trị thực nếu chưa biết scale?
2. Trong ví dụ ShiftGELU, vì sao không trả $G = 248$ làm output INT8 và vì sao kết quả cuối là 33?
3. Với $V_I = 101$, hãy kiểm tra hai bất đẳng thức của căn nguyên và giải thích vì sao không suy ra độ trễ ba clock.
4. Với $A = -7, M = 3, e = 2$, thay floor bằng cắt về zero làm thay đổi bao nhiêu mã?

Đối chiếu. Giá trị tái dựng là SI_x . G là mã hệ số unsigned 9-bit, phải nhân mẫu rồi dịch tám bit: $8680 = 33 \cdot 256 + 232$. Căn nguyên thỏa $100 \leq 101 < 121$; lịch phần cứng còn phụ thuộc bộ chia/thanh ghi. Floor cho -6 , cắt về zero cho -5 , lệch một mã.

Tóm tắt và chuyển sang datapath

Các ví dụ đã chốt phép tính nguyên, vị trí làm tròn và miền trung gian. Chưa có sơ đồ bố trí phần cứng hay chứng nhận 200 MHz chỉ từ các công thức đó. Chương 3 sẽ đặt phép tính vào thanh ghi, theo dõi valid cùng dữ liệu và kiểm tra độ rộng; STA sau triển khai mới quyết định mục tiêu timing có đạt hay không.

Chương 3

Thiết kế datapath và tối ưu hóa tài nguyên FPGA

Mục tiêu học tập

- Đặt phép nhân/cộng nguyên vào tầng thanh ghi và giải thích độ rộng từng tầng.
- Tính tay một cây cộng có số lá lẻ và một phép DSP packing có tích thấp âm.
- Phân biệt khoảng cách khởi phát, độ trễ và kết quả kiểm tra timing.

Cầu nối từ Chương 2. Công thức cho biết phải tính gì, nhưng chưa nói giá trị xuất hiện ở cạnh clock nào. Thanh ghi giống các khay giữ kết quả giữa những bước thao tác: mỗi cạnh clock chuyển trạng thái sang khay kế tiếp. Đường ống (Pipeline) cho phép nhiều mẫu cùng ở các bước khác nhau; nó không cho phép ghép tích của mẫu trước với hệ số của mẫu sau. Ví dụ dưới đây theo dõi cả giá trị lẫn nhãn valid.

Pre-check và proof state. Các chứng minh dùng số nguyên không tràn và mô hình thanh ghi hai trạng thái với các điều kiện đã nêu về reset và valid. Assertions trong A.3–A.5 là nghĩa vụ kiểm tra thực thi, không phải chứng thư Lean. Tách biệt số tầng thanh ghi với khoảng cách cạnh clock, và hai tích dùng chung toán hạng với hai MAC độc lập. Đặc biệt, guard bits không loại bỏ borrow của tích thấp âm; phép bù dấu là bắt buộc. Chưa có chứng nhận Lean kernel, mô phỏng HDL hay STA cho các listing.

Chương này chuyển các quy định số học của Chương 2 thành những khối xử lý có độ rộng, lịch thanh ghi và giao diện rõ ràng. Cần phân biệt ba mức: nguyên lý kiến trúc; RTL mô tả một chức năng xác định; kết quả đo sau synthesis/place-and-route. Một biểu thức chứa một phép nhân không tự chứng minh thiết kế sử dụng đúng một DSP hoặc đạt tần số mục tiêu.

3.1 Đơn vị nhân–tích lũy INT8

Đặt $\text{INT8} = \{z \in \mathbb{Z} : -128 \leq z \leq 127\}$ và $\text{INT32} = \{z \in \mathbb{Z} : -2^{31} \leq z \leq 2^{31} - 1\}$. Tiền điều kiện: $K_r \in \mathbb{Z}_{>0}$, $I_{A,r}, I_{B,r} \in \text{INT8}$, $r \in \{0, \dots, K_r - 1\}$, $I_b \in \mathbb{Z}$. Với K_r phần tử reduction, phép nhân–tích lũy có bias nguyên là:

$$p_r = I_{A,r} I_{B,r}, \quad p_r \in [-16256, 16384], \quad (3.1)$$

$$\text{Acc} = \sum_{r=0}^{K_r-1} p_r + I_b. \quad (3.2)$$

Tích cần 16 bit có dấu. Một độ rộng bảo thủ cho tổng không bias là:

$$W_{\text{sum}} = 16 + \lceil \log_2 K_r \rceil. \quad (3.3)$$

Chọn accumulator INT32 chỉ an toàn khi đã kiểm tra miền reduction và bias. Điều kiện đủ là $K_r 16384 + |I_b| \leq 2^{31} - 1$; không dùng overflow rồi mới saturation để thay thế điều kiện này.

Bất biến accumulator. Khởi tạo $A_0 = I_b$, cập nhật $A_{j+1} = A_j + p_j$ cho $0 \leq j < K_r$. Cơ sở là tổng rỗng; thay giả thiết $A_j = I_b + \sum_{r=0}^{j-1} p_r$ vào cập nhật cho $A_{j+1} = I_b + \sum_{r=0}^j p_r$. Vì $|p_r| \leq 16384$:

$$|A_j| \leq |I_b| + \sum_{r=0}^{j-1} |p_r| \leq |I_b| + 16384j \leq |I_b| + 16384K_r \leq 2^{31} - 1. \quad (3.4)$$

Do đó mọi trạng thái A_j , không chỉ tổng cuối, thuộc INT32 dưới tiền điều kiện cận trên. Nếu không thỏa cận, phải mở rộng accumulator hoặc chứng minh một cận chặt hơn từ dữ liệu.

Đặt $M \in \{0, \dots, 2^{31} - 1\}$, $e \in \{0, \dots, 31\}$; tích $A_{K_r} M$ phải giữ đầy đủ signed 64-bit trước khi floor, cho $U \in \mathbb{Z}$. Chỉ requantize sau khi cộng đủ các partial sum của cùng phần tử đầu ra:

$$U = \lfloor \text{Acc } M / 2^e \rfloor, \quad (3.5)$$

$$I_{\text{out}} = \begin{cases} 127, & U > 127, \\ -128, & U < -128, \\ U, & -128 \leq U \leq 127. \end{cases} \quad (3.6)$$

Phép lấy tám bit thấp trong RTL chỉ đúng ở nhánh cuối, sau khi đã xác nhận miền. Bảo hòa từng partial sum INT8 trước khi cộng các tile theo K_r nhìn chung không bằng bảo hòa một lần ở cuối.

Hậu điều kiện saturation. Hàm $\text{sat}_8 : \mathbb{Z} \rightarrow \text{INT8}$ được cho bởi ba nhánh trên: $U > 127$, $U < -128$, hoặc $-128 \leq U \leq 127$. Ba miền rời nhau và phủ $\mathbb{Z}$, nên hàm xác định duy nhất; tại $U = -128, 127$ nhánh cuối giữ nguyên giá trị. Phản ví dụ về thứ tự saturation: với partial sum 200 và -100 , $\text{sat}_8(200 - 100) = 100$ nhưng $\text{sat}_8(\text{sat}_8(200) - 100) = 27$.

Một MAC nối tiếp có một multiplier và accumulator feedback; một dot-product song song cần nhiều multiplier rồi giảm tổng. $II = 1$ của dot-product có nghĩa nhận một *cập vector mới* mỗi chu kỳ, không có nghĩa chỉ một multiplier thực hiện mọi tích trong chu kỳ đó.

3.2 Cây cộng pipelined và quản lý bit-growth

3.2.1 Ví dụ: cây cộng ba phần tử và một bubble

Cho $N_v = 3, W = 8$, hai vector INT8 $a = (2, -3, 4)$ và $b = (5, 6, -2)$. Khi nhận mẫu A tại cạnh k , tầng nhân chốt:

$$L_a = \lceil \log_2 3 \rceil = 2, \quad P_a = 4, \quad (3.7)$$

$$(\pi_0, \pi_1, \pi_2, \pi_3) = (2 \cdot 5, (-3) \cdot 6, 4(-2), 0) = (10, -18, -8, 0), \quad (3.8)$$

$$(t_{1,0}, t_{1,1}) = (10 - 18, -8 + 0) = (-8, -8), \quad (3.9)$$

$$t_{2,0} = -8 - 8 = -16. \quad (3.10)$$

Nhánh padding bằng zero giữ nguyên tổng $10 - 18 - 8 = -16$. Theo cận thiết kế, các tầng lần lượt dùng signed 16, 17 và 18 bit, dù riêng các số nhỏ này có thể vừa ít bit hơn. Cận phải bao phủ mọi input hợp lệ, không chỉ vector minh họa. Output 32 bit của cấu hình mặc định mở rộng dấu kết quả 18 bit.

Giả sử pipeline đã reset, không reset thêm và bên nhận luôn nhận output hợp lệ. Chèn bubble tại $k + 1$; nhận mẫu B có $a = (1, 0, 0)$, $b = (7, 0, 0)$ tại $k + 2$:

Cạnh	Input	(v_0, v_1, v_2) sau cạnh	Output sau cạnh
k	A, valid 1	$(1, 0, 0)$	Không hợp lệ
$k + 1$	Bubble, valid 0	$(0, 1, 0)$	Không hợp lệ
$k + 2$	B, valid 1	$(1, 0, 1)$	A: -16
$k + 3$	Bubble, valid 0	$(0, 1, 0)$	Không hợp lệ
$k + 4$	Bubble, valid 0	$(0, 0, 1)$	B: 7

Không kiểm tra payload ở hàng output không hợp lệ: RTL vẫn có thể cập nhật thanh ghi tại những cạnh đó. Khoảng cách nhận đến xuất của mỗi mẫu là hai clock; số tầng chốt là ba. Khoảng cách khởi phát (Initiation Interval; II) có thể bằng một dưới điều kiện timing đạt và không có stall, dù ví dụ cố ý không phát liên tục.

3.2.2 Quy luật độ rộng và lịch thanh ghi

Tiền điều kiện: $N_v \in \{1, \dots, 2^{30}\}$ là chiều vector, $W \in \mathbb{Z}_{\geq 2}$ là độ rộng toán hạng, mỗi toán hạng trong $[-2^{W-1}, 2^{W-1} - 1] \cap \mathbb{Z}$. Các tham số độ rộng phải biểu diễn được bởi kiểu `int signed` của elaboration, không overflow khi tính $2W + L_a$. Đặt:

$$L_a = \lceil \log_2 N_v \rceil, \quad P_a = 2^{L_a}, \quad W_s = 2W + s. \quad (3.11)$$

Tầng 0 chốt N_v tích và thêm $P_a - N_v$ số không. Ở tầng $s \geq 1$:

$$t_{s,j} = \text{sext}_{W_s}(t_{s-1,2j}) + \text{sext}_{W_s}(t_{s-1,2j+1}), \quad 0 \leq j < P_a/2^s. \quad (3.12)$$

Mỗi tầng cộng tăng một bit và chốt kết quả riêng. Đệm số không bảo đảm các trường hợp $N_v = 3, 5, 17$ không làm rơi nhánh lẻ như cách lấy đơn thuần $N_v \gg (s + 1)$. Với $N_v = 1$, tầng nhân chính là tầng cuối, không cần tầng cộng.

Chứng minh bit-growth bằng quy nạp. Gọi π_j là tích của hai toán hạng tại lá $j < N_v$, và $\pi_j = 0$ cho $N_v \leq j < P_a$. Mỗi tích của hai signed W bit nằm trong signed $2W$ bit; cụ thể tích lớn nhất $2^{2W-2} < 2^{2W-1}$ và tích âm nhỏ nhất $-2^{W-1}(2^{W-1} - 1) > -2^{2W-1}$. Mệnh đề tại tầng s là:

$$t_{s,j} = \sum_{r=j2^s}^{(j+1)2^s-1} \pi_r, \quad t_{s,j} \in [-2^{W_s-1}, 2^{W_s-1} - 1]. \quad (3.13)$$

Cơ sở $s = 0$ là một lá với $W_0 = 2W$. Giả sử hai con có độ rộng $w = W_{s-1}$ và cùng thuộc $[-2^{w-1}, 2^{w-1} - 1]$. Tổng hai con thuộc

$$[-2^w, 2^w - 2] \subseteq [-2^w, 2^w - 1], \quad (3.14)$$

là miền signed $w+1 = W_s$ bit. Hai khoảng lá con $[2j2^{s-1}, (2j+1)2^{s-1})$ và $[(2j+1)2^{s-1}, (2j+2)2^{s-1})$ kề nhau, không chồng lấp; cộng hai tổng ghép thành đúng khoảng cha. Suy ra

$W_s = 2W + s$, và tại INT8 là $16 + s$. Mở rộng dấu *trước* cộng giữ giá trị con; lấy đủ W_s bit không làm mất carry có ý nghĩa. Tầng cuối chứa tổng tất cả lá; các lá padding bằng không nên không đổi dot-product.

Chứng minh bất biến thời gian. Gọi $\pi_j[k]$ là tích của vector nhận ngay trước cạnh k và $t_{s,j}[k+s]$ là thanh ghi ngay sau cạnh $k+s$. Nếu không reset, stage 0 chốt $\pi_j[k]$; stage s đọc hai giá trị stage $s-1$ đã chốt ở cạnh trước, do đó:

$$t_{s,j}[k+s] = \sum_{r=j2^s}^{(j+1)2^s-1} \pi_r[k], \quad v_s[k+s] = v_{in}[k]. \quad (3.15)$$

Bước quy nạp dùng cùng phép ghép khoảng lá và phép gán valid từ stage trước; không có enable theo valid, nên bubble không đổi offset. Có $1 + L_a$ tầng nhận/chốt; khoảng thời gian từ cạnh nhận k đến cạnh ra $k + L_a$ là L_a chu kỳ. Nếu gọi latency theo số tầng thì $\text{Latency}_{\text{stages}} = 1 + \lceil \log_2 N_v \rceil$; hai cách đếm phải được ghi rõ trong scoreboard, nhất là $N_v = 1$. Mỗi tầng chỉ dùng trạng thái cũ và có thanh ghi riêng, nên có thể nhận vector mới mỗi cạnh khi đích không stall và timing đạt. Chèn thanh ghi không tự chứng minh đạt timing hay thông lượng trong hệ thống có backpressure.

Module ở Phụ lục A.3 có $L_a + 1$ tầng thanh ghi tính cả tầng nhân. Mẫu nhận ở cạnh k cho kết quả sau cạnh $k + L_a$. Valid tiến cùng dữ liệu qua mọi tầng, kể cả bubble; đầu ra chỉ được đọc khi valid bằng một. Không có backpressure, vì vậy khi ghép vào AXI-Stream phải bổ sung cơ chế giữ/stall toàn pipeline hoặc bộ đệm đủ sức chứa các giao dịch đã nhận.

Số multiplier song song là N_v ; padding không yêu cầu multiplier cho nhánh zero. RTL khai báo độ rộng từng tầng thay vì giữ 32 bit cho mọi phép cộng. Công cụ vẫn có thể thay đổi cách ánh xạ thanh ghi sau tối ưu hóa; cần xem báo cáo netlist và tài nguyên thực.

3.3 DSP packing trên DSP48E2

3.3.1 Ví dụ: đóng gói 12 nhân với 5 và âm 8

Hai phép nhân dùng chung $a = 12$, với $b = 5, c = -8$, đều là INT8. Ký hiệu chữ thường giữ đúng quy ước ký hiệu của module. Đóng gói bằng dịch trái sau mở rộng dấu và cộng, không phải ghép bit tùy ý:

$$OP_{27} = 5 \cdot 262144 - 8 = 1310712, \quad (3.16)$$

$$P_{45} = 12 \cdot 1310712 = 15728544 \quad (3.17)$$

$$= 60 \cdot 262144 - 96. \quad (3.18)$$

OP_{27} nằm trong signed 27-bit; P_{45} nằm trong signed 45-bit. Tích thấp $v = -96$ có mã bù hai:

$$P_{45} \bmod 2^{16} = 2^{16} - 96 = 65440 = \text{0xFFA0}. \quad (3.19)$$

Giải mã signed 16-bit trừ 65536, trả $65440 - 65536 = -96 = ac$. Để thấy rõ tại sao cần bù dấu, viết lại phép chia theo slot:

$$P_{45} = 59 \cdot 262144 + 262048, \quad 0 \leq 262048 < 262144, \quad (3.20)$$

$$\lfloor P_{45}/262144 \rfloor = 59, \quad (3.21)$$

$$\text{signed}_{16}(P_{45}[33:18]) + \mathbf{1}_{ac<0} = 59 + 1 = 60 = ab. \quad (3.22)$$

Trường cao thô là 0x003B, không phải 0x003C. Các bit [17 : 16] bằng 11, là dấu mở rộng của phần thấp âm trong ví dụ này. Một hình vẽ “hai ô không đè nhau” chưa đủ chứng minh đúng: số âm tạo phép mượn ở ranh giới slot. Quy tắc bù khôi phục đúng hai tích trước khi chúng được sử dụng bởi bộ tích lũy bên ngoài.

3.3.2 Đóng gói hai phép nhân dùng chung một toán hạng

DSP48E2 có bộ nhân bù hai 27×18 bit; cổng A vật lý rộng 30 bit nhưng nhánh multiplier chỉ dùng 27 bit thấp [4]. Phân biệt kích thước cổng vật lý, multiplier và accumulator 48 bit khi thiết kế packing.

Xét $a, b, c \in \text{INT8}$ (số nguyên, không phải số thực bất kỳ trong đoạn). Ghép hai toán hạng bằng *phép cộng có dấu*, không phải nối bit tùy ý:

$$x_{27} = b2^{18} + c, \quad a_{18} = \text{sext}_{18}(a), \quad (3.23)$$

$$P_{45} = a_{18}x_{27} = (ab)2^{18} + ac. \quad (3.24)$$

Biến đổi và cận toán hạng. Mở rộng dấu giữ nguyên giá trị a, b, c , còn dịch trái chỉ được thực hiện sau khi mở rộng b lên signed 27-bit. Luật phân phối cho:

$$a_{18}x_{27} = a(b2^{18} + c) = ab2^{18} + ac, \quad (3.25)$$

$$\min x_{27} = (-128)2^{18} - 128 = -33554560, \quad (3.26)$$

$$\max x_{27} = 127 \cdot 2^{18} + 127 = 33292415. \quad (3.27)$$

Vì $-2^{26} \leq -33554560$ và $33292415 < 2^{26}$, toán hạng vừa signed 27-bit, kể cả tổng sau khi dịch. Tích chính xác có $|a_{18}x_{27}| \leq 128 \cdot 33554560 = 4294983680 < 2^{33} < 2^{44}$, nên signed 45-bit đủ. Phép nhân không bị wrap trước khi trích trường. Hai phép nhân không hoàn toàn độc lập: chúng *dùng chung a*. Việc cấp dữ liệu và ánh xạ tile phải khai thác được sự dùng chung này.

3.3.3 Bù dấu khi tách trường cao

Đặt $u = ab$, $v = ac$. Vì $|v| < 2^{18}$:

$$\left\lfloor \frac{P_{45}}{2^{18}} \right\rfloor = u + \left\lfloor \frac{v}{2^{18}} \right\rfloor = u - \mathbf{1}_{v < 0}, \quad (3.28)$$

$$v = \text{signed}_{16}(P_{45}[15 : 0]), \quad (3.29)$$

$$u = \text{signed}_{16}(P_{45}[33 : 18]) + \mathbf{1}_{v < 0}. \quad (3.30)$$

Chứng minh trích trường thấp/cao. Với $u, v \in [-16256, 16384] \cap \mathbb{Z}$, $|v| \leq 16384 < 2^{15} < 2^{18}$. Hạng $u2^{18}$ chia hết cho 2^{16} , nên $P_{45} \bmod 2^{16} = v \bmod 2^{16}$. Vì v vừa signed 16-bit, giải mã bù hai của 16 bit thấp trả đúng v . Đây là đẳng thức modulo, không phải khẳng định sign extension không ảnh hưởng trường cao.

Nếu $v \geq 0$, $\lfloor v/2^{18} \rfloor = 0$; nếu $v < 0$, $-1 < v/2^{18} < 0$ nên floor bằng -1 . Bởi u nguyên, $\lfloor u + v/2^{18} \rfloor = u + \lfloor v/2^{18} \rfloor$. Giá trị thương này nằm trong $[-16257, 16384]$, vừa signed 16-bit; do đó giải mã $P[33 : 18]$ trả chính $u - \mathbf{1}_{v < 0}$, và cộng bù bằng signed 17-bit trả u trước khi lấy 16 bit kết quả.

Hai bit $P[17 : 16]$ là khoảng trống giữa hai trường nhưng không triệt tiêu borrow: với $a = 1, b = 0, c = -1$, $P = -1$, 16 bit thấp là 0xFFFF, trường cao cũng 0xFFFF; giải mã

cao cho -1 chứ không phải $ab = 0$. Cộng $\mathbf{1}_{v < 0} = 1$ mới trả 0. Đây là phản ví dụ cho lập luận “guard bits ngăn tràn bit dấu” khi không có hiệu chỉnh.

Bit 15 của sản phẩm packed chính là bit dấu của v trong miền INT8 đã chốt. Bởi vậy lấy trường cao mà không cộng bù sẽ sai một đơn vị khi $ac < 0$. Ví dụ $a = 1, b = 1, c = -1$ cho $P = 2^{18} - 1$: trường cao bằng không, trong khi $ab = 1$.

Hai bit $[17 : 16]$ là phần mở rộng dấu của tích thấp: bằng 00 khi không âm và 11 khi âm. Khoảng cách slot không làm mất phép mượn do số âm. Trong module A.4, phép cộng bù giữ 17 bit trước khi trả về kết quả 16 bit.

3.3.4 Hai tích mỗi chu kỳ khác hai MAC độc lập

Module PE chỉ trả hai *tích*, chưa chứa hai accumulator. Nếu trực tiếp tích lũy nhiều sản phẩm packed trong accumulator 48 bit, phần thấp có thể carry/borrow sang phần cao. Muốn đóng gói cả MAC phải chứng minh giới hạn số tích cộng, thêm headroom và hiệu chỉnh tương ứng; không được dùng nguyên phép tách của một tích cho một tổng tùy ý.

Bản RTL dùng một phép nhân 18×27 và gợi ý `use_dsp="yes"`; số DSP sau ánh xạ vẫn phải được xác nhận. Nó dùng reset đồng bộ cho các thanh ghi datapath nhằm phù hợp hơn với thanh ghi nội bộ DSP48E2. Giữ clock-enable/valid và cấu trúc pipeline đơn giản giúp tránh buộc thanh ghi sang fabric. Hai tầng nhận mẫu ở cạnh k và trả kết quả sau cạnh $k + 1$.

Mức tiết kiệm DSP lý tưởng của riêng cặp multiplier là từ hai xuống một, không phải tiết kiệm 50% DSP của cả accelerator. Các accumulator, packing adder, bù dấu, fanout và routing có thể trở thành chi phí mới. Chỉ công bố thông lượng hai tích/DSP/chu kỳ khi synthesis xác nhận một DSP và timing đạt.

3.4 Unified GEMM Engine và lịch systolic

Đặt $M, N, K, T, D, h, d_k \in \mathbb{Z}_{>0}$, $D = hd_k$. Với mô hình thực $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, $C \in \mathbb{R}^{M \times N}$, dùng $C = AB$. Với datapath INT8, thay hai miền toán hạng bằng $\text{INT8}^{M \times K}$ và $\text{INT8}^{K \times N}$; output trước requantization là số nguyên accumulator đủ rộng, không tự là INT8. Các phép toán trong một encoder chuyển thành descriptor khác nhau của cùng engine:

Chế độ	M	N	K
Projection QKV ghép	T	$3D$	D
Projection đầu ra	T	D	D
Score của một head	T	T	d_k
Attention nhân V	T	d_k	T
MLP mở rộng	T	$4D$	D
MLP thu hẹp	T	D	$4D$

Kiểm tra không gian từng mode. Một head có $Q_a \in \mathbb{R}^{T \times d_k}$, $K_a^T \in \mathbb{R}^{d_k \times T}$, nên score thuộc $\mathbb{R}^{T \times T}$; xác suất sau Softmax nhân $V_a \in \mathbb{R}^{T \times d_k}$ trả $\mathbb{R}^{T \times d_k}$. MLP nhân $(T \times D)(D \times 4D)$ trả $T \times 4D$, rồi nhân $(T \times 4D)(4D \times D)$ trả $T \times D$. Các shape tương ứng dùng mã INT8 chỉ sau khi đã xác định scale/zero-point của mỗi tensor. Descriptor đổi shape không tự bổ sung Softmax hoặc hỗ trợ mọi scale.

Với mảng output-stationary $P_M \times P_N$, K_t bước reduction và đủ dữ liệu cấp mỗi nhịp, mô hình lý tưởng cho một tile đầy là:

$$L_{\text{tile}} \approx K_t + P_M + P_N - 2 + L_{\text{extra}}, \quad (3.31)$$

$$\eta_{\text{fill}} \approx \frac{K_t}{K_t + P_M + P_N - 2}. \quad (3.32)$$

Trong mô hình này $P_M, P_N, K_t \in \mathbb{Z}_{>0}$ và $L_{\text{extra}} \in \mathbb{Z}_{\geq 0}$. Lấy chỉ số PE $i \in \{0, \dots, P_M - 1\}$, $j \in \{0, \dots, P_N - 1\}$ và reduction $r \in \{0, \dots, K_t - 1\}$. Với skew đầu vào một bước mỗi hàng/cột và mỗi hop một clock, hạng r đến PE (i, j) ở cạnh $r + i + j$. Cạnh cuối là $(K_t - 1) + (P_M - 1) + (P_N - 1)$; đếm cả cạnh đầu cho $K_t + P_M + P_N - 2$ cạnh hoạt động. Đây là lịch lý tưởng được giả thiết, chưa là schedule của shell loopback A.5.

L_{extra} bao gồm register bổ sung, drain và ghi kết quả; tile biên còn giảm tỷ lệ PE có dữ liệu hữu ích. Các phương trình là mô hình lịch, không phải số đo của RTL Batch 2. Chọn kích thước mảng khác ký hiệu K reduction để tránh nhầm một số chiều ma trận với số PE.

Descriptor tối thiểu cần kích thước, stride/layout, scale, địa chỉ tile, cờ khởi tạo/cộng tiếp accumulator và cờ tile cuối theo K . Chuyển K thành K^T thường là vấn đề addressing hoặc transpose buffer. Softmax cần hoàn tất reduction theo hàng; không thể thay bằng một mode GEMM. Mảng chung cũng không có nghĩa toàn bộ chế độ đạt cùng mức sử dụng DSP hay cùng tần số hữu dụng.

3.5 Bộ đệm nội chip và residual

3.5.1 Half-layer mapping và giữ token gốc

Tách encoder thành hai nửa theo quan hệ dữ liệu của phương trình (1.20): nửa LayerNorm–MSA–residual và nửa LayerNorm–MLP–residual. Hướng tổ chức half-layer mapping được thảo luận trong ViA [13]; sơ đồ dưới đây là cách diễn giải lịch dữ liệu cho thiết kế dự án, không phải bản sao netlist ViA.

Đệm shortcut giữ token đầu vào đến đúng thời điểm nhánh tính toán trả token cùng chỉ số. Hai nửa vẫn có phụ thuộc dữ liệu, không trở thành hai luồng độc lập chỉ vì có hai buffer. Thời điểm giải phóng ô nhớ phải dựa trên lần đọc cuối và handshake thực, không chỉ dựa trên số clock danh nghĩa.

3.5.2 Single-load policy và double S-buffer

ME-ViT nghiên cứu tái sử dụng buffer đa mục đích, giữ kết quả trung gian nội chip và giảm số lần nạp tham số [14]. Chính sách này là mục tiêu có điều kiện về dung lượng và lịch xử lý, không bảo đảm mọi mô hình đều không ghi DRAM trên một FPGA edge nhỏ.

BRAM là bộ nhớ *nội chip*, khác DDR4 ngoài chip. Một tensor được nạp một lần từ DDR vào BRAM vẫn có thể được đọc BRAM nhiều lần. Tương tự, nạp một tile trọng số một lần không đồng nghĩa toàn bộ trọng số mô hình cùng nằm trong BRAM.

Double S-buffer có thể phục vụ tích lũy partial sum của MLP: một bank đang tích lũy, bank kia đang drain. Nó không mặc nhiên là buffer lưu cả output tầng trước cho residual. Cần khai báo riêng tuổi thọ của partial sum, activation và shortcut; cùng dùng LUTRAM không làm chúng có thể ghi đè tùy ý.

3.6 Static Timing Analysis và timing closure

3.6.1 Setup và hold với quy ước skew rõ ràng

Xét hai thanh ghi cùng chu kỳ T_{clk} , đặt $s = t_{\text{capture}} - t_{\text{launch}}$ là skew, U_s, U_h là uncertainty. Mô hình đơn giản cho đường dữ liệu là:

$$S_{\text{setup}} = T_{\text{clk}} + s - U_s - T_{\text{setup}} - T_{\text{cq,max}} - T_{\text{logic,max}} - T_{\text{route,max}}, \quad (3.33)$$

$$S_{\text{hold}} = T_{\text{cq,min}} + T_{\text{logic,min}} + T_{\text{route,min}} - s - U_h - T_{\text{hold}}. \quad (3.34)$$

Dẫn xuất từ thời điểm arrival/required. Chọn cạnh launch làm mốc 0 ns, skew $s \in \mathbb{R}$ là thời điểm clock capture trừ clock launch. Mọi $T_{\text{cq}}, T_{\text{logic}}, T_{\text{route}}, T_{\text{setup}}, T_{\text{hold}}, U_s, U_h$ không âm, $T_{\text{clk}} > 0$; max/min là hai cận khác nhau của cùng loại delay. Với đường setup, dữ liệu đến muộn nhất tại $A_{\text{max}} = T_{\text{cq,max}} + T_{\text{logic,max}} + T_{\text{route,max}}$, phải đến trước $R_s = T_{\text{clk}} + s - U_s - T_{\text{setup}}$. Điều kiện $A_{\text{max}} \leq R_s$ cho $S_{\text{setup}} = R_s - A_{\text{max}} \geq 0$. Với hold, dữ liệu mới đến sớm nhất $A_{\text{min}} = T_{\text{cq,min}} + T_{\text{logic,min}} + T_{\text{route,min}}$ không được trước $R_h = s + U_h + T_{\text{hold}}$, nên $S_{\text{hold}} = A_{\text{min}} - R_h \geq 0$. Không dùng delay max để kết luận hold.

Nếu $U_s = U_h = 0$ và xét một đường, hai biểu thức rút về công thức setup/hold trong đề cương; WNS/WHS của toàn thiết kế phải lấy minimum qua mọi đường tương ứng, không đồng nhất với slack của một đường tùy chọn. Đây là mô hình suy diễn với clock cùng miền; báo cáo STA thực còn xét corner, quan hệ cạnh, clock pessimism và các ràng buộc đường cụ thể [8, 15].

$$\text{WNS} = \min_{p \in \mathcal{P}_{\text{setup}}} S_{\text{setup},p}, \quad \text{WHS} = \min_{p \in \mathcal{P}_{\text{hold}}} S_{\text{hold},p}. \quad (3.35)$$

Mặc dù tên chứa “negative”, WNS có thể dương. Mục tiêu là WNS và WHS không âm trên các đường đã ràng buộc đúng; cần kiểm tra thêm đường chưa ràng buộc, recovery/removal, pulse width và CDC. WNS tốt không chứng minh giao thức AXI hay thuật toán đúng.

3.6.2 Phân biệt cách sửa setup và hold

Mục tiêu thiết kế 200–250 MHz tương ứng chu kỳ 5–4 ns; đây chưa phải F_{max} đo được. Setup xấu có thể cần cắt adder tree, pipeline bộ chia hoặc retiming. Hold xấu liên quan đường dữ liệu quá nhanh/skew và thường cần sửa routing hoặc chèn delay phù hợp; thêm register hoặc hạ clock không phải cách chữa hold tổng quát.

Khi tăng latency một nhánh, phải sửa valid và mọi dữ liệu đi kèm của nhánh đó. Nếu backpressure tạo chuỗi tổ hợp TREADY dài, có thể dùng AXI register slice/skid buffer, nhưng phải kiểm tra lại dung lượng giữ các beat đang bay. Không dùng `set_false_path` để che lỗi timing của đường dữ liệu thực sự hoạt động.

Tự kiểm tra và đối chiếu

1. Bỏ nhánh thứ ba của vector trong Mục 3.2.1 sẽ cho tổng nào? Padding bằng zero khác bỏ một phần tử thật ở điểm nào?
2. Với $a = 12, b = 5, c = -8$, giải mã trường cao mà không bù dấu sai bao nhiêu?
3. Tại cạnh $k + 3$ trong bảng valid, vì sao không được lấy payload làm kết quả mới?

4. Ba tầng thanh ghi có tự bảo đảm 200 MHz không?

Đối chiếu. Bỏ tích -8 cho -8 thay vì -16 ; thêm zero không làm đổi tổng. Trường cao
thô 59 thiếu một đơn vị so với 60. Tại $k + 3$, valid cuối bằng zero. Tần số cần được kiểm
tra bằng constraints, netlist và STA sau triển khai, không suy từ số tầng.

Tóm tắt và chuyển sang hệ thống bộ nhớ

Một PE đúng chưa bảo đảm cả hệ thống nhanh: PE vẫn phải chờ toán hạng và nơi nhận
kết quả. Chương 4 bổ sung nơi lưu tensor, lịch tile, quyền sở hữu bank và handshake. Khi
chuyển từ pipeline không có ready sang hệ thống có backpressure, các quy tắc nhận, giữ
và phát dữ liệu phải được bảo toàn chứ không chỉ nối các cổng dữ liệu.

Chương 4

Phân cấp bộ nhớ và tích hợp SoC thời gian thực

Mục tiêu học tập

- Phân biệt nơi lưu dữ liệu với bộ máy vận chuyển dữ liệu.
- Đếm tile theo đủ ba chiều và tính dung lượng buffer của một cấu hình nhỏ.
- Theo dõi bank, cache và tín hiệu ready để tránh ghi đè dữ liệu đang được dùng.

Từ PE đến luồng dữ liệu. Chương 3 trả lời “tính một kết quả ra sao”. Chương này hỏi “ai giữ toán hạng trước khi PE dùng, ai nhận kết quả sau đó”. Hai câu hỏi có thể quyết định thời gian chờ lớn hơn chính phép nhân. Bản đồ ở trang vi đặt phần này tại ranh giới giữa datapath và SoC.

Pre-check và proof state. Chương này mô tả quyền sở hữu buffer, thứ tự giao dịch và lịch tile. Các chứng minh có điều kiện không biến DMA thành một mức RAM, không biến loopback thành attention, và không suy ra deadline thời gian thực từ số buffer. Phân biệt safety (không ghi đè/mất beat) với liveness (cuối cùng hoàn tất); liveness cần giả thiết nguồn/dịch tiến triển. Chưa có chứng thư Lean hoặc protocol-checker chạy trên RTL.

4.1 Bốn chặng dữ liệu từ lưu trữ đến datapath

4.1.1 Ấn dụ nhà kho, khu tập kết và băng chuyền

Có thể hình dung SD card là kho lưu trữ lâu dài, DDR4 là khu tập kết hàng đã chuẩn bị, DMA là phương tiện chuyển hàng và BRAM là khay nguyên liệu sát bàn thao tác PE. Luồng stream giống băng chuyền có cơ chế báo “chưa nhận thêm”. Mỗi vai trò giải quyết một việc khác nhau: lưu lâu dài, giữ hàng sẵn dùng, vận chuyển, hoặc cấp dữ liệu gần nơi tính.

Ấn dụ dừng ở vai trò đó: DMA không phải một tầng RAM; dữ liệu không tự được copy chỉ vì có địa chỉ; hai khay không tự tạo lịch chồng lấp. Các ràng buộc byte, cache, cổng bộ nhớ và handshake phải được khai báo riêng [5].

4.1.2 Đối chiếu với hệ thống thực

Luồng hệ thống được tổ chức thành:

$$\text{SD card} \longrightarrow \text{DDR4/PS buffer} \longrightarrow \text{AXI DMA} \longrightarrow \text{BRAM/LUTRAM}. \quad (4.1)$$

Đây là bốn *chặng* của đường dữ liệu, không phải bốn mức bộ nhớ đều có chức năng lưu trữ: DMA là bộ máy vận chuyển, không phải cache hoặc RAM lớn.

1. SD card giữ hệ điều hành, checkpoint đã kiểm tra và dữ liệu khởi tạo. Giải mã tệp, kiểm tra shape và nạp trọng số thuộc giai đoạn chuẩn bị, không được đưa vào latency inference nếu benchmark công bố chỉ đo steady state.
2. DDR4 giữ vùng đệm do PS quản lý. CMA là *Contiguous Memory Allocator*, không phải “continuous allocation”. Trong PYNQ, dùng `allocate()` để có buffer cùng địa chỉ device và cơ chế đồng bộ cache [9].
3. AXI DMA chuyển dữ liệu giữa AXI memory-mapped phía DDR và AXI-Stream phía PL. MM2S đọc bộ nhớ ra stream; S2MM ghi stream vào bộ nhớ [5].
4. BRAM/LUTRAM giữ tile gần PE. Cần đủ số bank và cổng đọc/ghi cho lịch tính toán, không chỉ đủ tổng số byte.

4.1.3 Bảng thông cực đại và bảng thông hữu dụng

Đặt $W_b \in 8\mathbb{Z}_{>0}$ bit, $f \in \mathbb{R}_{>0}$ Hz, $C_t \in \mathbb{Z}_{>0}$ chu kỳ quan sát, $C_f \in \{0, \dots, C_t\}$ chu kỳ handshake và $\eta = C_f/C_t \in [0, 1]$. Mỗi beat có tối đa $W_b/8$ byte hữu ích, nên tổng byte không vượt $C_f W_b/8$. Chia cho cửa sổ thời gian C_t/f cho:

$$\text{BW}_{\text{stream}} \leq \frac{W_b}{8} f \eta. \quad (4.2)$$

Ví dụ lý thuyết $W_b = 64$, $f = 200$ MHz, $\eta = 1$ cho 1.6 GB/s theo đơn vị thập phân. Đây không phải số đo DDR4 hay thông lượng hai chiều tổng cộng. Arbitration, burst, refresh, contention với ARM, beat cuối không đầy và backpressure đều có thể giảm băng thông.

4.1.4 Cache coherency và vòng đời buffer

Trước khi PL đọc dữ liệu PS vừa ghi, đồng bộ buffer nguồn ra device; sau khi S2MM hoàn tất, invalidate/sync buffer đích trước khi CPU đọc [9, 16]. Hai buffer không được chồng lấp nếu accelerator đồng thời đọc nguồn và ghi đích. Vùng nhận cũng phải được làm sạch trước khi DMA sở hữu nó để tránh dirty cache line cũ ghi đè dữ liệu DMA.

Không thay thế một đối tượng đã `allocate()` bằng một ndarray mới rồi gửi địa chỉ CPU thông thường cho DMA. Không giải phóng hoặc tái sử dụng buffer khi engine còn có thể truy cập nó. Trong triển khai trên board, việc chờ timeout chỉ báo lỗi phần mềm; trước khi trả CMA phải dừng/reset DMA và xác nhận engine không còn truy cập bộ nhớ.

4.2 Matrix tiling theo đủ ba chiều

Tiền điều kiện: $M, N, K, T_M, T_N, T_K \in \mathbb{Z}_{>0}$, $A \in \text{INT8}^{M \times K}$, $B \in \text{INT8}^{K \times N}$, zero-point bằng không. Xét $C = AB \in \mathbb{Z}^{M \times N}$ trước requantization. Accumulator phần cứng phải thỏa cận của Chương 3, kể cả bias nếu có. Số tile là:

$$N_M = \lceil M/T_M \rceil, \quad N_N = \lceil N/T_N \rceil, \quad N_K = \lceil K/T_K \rceil, \quad (4.3)$$

$$N_{\text{output tiles}} = N_M N_N, \quad N_{\text{tile GEMM}} = N_M N_N N_K. \quad (4.4)$$

Phân hoạch chỉ số. Với hàng $r \in \{0, \dots, M-1\}$, chia Euclid $r = iT_M + \rho$, $0 \leq \rho < T_M$, cho duy nhất $i = \lfloor r/T_M \rfloor < N_M$. Chia cột theo T_N và reduction theo T_K cho cùng tính duy nhất. Vì mỗi phần tử có đúng một chỉ số tile mỗi chiều, các tile phủ ma trận, không chồng phần tử hữu ích. Riêng tensor $X \in \text{INT8}^{N_t \times D}$ với $N_t, D, T_t, T_d \in \mathbb{Z}_{>0}$, số tile lưu trữ là

$$N_{\text{storage}} = \lceil N_t/T_t \rceil \lceil D/T_d \rceil. \quad (4.5)$$

Ví dụ tile 64×32 dùng $(T_t, T_d) = (64, 32)$. Đây là công thức hai chiều trong đề cương, không phải số lần GEMM của ba chiều (M, N, K) .

Công thức tích hai số ceil chỉ đếm tile của một ma trận, không đếm đủ các phép GEMM tích lũy theo chiều K . Với mỗi tile đầu ra:

$$C_{ij}^{\text{acc}} = \sum_{k=0}^{N_K-1} A_{i,k_t} B_{k_t,j}. \quad (4.6)$$

Bất biến partial sum. Với $i \in \{0, \dots, N_M-1\}$, $j \in \{0, \dots, N_N-1\}$, xét tile output (i, j) , định nghĩa $C_{ij}^{(0)} = 0$ và $C_{ij}^{(q+1)} = C_{ij}^{(q)} + A_{i,q} B_{q,j}$, $0 \leq q < N_K$. Quy nạp từ tổng rỗng rồi thêm một hạng cho $C_{ij}^{(q)} = \sum_{u=0}^{q-1} A_{i,u} B_{u,j}$. Với tọa độ hữu ích (ρ, ν) :

$$C_{ij}^{(N_K)}[\rho, \nu] = \sum_{q=0}^{N_K-1} \sum_{v=0}^{T_K-1} A[iT_M + \rho, qT_K + v] B[qT_K + v, jT_N + \nu] \quad (4.7)$$

$$= \sum_{k=0}^{K-1} A[iT_M + \rho, k] B[k, jT_N + \nu]. \quad (4.8)$$

Ở dòng đầu, định nghĩa mọi truy cập padding $k \geq K$ bằng zero. Phân hoạch duy nhất $k = qT_K + v$ cho đẳng thức thứ hai; các hạng padding bằng không. Requantize chỉ sau $q = N_K$ để giữ bất biến tổng nguyên.

Tile biên có kích thước hữu dụng $m_t = \min(T_M, M - iT_M)$, $n_t = \min(T_N, N - jT_N)$ và $k'_t = \min(T_K, K - k_tT_K)$. Padding phải bằng mã zero-point; theo quy ước lượng tử hóa symmetric của dự án, mã đó bằng không. Chỉ xuất các phần tử hữu dụng, không để padding biến thành token mới.

4.2.1 Dung lượng và reuse

Với một byte/mã INT8 và bốn byte/mã INT32, tile A chứa $T_M T_K$ byte, tile B chứa $T_K T_N$ byte, tile accumulator chứa $4T_M T_N$ byte. Hai bank độc lập cho mỗi toán hạng nhân đôi hai hạng đầu. Nếu double-buffer cả hai toán hạng INT8 nhưng chỉ giữ một tile accumulator INT32, cận dung lượng dữ liệu là:

$$B_{\text{tile}} = 2T_M T_K + 2T_K T_N + 4T_M T_N \quad \text{byte}. \quad (4.9)$$

Với $(T_M, T_N, T_K) = (32, 32, 64)$, cận này bằng 12288 byte, chưa tính bias, metadata, residual và padding vật lý của BRAM. Double-buffer cả accumulator thêm $4T_M T_N$ byte. Số block BRAM còn phụ thuộc cấu hình độ rộng, banking và cổng.

Nếu giữ một tile A qua nhiều tile N, lưu lượng nạp A có thể giảm nhưng phải giữ dữ liệu lâu hơn. Nếu chọn thứ tự lặp khác để giữ B, lợi ích chuyển sang reuse trọng số. Tile theo K phải giữ accumulator cùng scale đến tile cuối; requantize từng tile làm thay đổi kết quả. Một bitstream chỉ hỗ trợ nhiều kích thước trong giới hạn descriptor, bộ đếm, SRAM, scale và tập toán tử đã hiện thực. Không suy ra rằng nó tự chạy mọi biến thể MobileViT/ViT chỉ nhờ thay N, D .

4.2.2 Ví dụ: ma trận nhỏ vẫn cần đếm ba chiều

Xét $C = AB$ với $A \in \text{INT}8^{3 \times 5}$, $B \in \text{INT}8^{5 \times 4}$, tích lũy không bias trong INT32. Chọn $(T_M, T_N, T_K) = (2, 3, 2)$. Khi đó:

$$N_M = \lceil 3/2 \rceil = 2, \quad N_N = \lceil 4/3 \rceil = 2, \quad N_K = \lceil 5/2 \rceil = 3, \quad (4.10)$$

$$N_{\text{output tiles}} = 2 \cdot 2 = 4, \quad N_{\text{tile GEMM}} = 2 \cdot 2 \cdot 3 = 12. \quad (4.11)$$

Bốn tile đầu ra không có nghĩa chỉ bốn lần tính: mỗi tile phải cộng ba phần theo chiều K. Cận tích lũy $5 \cdot 16384 = 81920 < 2^{31} - 1$ đủ cho phép tính này không bias.

Dùng chỉ số bắt đầu từ zero; cho riêng hàng $A_{2,:} = (1, 2, 3, 4, 5)$ và cột $B_{:,3} = (5, 4, 3, 2, 1)$. Phần tử góc cuối nhận ba partial sum:

$$s_0 = 1 \cdot 5 + 2 \cdot 4 = 13, \quad (4.12)$$

$$s_1 = 3 \cdot 3 + 4 \cdot 2 = 17, \quad (4.13)$$

$$s_2 = 5 \cdot 1 = 5, \quad C_{2,3} = 13 + 17 + 5 = 35. \quad (4.14)$$

Tile đầu ra cuối có kích thước hữu dụng 1×1 ; lần reduction cuối có một phần tử thật. Padding thêm một cặp zero cho K không đổi s_2 . Các phần tử padding không được xuất thành phần tử ma trận mới.

Double-buffer hai toán hạng, một tile accumulator INT32 theo công thức (4.9) cần:

$$B_{\text{tile}} = 2(2)(2) + 2(2)(3) + 4(2)(3) = 8 + 12 + 24 = 44 \text{ byte}. \quad (4.15)$$

Đây là số byte dữ liệu lý tưởng, không phải dung lượng BRAM vật lý sau banking. Ví dụ minh họa kiến trúc GEMM mục tiêu; shell loopback A.5 chưa thực thi phép nhân này.

4.2.3 Ping-pong là giao thức sở hữu bank

Mỗi bank có vòng đời $\text{EMPTY} \rightarrow \text{FILLING} \rightarrow \text{FULL} \rightarrow \text{COMPUTING} \rightarrow \text{EMPTY}$. Một bank chỉ chuyển thành FULL khi nhận đủ tile hợp lệ; chỉ được ghi lại sau khi mọi consumer đã đọc xong. Nếu hai bank đều bận, nguồn phải nhận backpressure.

Bất biến quyền sở hữu. Với bank $b \in \{0, 1\}$ và cạnh $k \in \mathbb{Z}_{\geq 0}$, đặt $w_b[k], r_b[k] \in \{0, 1\}$ chỉ việc DMA ghi và PE đọc dữ liệu thuộc cùng vùng tile. Profile này yêu cầu $w_b[k] = 1 \Rightarrow \text{state}_b = \text{FILLING}$ và $r_b[k] = 1 \Rightarrow \text{state}_b = \text{COMPUTING}$. Một bank có đúng một state, nên $w_b[k]r_b[k] = 0$. Bộ đếm beat chỉ tăng tại handshake; FILLING chuyển FULL sau khi đủ byte hữu ích, COMPUTING chuyển EMPTY sau consumer cuối. Reset đưa bank về EMPTY và hủy giao dịch cũ; reset RAM không thay thế việc quiesce DMA ngoài chip. Đây là quy tắc hoạt động của bộ lập lịch mục tiêu, không phải state machine đã hiện thực trong shell A.5.

Chứng minh điều kiện ắc tải. Với tile $i \in \{0, \dots, J-2\}$, $J \in \mathbb{Z}_{>0}$, gọi $c_i \in \mathbb{R}_{\geq 0}$ là thời điểm bắt đầu PE tile i , $p_i \geq 0$ thời gian compute, $\ell_{i+1} \geq 0$ thời gian tải tile sau. Giả sử bank sau đã EMPTY, hai tài nguyên hoạt động độc lập, tải tile $i+1$ bắt đầu đúng tại c_i và không có contention ngoài các thời gian đã đo. Khi đó tải xong tại $c_i + \ell_{i+1}$, PE rảnh tại $c_i + p_i$:

$$c_{i+1} = \max(c_i + p_i, c_i + \ell_{i+1}) = c_i + \max(p_i, \ell_{i+1}), \quad (4.16)$$

$$\text{stall}_i = c_{i+1} - (c_i + p_i) = \max(0, \ell_{i+1} - p_i). \quad (4.17)$$

Do đó $\text{stall}_i = 0 \iff \ell_{i+1} \leq p_i$, chính là điều kiện $T_{\text{DMA}}(\text{Tile}_{i+1}) \leq T_{\text{PE}}(\text{Tile}_i)$ dưới các giả thiết lịch trên. Nếu DMA khởi động trễ $d_i \geq 0$, thay ℓ_{i+1} bằng $d_i + \ell_{i+1}$; bất đẳng thức cũ riêng lẻ không còn đủ. Nó cũng không loại bỏ startup/drain hoặc bảo đảm hai giao dịch DDR có thể chạy đồng thời.

Với J tile và lịch đủ điều kiện chồng lấp, thời gian lý tưởng có dạng:

$$t_{\text{pipeline}} \approx t_{\text{load}} + (J-1) \max(t_{\text{load}}, t_{\text{compute}}) + t_{\text{compute}} + t_{\text{drain}}. \quad (4.18)$$

Trong trường hợp đồng nhất $p_i = t_{\text{compute}}$, $\ell_i = t_{\text{load}}$, tải tile đầu tiên bắt đầu ở thời điểm zero, nên $c_0 = t_{\text{load}}$. Lặp quan hệ trên cho $c_{J-1} = t_{\text{load}} + (J-1) \max(t_{\text{load}}, t_{\text{compute}})$. Cộng thời gian compute tile cuối và $t_{\text{drain}} \geq 0$ cho biểu thức đã nêu; dấu xấp xỉ nhắc rằng đây là mô hình chứ chưa là phép đo. Đây là mô hình hai pha có drain cuối; writeback cạnh tranh cùng DDR có thể làm mô hình không còn áp dụng. Ắn được tải vào đòi hỏi $t_{\text{load}} \leq t_{\text{compute}}$ và đủ cổng/bank độc lập. Hai bank không bảo đảm “không bao giờ stall”.

4.2.4 Ví dụ: khi nào hai bank thực sự che được tải vào?

Giả sử ba tile độc lập, tải một tile mất ba clock, tính một tile mất năm clock, DMA và PE có đường truy cập không tranh chấp. Đếm thời gian từ mốc zero, bỏ qua writeback và drain trong mô hình nhỏ này:

Khoảng thời gian	Bộ máy tải	PE
[0, 3)	Nạp tile 1 vào Ping	Chờ tile 1
[3, 6)	Nạp tile 2 vào Pong	Tính tile 1 trong [3, 8)
[6, 8)	Pong đã đầy, chờ	Hoàn tất tile 1
[8, 11)	Nạp tile 3 vào Ping	Tính tile 2 trong [8, 13)
[11, 13)	Ping đã đầy, chờ	Hoàn tất tile 2
[13, 18)	Không còn tile nạp	Tính tile 3

Ping chỉ được ghi tile 3 sau mốc 8, khi PE đã đọc xong tile 1. Lịch hoàn tất tại $3+3 \cdot 5 = 18$ clock thay vì $3(3+5) = 24$ clock nếu tải và tính hoàn toàn nối tiếp. Bất đẳng thức $3 \leq 5$ giúp tải tile kế tiếp kịp, nhưng kết luận chỉ áp dụng dưới giả thiết đường truy cập độc lập. Đây không phải số đo của driver hay bo mạch.

4.2.5 Ví dụ quyền sở hữu buffer trong một phiên loopback

Trong demo thật, không có PE GEMM của ví dụ trên: CPU chuẩn bị payload rồi DMA truyền qua shell. Thứ tự người học cần theo dõi là CPU ghi TX, đồng bộ TX ra device, giao quyền đọc TX/ghi RX cho DMA, chờ hoàn tất, đồng bộ RX về CPU rồi mới đọc.

D.1/D.2 còn làm sạch RX trước khi DMA ghi để tránh dữ liệu cache cũ ghi đè kết quả [9, 16].

Bộ đệm TX/RX phải còn sống trong toàn phiên. Timeout chỉ nói phần mềm chờ quá hạn, không nói DMA đã dừng. Vì vậy nhánh xử lý lỗi phải xác nhận quiescence trước khi free CMA. Hãy đọc driver với câu hỏi “ai đang sở hữu buffer này?”, không chỉ với câu hỏi “đã gọi hàm nào?”.

4.3 Giao diện AXI-Stream và AXI4-Lite

4.3.1 Handshake, byte lane và biên packet

Với $v[k], r[k] \in \{0, 1\}$ là TVALID/TREADY lấy mẫu tại cạnh lên k trong trạng thái không reset, định nghĩa sự kiện chuyển beat bằng $\text{Transfer}[k] \iff (v[k] = 1 \wedge r[k] = 1)$. Không sử dụng TLAST như điều kiện handshake. Một beat chỉ được truyền tại cạnh clock có:

$$\text{fire} = \text{TVALID} \wedge \text{TREADY}. \quad (4.19)$$

Nguồn phải giữ payload và sideband ổn định trong lúc valid bằng một nhưng ready bằng không; không được chờ ready mới phát valid. TLAST là cờ cuối packet, không phải điều kiện thứ ba để mỗi beat được chấp nhận [6].

Stream 64-bit mang tám byte; lane 0 ứng với TDATA[7 : 0]. TKEEP[j] đánh dấu byte lane j hữu dụng. Với payload dài $B > 0$ byte, đặt:

$$N_{\text{beat}} = \lceil B/8 \rceil, \quad r = B - 8(N_{\text{beat}} - 1), \quad \text{TKEEP}_{\text{last}} = 2^r - 1. \quad (4.20)$$

Quy ước của dự án yêu cầu các beat trước cuối có TKEEP bằng 0xFF, byte cuối liên tục từ lane thấp và TLAST trùng beat cuối. Đây là một profile packet cụ thể, chặt hơn các mẫu byte hợp lệ tổng quát mà AXI-Stream có thể biểu diễn.

Bất biến elastic register của A.5. Đặt $q[k] \in \{0, 1\}$ là số beat đang giữ trước cạnh k . Giá trị này chính là valid đầu ra. $a[k], d[k] \in \{0, 1\}$ lần lượt là input/output fire. Readiness cho phép nhận nếu register trống hoặc beat cũ ra cùng cạnh, nên $a[k] \leq 1 - q[k] + d[k]$, và $d[k] \leq q[k]$. Bảng chuyển là:

$a[k]$	$d[k]$	$q[k + 1]$	Tác động payload
0	0	$q[k]$	Giữ nguyên
0	1	0	Beat cũ đã phát
1	0	1	Chốt beat mới
1	1	1	Phát cũ, chốt mới

Mọi trường hợp cho $q[k + 1] = q[k] + a[k] - d[k] \in \{0, 1\}$. Reset có $q[0] = 0$; cộng telescoping cho $\sum_{u=0}^{k-1} a[u] - \sum_{u=0}^{k-1} d[u] = q[k]$. Do đó số beat đã nhận trừ đã phát chính xác bằng số beat đang giữ. Khi stall ($q[k] = 1$, ready bằng zero), readiness đầu vào cũng zero, nên $a[k] = d[k] = 0$; payload, TKEEP, TLAST và valid giữ nguyên tới cạnh sau. FIFO sâu một giữ thứ tự. Nếu đích stall vĩnh viễn, bất biến vẫn đúng nhưng không chứng minh hoàn tất.

Payload byte chỉ được bảo toàn trên packet có framing hợp lệ: source tuân thủ TKEEP/TLAST đúng NBYTES. Với packet lỗi, shell chuẩn hóa sideband và đặt ERROR; không được dùng bất biến số beat để kết luận tất cả byte đầu ra là dữ liệu hợp lệ. DONE được đặt khi beat cuối đầu ra handshake, không phải khi chỉ nhận TLAST đầu vào.

Với attention thực, cần công bố tensor order, chẳng hạn các packet Q, K, V theo head/token/channel, cùng độ dài và scale. Payload ba tensor INT8 có $3Thd_k$ byte; gửi một mình Q không đủ tính attention. Nếu dùng ba lệnh DMA thì có ba biên TLAST, không được coi chúng là một packet duy nhất mà không có parser tương ứng.

4.3.2 AXI4-Lite có năm kênh độc lập

AXI4-Lite gồm AW, W, B, AR và R. AW và W có thể đến ở các chu kỳ khác nhau; slave phải giữ riêng địa chỉ và dữ liệu trước khi thực hiện ghi, rồi trả BVALID/BRESP. Đọc cần ARREADY, RVALID, RDATA và RRESP;WSTRB chọn từng byte được ghi [7].

Bất biến ghép AW/W. Gọi $N_{AW}, N_W, N_C, N_B \in \mathbb{Z}_{\geq 0}$ là số lần nhận địa chỉ, nhận dữ liệu, commit và handshake phản hồi kể từ reset. Với $p_{AW}, p_W, b \in \{0, 1\}$ lần lượt là aw_pending, w_pending và BVALID:

$$N_{AW} - N_C = p_{AW}, \quad N_W - N_C = p_W, \quad N_C - N_B = b. \quad (4.21)$$

Cơ sở reset cho tất cả bằng zero. Handshake AW hoặc W tăng đúng bộ đếm và pending tương ứng; commit chỉ khi cả pending bằng một và BVALID bằng zero, tăng N_C , xóa hai pending và đặt BVALID. Handshake B tăng N_B và xóa BVALID. Readiness không nhận AW/W mới khi pending tương ứng hoặc phản hồi còn giữ, nên không ghi đè giao dịch. Với đọc, $N_{AR} - N_R = RVALID \in \{0, 1\}$ theo cùng lập luận nhận AR/chốt R rồi giữ tới handshake. Assertions kiểm tra ổn định phản hồi; liveness cần master cuối cùng nhận B/R và cung cấp đủ AW/W.

Bản ở Phụ lục A.5 hỗ trợ một giao dịch ghi đang chờ và một phản hồi đọc. Nó không bỏ địa chỉ chỉ vì dữ liệu chưa tới, không bỏ phản hồi khi BREADY/RREADY thấp và không nhận thêm ghi trong lúc phản hồi cũ còn chờ. Các truy cập chưa định nghĩa, không cần hàng 32-bit hoặc ghi cấu hình lúc BUSY trả SLVERR.

4.4 Quy định giao diện và hoạt động của top-level Batch 2

Tên tệp `attention_core_top.sv` được giữ theo cấu trúc dự án, nhưng chức năng hiện thực trong Batch 2 là **loopback kiểm tra đường truyền có điều khiển**. Nó không chứa GEMM, Softmax, LayerNorm hoặc classifier. Hai module toán học ở A.3–A.4 là các khối độc lập, chưa được nối thành attention engine trong top này.

Offset	Thanh ghi	Ý nghĩa
0x00	CONTROL/STATUS	Đọc BUSY[0], DONE[1], ERROR[2]
0x04	NBYTES	Độ dài payload, từ 1 đến $2^{26} - 1$
0x08	CAPABILITY	Chỉ đọc; 1 nghĩa là loopback
0x0C	ID	Chỉ đọc; 0x56495432
0x10	N	Metadata số token, chưa dùng tính toán
0x18	HEAD_DIM	Metadata d_k , chưa dùng tính toán
0x1C	HEADS	Metadata h , chưa dùng tính toán

Ghi bit 0 vào CONTROL để START; ghi bit 1 để xóa DONE/ERROR khi idle. Nếu cả hai bit bằng một thì START ưu tiên. START hợp lệ xóa cờ cũ và chốt độ dài. DONE

là sticky, chỉ đặt khi beat cuối đầu ra đã handshake; không đặt khi beat cuối mới đi vào buffer. NBYTES không tự lấy từ N, d_k, h , vì phiên bản này truyền byte chứ chưa diễn giải tensor.

Một thanh ghi elastic giữ payload, TKEEP và TLAST khi đích stall. Nếu framing sai, ERROR được giữ. Shell tái tạo TKEEP theo số byte dự kiến để không phát quá NBYTES và có thể đóng packet tại biên độ dài dự kiến hoặc TLAST đến sớm. Các lane không hợp lệ ở đầu vào không trở thành dữ liệu tin cậy chỉ vì TKEEP đầu ra đã được chuẩn hóa. Kết quả của packet lỗi không hợp lệ và không được dùng suy luận; host phải kiểm tra cả DMA status lẫn ERROR. Reset toàn miền giao tiếp là cách bỏ một phiên lỗi; không có lệnh abort tùy tiện xóa beat đang bị stall.

Giới hạn trên của NBYTES do shell quy định. Giới hạn transfer thực còn phải nhỏ hơn hoặc bằng giới hạn LENGTH của cấu hình AXI DMA; script kiểm tra metadata HWH trước khi truyền. Một IP có địa chỉ MMIO hợp lệ nhưng thiếu clock/reset vẫn có thể khiến chính lệnh MMIO của PS không hoàn tất: timeout Python không thay thế bảo vệ bus ở cấp SoC.

4.5 Luồng webcam HIL và lộ trình suy luận thực

4.5.1 Phân hoạch mục tiêu khi có mô hình đã huấn luyện

Luồng mục tiêu là camera $\rightarrow$ tiền xử lý PS $\rightarrow$ patch embedding/chuẩn hóa/projection $\rightarrow$ QKV $\rightarrow$ attention PL $\rightarrow$ các block/head còn lại $\rightarrow$ nhãn. Phải cố định checkpoint, crop/resize, thứ tự kênh, normalization, scale và vị trí phân hoạch. LayerNorm/residual cần nằm đúng vị trí của graph, không gom tùy ý tất cả LayerNorm về sau attention.

OpenCV thường trả ảnh màu BGR, do đó cần chuyển rõ sang RGB nếu mô hình dùng RGB [17]. Resize ảnh rồi tạo QKV ngẫu nhiên không phải inference. Patch embedding và projection cần trọng số đã huấn luyện; không thể suy ra chúng chỉ từ kích thước 224×224 .

4.5.2 Demo thực thi được mô tả trong Phụ lục D.1

Script ở Phụ lục D.1 chuyển ảnh camera thành RGB 224×224 , ánh xạ byte u thành mã signed $u - 128$, rồi truyền đi và đối chiếu byte nhận qua loopback. Payload mặc định có 150528 byte, chia hết cho tám. Ảnh nhận chỉ được hiển thị khi toàn bộ byte khớp và status không lỗi. Không có tensor QKV ngẫu nhiên, nhãn dự đoán giả hoặc số đo được gọi nhầm là FPS ViT.

Demo cần overlay `.bit/.hwh` cùng phiên bản, AXI DMA simple mode hai chiều, stream 64-bit và shell đúng ID/CAPABILITY. Script dùng PYNQ để nạp overlay/cấp buffer nhưng điều khiển DMA qua MMIO theo PG021 nhằm có vòng chờ hữu hạn cho giao dịch; không trộn nó với trạng thái nội bộ của driver `transfer()` và `wait()` trong cùng phiên [5, 18].

4.5.3 Quy trình năm bước PS–PL và điều kiện thực hiện

Đặt $H_f, W_f \in \mathbb{Z}_{>0}$, $B_f = 3H_fW_f \leq \min(2^{L_{\text{DMA}}} - 1, 2^{26} - 1)$; L_{DMA} là độ rộng LENGTH cấu hình. Buffer TX/RX là hai `PynqBuffer` shape $(B_f,)$, dtype INT8, địa chỉ device không chồng lấp, căn hàng theo cấu hình DMA và không vượt miền địa chỉ bus. Mỗi bước phải đáp ứng các điều kiện sau:

1. **Capture:** camera trả mảng BGR uint8 shape $(H_c, W_c, 3)$ với $H_c, W_c > 0$; thất bại đọc frame dừng phiên, không phát buffer cũ.
2. **PS prepare:** resize về $(H_f, W_f, 3)$, đổi RGB, ánh xạ từng byte $u \in \{0, \dots, 255\}$ thành $z = u - 128 \in \text{INT8}$ qua trung gian int16. Flatten C-order cho offset $3(rW_f + c) + j$ với $0 \leq r < H_f, 0 \leq c < W_f, 0 \leq j < 3$. Ghi TX, làm sạch cache TX/RX trước khi nhường quyền DMA.
3. **Arm/transfer:** cấu hình NBYTES khi shell idle; arm S2MM trước, START shell, rồi arm MM2S. LENGTH chỉ được ghi sau địa chỉ và trạng thái chạy. Từ khi giao DMA đến khi hoàn tất/quiescent, PS không sửa hay free hai buffer.
4. **PL/receive:** shell loopback truyền beat theo quy tắc của bộ đệm elastic: giữ payload ổn định khi đích chưa sẵn sàng nhận. Host chờ cả hai kênh hoàn tất, không báo lỗi, shell DONE và không ERROR, sau đó kiểm tra đúng số byte S2MM và invalidate RX.
5. **PS verify/release:** so sánh toàn bộ mã RX với TX; chỉ khi bằng nhau mới giải mã $u = z + 128$ và hiển thị. Hậu điều kiện kiểm tra chỉ là bảo toàn byte của frame đã tiền xử lý, không là nhãn ViT. Khi kết thúc hoặc có ngoại lệ, chỉ free CMA sau khi chứng minh DMA đã quiescent bằng reset/HALTED.

Phép giải mã bảo toàn byte vì $(u - 128) + 128 = u$ trên số nguyên và mọi trung gian nằm trong int16. Dữ liệu frame sau resize không bằng frame camera gốc từng pixel; phép so sánh chỉ đối với payload TX. Nhánh cleanup không có quyền suy ra quiescent từ timeout; nếu không thể xác nhận dừng, giữ buffer và chuyển trạng thái fail-stop. Đây là ưu tiên safety, không bảo đảm tiến trình luôn thoát trong thời gian hữu hạn.

4.5.4 Latency, FPS và các giới hạn báo cáo

Đặt các mốc thời gian bằng clock monotonic có độ phân giải cao. Báo cáo ít nhất thời gian truyền/đợi và thời gian vòng xử lý gồm camera, tiền xử lý, kiểm tra và UI. Với $J \in \mathbb{Z}_{>0}$ frame hoàn tất và $t_{\text{last}} > t_{\text{start}}$ theo đơn vị giây:

$$\text{FPS}_{\text{loop}} = \frac{J}{t_{\text{last}} - t_{\text{start}}}. \quad (4.22)$$

Khác với $1/t_{\text{DMA}}$, chỉ tiêu này chứa overhead của host; vẫn chỉ là FPS loopback HIL. Suy luận thời gian thực cần thêm hạn chót, phân vị latency, độ chính xác tác vụ và chính sách frame bị trễ. Batch 2 chưa có bitstream đo trên board, chưa cung cấp kết quả FPS, WNS/WHS hoặc GOPS/W thực nghiệm.

Tự kiểm tra và đối chiếu

1. Vì sao ví dụ 3×5 nhân 5×4 có bốn tile đầu ra nhưng mười hai phép tile GEMM?
2. Nếu ghi tile 3 vào Ping ở mốc 6 trong lịch minh họa, bất biến nào bị phá vỡ?
3. Timeout có cho phép giải phóng buffer ngay không?
4. Với stream 64-bit và payload 9 byte theo profile của dự án, beat cuối có TKEEP nào?

Đối chiếu. Mỗi tile đầu ra có ba đoạn reduction, nên $4 \cdot 3 = 12$. Tại mốc 6, Ping vẫn thuộc PE đến mốc 8; ghi sớm có thể phá dữ liệu đang tính. Chỉ free sau khi DMA được xác nhận dừng truy cập. Payload 9 byte có hai beat, beat cuối dùng một lane thấp nên TKEEP bằng 0x01 và TLAST bằng một.

Tóm tắt và chuyển sang đồng kiểm thử

Bộ nhớ và giao thức thêm các nghĩa vụ mà phép nhân đúng chưa bao phủ: không mất beat, không đọc nhầm token, không ghi đè bank và không free sớm. Chương 5 chuyển các nghĩa vụ này thành lịch kích thích, scoreboard và tiêu chí thất bại. Chỉ sau mô phỏng RTL và nghiệm thu triển khai mới được dùng số đo để kết luận hệ thống đáp ứng yêu cầu thực tế.

Chương 5

Đồng kiểm thử, tích hợp SoC và đánh giá thực nghiệm

Mục tiêu học tập

- Giải thích vai trò driver, monitor và scoreboard bằng một chuỗi giao dịch nhỏ.
- Phân biệt lỗi giá trị, lỗi valid và mất giao dịch, dù chương trình test đều có thể chạy đến cuối.
- Tách kiểm tra reference, mô phỏng RTL, STA và đo trên bo mạch khi báo cáo đồ án.

Từ yêu cầu thiết kế đến phép thử. Sau Chương 4, câu hỏi không chỉ là “đáp án có đúng không” mà còn là “đáp án của mẫu nào, đến lúc nào, có bị mất không”. Hãy xem scoreboard như một sổ giao nhận: ghi phiếu khi nhận công việc, đối chiếu đúng phiếu khi có kết quả, và kiểm tra không còn phiếu thiếu khi kết thúc. Phần sau biến hình ảnh này thành hàng đợi và bất biến đếm.

Vì sao dùng Python, và điều gì chưa thể kết luận?

Driver đặt input, monitor quan sát tín hiệu, còn scoreboard so sánh với reference. Cocotb cho phép viết các thành phần test bằng Python và điều khiển DUT trong simulator [10]. Lợi ích sử dụng ở đây là tái sử dụng tư duy lập trình và reference số học, để sinh viên tập trung vào các lỗi về thời điểm và giao thức. Không vì vậy mà bỏ simulator hay bỏ kiến thức HDL.

Có thể viết testbench và scoreboard tốt bằng SystemVerilog; lựa chọn Python không tự bảo đảm debug nhanh hơn. Muốn báo tỷ lệ tiết kiệm, phải cố định cùng DUT, tập lỗi, tiêu chí hoàn tất, công cụ và kinh nghiệm người thực hiện. Nếu thời gian đo được là $t_{SV} > 0$ và $t_{Python} > 0$, tỷ lệ giảm mới được tính bằng $(t_{SV} - t_{Python})/t_{SV}$. Dự án chưa có phép đo đó, nên không công bố mức giảm 80%.

5.1 Phạm vi, phiên bản và trạng thái chứng minh

Batch 3 hoàn thiện phương pháp, mã kiểm thử và quy trình build cho các khối đã hiện thực, không biến shell loopback thành một attention engine. Tập ghi chú thô và danh mục nguồn là đầu vào biên tập, không phải bằng chứng rằng đã có bitstream hay số đo

trên KV260. Các bảng 408.1 FPS, 2.45 ms, 81.6 GOPS/W và tỷ lệ tài nguyên trong bản nháp không có log đo kèm theo, nên không được dùng làm kết quả thực nghiệm.

Có bốn mức xác nhận độc lập: chứng minh toán học dưới giả thiết; chạy reference số nguyên; chạy simulator với RTL; synthesis/STA và đo bo mạch. Một mức đạt không kéo theo mức kế tiếp. Tài liệu chưa có chứng thư Lean kernel. Assertions là các mệnh đề cần thực thi hoặc chứng minh, không tự là chứng nhận đúng.

Các listing Cocotb nhắm API 2.0.1, dùng `unit="ns"`, `ReadOnly` và biến Makefile `COCOTB_TEST_MODULES` [10, 19, 20]. Phải ghi phiên bản của Python, NumPy, Cocotb và simulator; compiler C++ và commit RTL vào log. Verilator và Icarus có tập con ngôn ngữ/khả năng quan sát khác nhau; wrapper phẳng trong C.1 tránh phụ thuộc truy cập cổng mảng unpacked từ Python. Verilator biên dịch mô hình để mô phỏng [21]; không có căn cứ cho một hệ số tăng tốc cố định 10–100 lần hoặc giảm 80% thời gian xác minh trên dự án này.

5.2 Scoreboard theo giao dịch, không theo vòng phát dữ liệu

5.2.1 Ví dụ: hai phiếu giao nhận và ba lỗi chủ động

Dùng adder tree $N_v = 3$ ở Mục 3.2.1, đã reset, không reset thêm và không có ready. Driver phát A tại cạnh 0 với đáp án -16 , phát B tại cạnh 2 với đáp án 7; các cạnh còn lại là bubble. Với quy ước quan sát output sau cạnh, monitor và scoreboard mong đợi:

Cạnh	Nhận	Output hợp lệ sau cạnh	Hàng đợi sau đối chiếu
0	A	Không	$[A : -16]$
1	Không	Không	$[A : -16]$
2	B	A: -16	$[B : 7]$
3	Không	Không	$[B : 7]$
4	Không	B: 7	$[\]$

Tại cạnh 2, thêm B vào cuối trước khi bỏ A ở đầu; hàng đợi sau đối chiếu còn đúng B. Trong toàn phiên nhận hai, phát hai; thứ tự A rồi B và giá trị khớp reference.

Ba phép gây lỗi có chủ đích giúp hiểu tiêu chí của test, nhưng bảng dưới chưa phải báo cáo đã chạy simulator:

1. **Sai giá trị:** thay output A bằng -15 tại cạnh 2. Lịch valid đúng nhưng $-15 \neq -16$, nên phép so sánh dữ liệu phải thất bại.
2. **Sai thời điểm:** phát A tại cạnh 1 thay vì 2. Hàng đợi có thể vẫn chứa đáp án A, nhưng lịch latency dự kiến chưa cho phép valid; kiểm tra chỉ giá trị sẽ bỏ sót lỗi này.
3. **Mất giao dịch:** A đúng nhưng không bao giờ phát B. Cuối ngân sách drain hữu hạn, còn $[B : 7]$, số nhận là hai và số phát là một; test phải thất bại thay vì chờ vô hạn.

Nếu có reset giữa hai sự kiện, mẫu đang bay được hủy theo quy tắc reset đã nêu và scoreboard mở epoch mới. Không dùng quy tắc “mọi input đều phải có output” xuyên qua reset mà bỏ điều kiện hủy. Với AXI-Stream, chỉ enqueue khi valid AND ready; dữ liệu

được giữ nhiều clock do backpressure vẫn chỉ là một beat chưa nhận, không phải nhiều giao dịch.

5.2.2 Miền, tiền điều kiện và hậu điều kiện

Đặt tập mã signed w bit là $\mathbb{I}_w = \{z \in \mathbb{Z} : -2^{w-1} \leq z \leq 2^{w-1} - 1\}$ với $w \in \mathbb{Z}_{>0}$. Tại cạnh $k \in \mathbb{Z}_{\geq 0}$ không reset, $a[k], d[k] \in \{0, 1\}$ là sự kiện nhận/phát giao dịch. Với stream, sự kiện bằng valid AND ready; với pipeline không ready, sự kiện nhận bằng in_valid. Golden function $g : \mathcal{X} \rightarrow \mathcal{Y}$ phải cùng miền, scale, round, saturation và layout với DUT.

Gọi Q_k là hàng đợi các cặp $(id, g(x))$ đã nhận nhưng chưa phát. Khi nhận, nối một phần tử vào cuối; khi phát, so sánh và bỏ phần tử đầu. Tiền điều kiện kiểm tra một output là hàng đợi không rỗng. Nếu pipeline có latency bằng không theo khoảng cách cạnh ($N_v = 1$), thêm mẫu nhận ở cạnh đó trước khi kiểm tra output sau cạnh. Hậu điều kiện bit-exact của một phiên hoàn tất là:

$$N_{\text{out}} = N_{\text{in}}, \quad y_j = g(x_j), \quad j \in \{0, \dots, N_{\text{in}} - 1\}, \quad Q_{\text{end}} = \emptyset. \quad (5.1)$$

Ba điều kiện kiểm tra cả giá trị, số lượng và thứ tự. Kiểm tra đúng vài output nhưng bỏ mất output trong lúc driver còn phát không thỏa hậu điều kiện.

5.2.3 Bất biến đếm và reset

Trong một epoch bắt đầu sau reset, đặt $A_k = \sum_{u < k} a[u]$, $D_k = \sum_{u < k} d[u]$. Cơ sở hàng đợi rỗng cho $|Q_0| = 0 = A_0 - D_0$. Enqueue tăng cả A và độ dài một; dequeue hợp lệ tăng D và giảm độ dài một; hai sự kiện cùng cạnh triệt tiêu thay đổi độ dài. Do đó:

$$|Q_k| = A_k - D_k \geq 0. \quad (5.2)$$

Reset hủy các giao dịch đang bay và bắt đầu epoch mới; không so sánh output sau reset với mẫu bị hủy trước reset. Với adder tree có $N_v \in \mathbb{Z}_{>0}$ phần tử, $L_a = \lceil \log_2 N_v \rceil$ và output sau cạnh $k + L_a$ ứng với mẫu ở cạnh k , theo quy ước Chương 3. Test theo lịch cạnh còn phát hiện valid sớm/muộn ngay cả khi thứ tự giá trị vẫn đúng.

5.2.4 Tránh race và đợi vô hạn

Test điều khiển clock thủ công: đưa clock về zero, drive input, chờ nửa chu kỳ để combinational ổn định, lấy snapshot handshake ngay trước cạnh lên, tạo cạnh lên, rồi ReadOnly để quan sát trạng thái sau nonblocking. Không drive tín hiệu trong pha ReadOnly. Mỗi bước tiếp theo bắt đầu bằng Timer để rời pha này [19]. Không chờ hết phát vector mới bắt đầu monitor; driver và kiểm tra cùng chạy ở từng bước clock.

Mọi vòng poll có ngân sách hữu hạn $B_{\text{cycles}} \in \mathbb{Z}_{>0}$. Timeout báo thất bại của kịch bản dưới lịch ready đã chọn, không chứng minh DUT sai với mọi môi trường. Test AXI tạo AW/W đến lệch nhau, giữ B/R bị stall, thử byte strobe, packet không đầy beat cuối, backpressure, malformed TLAST và reset khi còn output. Liveness của thiết kế cần giả thiết nguồn cuối cùng cung cấp dữ liệu và đích cuối cùng ready.

5.3 Độ chính xác số học và định dạng vector

5.3.1 Bit-exact khác sai số xấp xỉ

Golden của B.3 chỉ dùng số nguyên cho các unit hiện có và trả bản sao payload cho loopback. Nó không gọi `np.exp` rồi đặt tên integer Softmax. Chưa có RTL Softmax, QKV và GEMM tích hợp, nên không cung cấp output attention giả làm đáp án bit-exact cho shell. QKV xuất ở B.2 là đầu vào cho công việc tích hợp sau, không có tệp `golden_output.hex` attention đi kèm.

Với $n \in \mathbb{Z}_{>0}$ phần tử, mã $q_j \in \mathbb{I}_w$, scale $s_j \in \mathbb{R}_{>0}$, zero-point $z_j \in \mathbb{Z}$ và reference thực $y_j \in \mathbb{R}$ hữu hạn, phải tái dựng $\hat{y}_j = s_j(q_j - z_j)$ trước khi đo:

$$e_j = \hat{y}_j - y_j, \quad (5.3)$$

$$\text{MSE} = \frac{1}{n} \sum_{j=0}^{n-1} e_j^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad (5.4)$$

$$E_s = \sum_{j=0}^{n-1} y_j^2, \quad E_e = \sum_{j=0}^{n-1} e_j^2, \quad (5.5)$$

$$\text{SNR}_{\text{dB}} = 10 \log_{10}(E_s/E_e) \quad (E_s > 0, E_e > 0). \quad (5.6)$$

Nếu $E_e = 0 < E_s$, quy ước SNR bằng $+\infty$; nếu $E_s = 0 < E_e$, bằng $-\infty$; nếu cả hai zero, báo không xác định thay vì chia 0/0. Không có bất đẳng thức MSE nhỏ hơn một ngưỡng bất kỳ nếu chưa đặt ngưỡng và đo/chứng minh trên miền đầu vào. MSE bằng zero với reference nguyên chỉ xác nhận datapath, không chứng minh accuracy của mô hình FP32.

5.3.2 Chứng minh mã hex bù hai

Với $q \in \mathbb{I}_w$, đặt $u = q \bmod 2^w \in \{0, \dots, 2^w - 1\}$. Nếu $q \geq 0$ thì $u = q$; nếu $q < 0$ thì $u = q + 2^w$. Giải mã $q' = u$ khi $u < 2^{w-1}$, ngược lại $q' = u - 2^w$, cho $q' = q$ ở cả hai trường hợp. INT8 cần một byte, tức hai chữ số hex; không gọi là “hex 2-byte”. INT32 cần tám chữ số. Một dòng là một phần tử, C-order, không ghép nhiều byte thành một word DMA trừ khi manifest nói rõ. Manifest lưu shape, scale, kiểu, SHA-256 và round rule. Nạp bằng `$readmemh` vào mảng có đúng độ rộng phần tử; endianness của packing 64-bit là lớp giao thức khác.

5.4 STA, đóng gói IP và triển khai PYNQ

Mô phỏng chức năng không có delay routing nên không chứng minh 200 MHz. Với một đường đã ràng buộc đúng, dùng delay max cho setup, min cho hold, skew s và uncertainty $U_s, U_h \geq 0$:

$$S_s = T_{\text{clk}} + s - U_s - T_{\text{setup}} - (T_{\text{cq,max}} + T_{\text{logic,max}} + T_{\text{route,max}}), \quad (5.7)$$

$$S_h = T_{\text{cq,min}} + T_{\text{logic,min}} + T_{\text{route,min}} - s - U_h - T_{\text{hold}}, \quad (5.8)$$

$$\text{WNS} = \min_{p \in \mathcal{P}_s} S_{s,p}, \quad \text{WHS} = \min_{p \in \mathcal{P}_h} S_{h,p}. \quad (5.9)$$

Giả thiết $\mathcal{P}_s, \mathcal{P}_h$ không rỗng; các miền thời gian và dẫn xuất ở Chương 3. Điều kiện cần là hai minimum không âm; không bỏ qua unconstrained paths, recovery/removal, pulse

width hoặc CDC [8, 22]. Số logic level lớn không tự là lỗi, và chèn register không tự sửa hold. Sửa kiến trúc phải cập nhật latency/valid và chạy lại regression.

E.1 có hai chế độ: synthesis out-of-context để khảo sát core, và build một project SoC đã được người tích hợp tạo/kiểm tra. Một top có chân AXI rồi không tự có PS, DDR, AXI DMA, clock/reset hay pin constraints KV260. E.2 chỉ là ngân sách timing cho thử nghiệm OOC, không phải XDC bo mạch. Overlay cần block design phù hợp, tên IP và địa chỉ như D.1/D.2; tệp bit và HWH cùng tên gốc [23]. Thiếu project SoC là tiền điều kiện build chưa thỏa, không được xuất tệp bit rỗng hoặc giả HWH.

5.5 Đo hiệu năng và bảng trạng thái bằng chứng

Với $J \in \mathbb{Z}_{>0}$ frame hoàn tất trong $\Delta t > 0$ giây, $C_{\text{MAC}} \in \mathbb{Z}_{\geq 0}$ MAC thực sự thực thi mỗi frame và công suất trung bình $P > 0$ watt:

$$\text{FPS} = J/\Delta t, \quad (5.10)$$

$$\text{GOPS} = \frac{2JC_{\text{MAC}}}{10^9 \Delta t}, \quad (5.11)$$

$$\text{GOPS/W} = \frac{2JC_{\text{MAC}}}{10^9 \Delta t P}. \quad (5.12)$$

Hệ số hai là quy ước hai operation/MAC; công thức chỉ áp dụng khi đếm đúng phần workload được đo, không suy từ FPS loopback. Với $t_j > 0$ là latency frame j , báo trung bình, phân vị và deadline miss; $1/t_j$ không tự là throughput khi có pipeline/chồng lắp. Công suất phải ghi rõ toàn board hay PL, baseline idle, phương tiện đo và cửa sổ lấy mẫu. Dung lượng chip và utilization lấy từ đúng part/project, không sao chép một bảng tài nguyên thiếu provenance.

Hạng mục	Trạng thái tài liệu
Golden unit số nguyên, vector hex Cocotb và wrapper RTL	Đã chạy kiểm tra Python cục bộ Có mã; chưa chạy simulator trong môi trường này
Driver DMA	Có mã; kiểm tra mock không thay bo mạch
Core attention hoàn chỉnh	Chưa hiện thực; A.5 là loopback
Vivado synthesis/STA/bitstream	Chưa chạy; cần công cụ và project SoC
FPS, accuracy, GOPS/W KV260	Chưa đo; không công bố số giả định

Trong lần kiểm tra biên tập Batch 3 ngày 14/09/2026, đã chạy round-trip toàn bộ 256 mã INT8 và các biên INT32; kiểm tra exporter dot với $N_v = 1, 3, 16, 17$, loopback 129 byte và QKV input-only. Reference căn nguyên và DSP packing được đối chiếu thêm với 10000 mẫu ngẫu nhiên mỗi loại. Logic scoreboard adder và AXI được chạy với mô hình Python thay cho simulator; driver DMA được chạy với MMIO/CMA giả lập, gồm chuyển lắp, lỗi dữ liệu và thất bại cấp phát. Các kiểm tra này không thực thi RTL hay truy cập bo mạch và không thay thế Cocotb chạy trên HDL.

Tự kiểm tra và đối chiếu

1. Nếu chỉ so sánh giá trị mà không kiểm tra lịch valid, lỗi nào của Mục 5.2.1 có thể bị bỏ sót?

2. Khi mất B, hai tiêu chí độc lập nào cho thấy test chưa đạt?
3. Vì sao không so MSE trực tiếp giữa mã INT8 và giá trị thực nếu chưa giải lượng tử?
4. Một bộ test Python chạy trên mock có chứng minh RTL đạt timing không?

Đối chiếu. Lỗi valid sớm/muộn có thể vẫn khớp thứ tự giá trị. Mất B để lại hàng đợi chưa rỗng và số nhận khác số phát. Mã và giá trị thực khác đơn vị; cần scale và zero-point. Mock kiểm tra logic ở mức mô hình, không thực thi netlist và không thay STA.

Tóm tắt và lộ trình thực hành tiếp theo

Bắt đầu từ ví dụ nhỏ đã tính tay, chạy reference, rồi chạy test trên HDL với phiên bản công cụ đã chốt. Khi thay đổi pipeline, cập nhật lịch mong đợi thay vì chỉ sửa số đầu ra. Các listing ở Phụ lục B/C giúp tái lập kiểm tra; Phụ lục D/E hướng dẫn bước hệ thống sau đó. Chỉ nâng mức kết luận khi có bằng chứng của chính mức kiểm tra đó.

Kết luận và hướng phát triển

Tài liệu thiết lập quy định tính toán và lịch xử lý pipeline cho các unit INT8, chứng minh bù dẫu DSP packing, mô tả quyền sở hữu buffer và cung cấp shell loopback cùng suite kiểm thử/build. Nó không kết luận đã triển khai thành công ViT end-to-end, ShiftGELU đã đo 0 DSP, toàn bộ LayerNorm có RTL, hoặc mọi quy mô mô hình dùng chung một bitstream.

Bước nghiệm thu tiếp theo là chạy Cocotb trên hai simulator, giải quyết khác biệt elaboration và assertion, tích hợp GEMM/Softmax cùng giao thức QKV đã chốt, rồi build SoC và đo đúng workload. Chỉ sau khi có log và artifact tái lập mới điền kết quả hiệu năng. Mở rộng INT4, Swin hoặc sparse attention cần quy định biểu diễn, xử lý và mô hình tham chiếu riêng, không kế thừa tự động chứng minh INT8 này.

Transformer gốc [24], ViT [1], I-ViT [3] và các công trình FPGA đã trích dẫn cung cấp cơ sở nghiên cứu; không chuyển số đo của họ sang hệ thống hiện tại. Metadata BibTeX dùng nguồn đã đối chiếu, không chép các mục sai venue/năm từ bản nháp hoặc đưa toàn bộ danh sách chưa kiểm chứng vào bibliography.

Phụ lục A

Bộ mã nguồn SystemVerilog RTL

Phụ lục này chứa hai module của Batch 1 và ba module của Batch 2. Mỗi listing là nội dung đầy đủ của tệp mang tên trong tiêu đề; không ghép các khai báo module vào phần thân lý thuyết. Hai module của Batch 1 nhận dữ liệu theo `in_valid`, không có backpressure. Dữ liệu khi valid bằng không được bỏ qua, nhưng thanh ghi vẫn tiến theo clock. Reset bất đồng bộ mức thấp xóa pipeline; khi tích hợp SoC cần bảo đảm nhả reset đồng bộ với miền clock.

A.1 ShiftGELU năm tầng: `shift_gelu_pipeline.sv`

A.1.1 Quy định đầu vào, đầu ra và phân tầng

Đầu vào/ra là INT8 có dấu với $S_{\text{in}} = S_{\text{out}} = 1/16$. Module cố định tám bit và năm tầng; không giữ tham số `LATENCY` hay `DATA_WIDTH` không thực sự điều khiển đầy đủ datapath. Khi cần định dạng khác, phải thiết kế lại hằng số `ShiftExp` và kiểm tra lại miền giá trị.

Tầng	Dữ liệu chính được chốt	Dữ liệu đi kèm
0	p , 16 bit có dấu	Mẫu gốc INT8
1	E , 13 bit không dấu	Mẫu gốc và dấu của p
2	G , 9 bit không dấu	Mẫu gốc
3	$I_x G$, 18 bit có dấu	Valid
4	Kết quả INT8 sau dịch/kep	Valid đầu ra

Proof state của pipeline. Đặt $x[k] \in \text{INT8}$ và $v[k] \in \{0, 1\}$ là dữ liệu/valid ngay trước cạnh nhận $k \in \mathbb{Z}_{\geq 0}$; dùng trạng thái thanh ghi ngay sau cạnh và giả sử không reset trong cửa sổ đang xét. Gọi f_p, f_E, f_G là các hàm số nguyên ở Chương 2, trong đó f_E nhận p qua $\delta = -|p|$ và f_G nhận p, E . Quy nạp theo các phép gán nonblocking cho:

$$p_0[k] = f_p(x[k]), \quad x_0[k] = x[k], \quad (\text{A.1})$$

$$E_1[k+1] = f_E(p_0[k]), \quad x_1[k+1] = x[k], \quad (\text{A.2})$$

$$G_2[k+2] = f_G(p_0[k], E_1[k+1]), \quad x_2[k+2] = x[k], \quad (\text{A.3})$$

$$P_3[k+3] = x[k]G_2[k+2], \quad (\text{A.4})$$

$$y[k+4] = \text{sat}_8(\mathcal{F}_8(P_3[k+3])). \quad (\text{A.5})$$

Dấu của $p_0[k]$ được chốt cùng $E_1[k+1]$, nên f_G không đọc dấu của mẫu kế tiếp. Với stage $j \in \{0, \dots, 4\}$, quy nạp $v_j[k+j] = v[k]$: stage 0 chốt $v[k]$, stage sau đọc stage trước ở cạnh

cũ. Vì dữ liệu không gated theo valid, bubble không thay đổi offset. Reset làm $v_j = 0$ và hủy giao dịch chưa hoàn tất. Đây là chứng minh trên mô hình thanh ghi hai trạng thái; mỗi trường phải bảo đảm input/control đã biết, setup/hold và nhả reset hợp lệ. Không khẳng định mô phỏng HDL đã được chạy.

Bộ chia tính chính xác $\lfloor 256U/V \rfloor$ với $0 \leq U \leq V$. Vòng lặp chín bước trong function là logic tổ hợp được triển khai song song theo cấu trúc nối tiếp, không phải chín chu kỳ. Bộ nhân shift-add cũng là logic tổ hợp trước thanh ghi tầng 3. Các lựa chọn này ưu tiên mô tả LUT, chưa được chứng nhận timing hay tài nguyên trên FPGA.

```

1  `timescale 1ns / 1ps
2
3  // PRE: known INT8 input on in_valid; input/output scale = 1/16.
4  // PRE: receiver accepts every valid output; no ready port.
5  // RESET: asynchronous assertion; synchronous release by integrator.
6  // POST: output = sat8(floor(input * gain / 256)), gain in [0,256].
7  // INV: accepted edge k -> output after edge k+4 without reset.
8  // INV: bubbles advance each edge; reset discards pending samples.
9  // TIMING: II=1 is conditional on implemented setup/hold closure.
10 module shift_gelu_pipeline (
11     input  logic unsigned [0:0] clk,
12     input  logic unsigned [0:0] rst_n,
13     input  logic unsigned [0:0] in_valid,
14     input  logic signed  [7:0] i_in,
15     output logic unsigned [0:0] out_valid,
16     output logic signed  [7:0] i_out
17 );
18     logic signed [15:0] input_extended;
19     logic signed [15:0] power_stage0;
20     logic signed [7:0] sample_stage0;
21     logic signed [7:0] sample_stage1;
22     logic signed [7:0] sample_stage2;
23     logic unsigned [12:0] exponent_stage1;
24     logic unsigned [0:0] negative_stage1;
25     logic unsigned [8:0] gain_stage2;
26     logic signed [17:0] product_stage3;
27     logic signed [17:0] scaled_product;
28     logic unsigned [4:0] valid_pipe;
29
30     // PRE: delta in [-216,0]. POST: result in [0,4096].
31     // INV: J in [-310,0], whole in [0,19], remainder in [-15,0].
32     function automatic logic unsigned [12:0] shift_exp_negative(
33         input logic signed [15:0] delta
34     );
35         logic signed [15:0] base2_power;
36         logic signed [15:0] magnitude;
37         logic signed [15:0] remainder;
38         logic unsigned [4:0] whole;
39         logic signed [31:0] fraction;
40         logic unsigned [31:0] shifted;
41         begin
42             base2_power = delta + (delta >>> 1)
43                 - (delta >>> 4);

```

```

44     magnitude = -base2_power;
45     whole = magnitude[8:4];
46     remainder = base2_power
47         + $signed({7'b0, whole, 4'b0});
48     fraction = 32'sd4096
49         + ($signed({16{remainder[15]}},
50             remainder}) <<< 7);
51     shifted = $unsigned(fraction) >> whole;
52     shift_exp_negative = shifted[12:0];
53 end
54 endfunction
55
56 // PRE: 0 <= numerator <= 4096, 4096 <= denominator <= 8192.
57 // INV: after i steps,  $2^i U = 2VQ_i + R_i$ ,  $0 \leq R_i < 2V$ .
58 // POST: floor(256*U/V) in [0,256]; nine combinational steps.
59 function automatic logic unsigned [8:0] sigmoid_ratio(
60     input logic unsigned [12:0] numerator,
61     input logic unsigned [13:0] denominator
62 );
63     logic unsigned [14:0] remainder;
64     logic unsigned [8:0] quotient;
65     begin
66         remainder = {2'b0, numerator};
67         quotient = '0;
68         for (int signed bit_index = 8; bit_index >= 0;
69             bit_index = bit_index - 1) begin
70             if (remainder >= {1'b0, denominator}) begin
71                 remainder = remainder
72                     - {1'b0, denominator};
73                 quotient[bit_index] = 1'b1;
74             end
75             remainder = remainder << 1;
76         end
77         sigmoid_ratio = quotient;
78     end
79 endfunction
80
81 // PRE: sample in [-128,127], gain in [0,256].
82 // INV: prefix sum = sample * processed gain prefix.
83 // POST: exact product in [-32768,32512], held in signed 18 bits.
84 function automatic logic signed [17:0] multiply_gain(
85     input logic signed [7:0] sample,
86     input logic unsigned [8:0] gain
87 );
88     logic signed [17:0] extended_sample;
89     logic signed [17:0] result;
90     begin
91         extended_sample = {{10{sample[7]}}, sample};
92         result = '0;
93         for (int signed bit_index = 0; bit_index < 9;
94             bit_index = bit_index + 1) begin

```

```

95         if (gain[bit_index])
96             result = result
97                 + (extended_sample <<< bit_index);
98         end
99         multiply_gain = result;
100     end
101 endfunction
102
103 assign input_extended = {{8{i_in[7]}}}, i_in};
104 assign scaled_product = product_stage3 >>> 8;
105 assign out_valid = valid_pipe[4];
106
107 always_ff @(posedge clk or negedge rst_n) begin
108     if (!rst_n) begin
109         power_stage0 <= '0;
110         sample_stage0 <= '0;
111         sample_stage1 <= '0;
112         sample_stage2 <= '0;
113         exponent_stage1 <= '0;
114         negative_stage1 <= 1'b0;
115         gain_stage2 <= '0;
116         product_stage3 <= '0;
117         i_out <= '0;
118         valid_pipe <= '0;
119     end else begin
120         valid_pipe <= {valid_pipe[3:0], in_valid};
121         power_stage0 <= input_extended
122             + (input_extended >>> 1)
123             + (input_extended >>> 3)
124             + (input_extended >>> 4);
125         sample_stage0 <= i_in;
126
127         exponent_stage1 <= shift_exp_negative(
128             power_stage0[15] ? power_stage0
129                 : -power_stage0);
130         negative_stage1 <= power_stage0[15];
131         sample_stage1 <= sample_stage0;
132
133         gain_stage2 <= sigmoid_ratio(
134             negative_stage1 ? exponent_stage1
135                 : 13'd4096,
136             14'd4096 + {1'b0, exponent_stage1});
137         sample_stage2 <= sample_stage1;
138
139         product_stage3 <= multiply_gain(
140             sample_stage2, gain_stage2);
141
142         if (scaled_product > 18'sd127)
143             i_out <= 8'sd127;
144         else if (scaled_product < -18'sd128)
145             i_out <= 8'sh80;

```

```

146         else
147             i_out <= scaled_product[7:0];
148         end
149     end
150 endmodule

```

Listing A.1: shift_gelu_pipeline.sv: fixed-scale integer ShiftGELU

A.1.2 Kiểm tra số học và giới hạn xác nhận

Quét đều toàn bộ 256 mã $I_x \in [-128, 127]$ tại scale $1/16$, đánh giá phương trình (2.60) bằng số nguyên và so với $x\Phi(x)$ tính bằng SciPy. Sai số đo theo đơn vị số thực ở đầu ra; không bao gồm lượng tử hóa một đầu vào FP32 tùy ý trước khi vào miền $Q_{4.4}$.

Chỉ tiêu của mô hình số học	Giá trị
Sai số tuyệt đối lớn nhất	0.09360277
Sai số tuyệt đối trung bình	0.02481350
Cần sai số bình phương trung bình	0.03745040
Mã đầu vào tại sai số lớn nhất	35

Các vector biên của mô hình là $-128 \mapsto 0$, $-16 \mapsto -3$, $0 \mapsto 0$, $16 \mapsto 13$ và $127 \mapsto 127$. Các số này chỉ xác nhận biểu thức số học đã chọn, **chưa xác nhận mô phỏng RTL**, synthesis, 0 DSP hay độ chính xác phân loại. Khi có simulator, scoreboard phải thử luồng liên tục, bubble xen kẽ, reset giữa luồng và đối chiếu đầu ra tại cạnh $k+4$ với mẫu nhận ở cạnh k .

A.2 Residual dyadic: requant_residual_add.sv

Module có ba tầng: nhân/chốt hệ số, dịch, cộng/kep. Mẫu nhận tại cạnh k xuất hiện ở đầu ra sau cạnh $k+2$. M_F, M_X là số nguyên không âm nằm trong miền INT32 có dấu; $e_F, e_X \in [0, 31]$. Có thể đổi hệ số mỗi mẫu vì số mũ được chốt cùng dữ liệu.

Bất biến ghép hệ số. Gọi $F[k], X[k] \in \text{INT8}$, $M_F[k], M_X[k] \in \{0, \dots, 2^{31} - 1\}$ và $e_F[k], e_X[k] \in \{0, \dots, 31\}$ là một giao dịch. Sau cạnh k , stage 0 giữ $F[k]M_F[k]$, $X[k]M_X[k]$ và chính $e_F[k], e_X[k]$. Sau cạnh $k+1$, stage 1 giữ floor của hai tích này theo hai số mũ đó; sau cạnh $k+2$, thanh ghi đầu ra giữ saturation của tổng. Không có truy cập số mũ runtime mới trong stage 1. Tín hiệu valid ba bit cho $v_{\text{out}}[k+2] = v_{\text{in}}[k]$ khi không reset, bằng cùng quy nạp như A.1. Điều kiện ghép đúng token là giả thiết đầu vào, không phải chức năng mà module tự chứng minh cho hệ thống.

SCALE_LOCKED=1 là lựa chọn lúc elaboration, không phải cổng runtime. Nó giữ nguyên giao diện và độ trễ nhưng bỏ hai phép nhân, bỏ qua các cổng hệ số. Trong cả hai chế độ, nguồn chịu trách nhiệm ghép đúng hai token, receiver không được stall và chỉ đọc kết quả khi out_valid=1.

```

1  'timescale 1ns / 1ps
2
3  // PRE: matching tokens i_f/i_x; receiver never stalls.
4  // PRE: unlocked m_f,m_x in [0,2147483647], e_f,e_x in [0,31].
5  // PRE: locked mode requires equal physical input/output scales.
6  // POST: sat8(floor(i_f*m_f/2^e_f) + floor(i_x*m_x/2^e_x)).

```

```

7 // POST: locked mode returns sat8(i_f+i_x), ignoring M/e ports.
8 // INV: accepted edge k -> output after edge k+2 without reset.
9 // INV: exponents travel with products, including through bubbles.
10 // WIDTH: 40-bit products/shifts, 41-bit sum before saturation.
11 module requant_residual_add #(
12     parameter bit unsigned [0:0] SCALE_LOCKED = 1'b0
13 ) (
14     input  logic unsigned [0:0] clk,
15     input  logic unsigned [0:0] rst_n,
16     input  logic unsigned [0:0] in_valid,
17     input  logic signed  [7:0] i_f,
18     input  logic signed  [31:0] m_f,
19     input  logic unsigned [4:0] e_f,
20     input  logic signed  [7:0] i_x,
21     input  logic signed  [31:0] m_x,
22     input  logic unsigned [4:0] e_x,
23     output logic unsigned [0:0] out_valid,
24     output logic signed  [7:0] i_out
25 );
26     logic signed [39:0] product_f_stage0;
27     logic signed [39:0] product_x_stage0;
28     logic unsigned [4:0] exponent_f_stage0;
29     logic unsigned [4:0] exponent_x_stage0;
30     logic signed [39:0] scaled_f_stage1;
31     logic signed [39:0] scaled_x_stage1;
32     logic signed [40:0] sum_full;
33     logic unsigned [2:0] valid_pipe;
34
35     assign sum_full = $signed({scaled_f_stage1[39],
36                               scaled_f_stage1})
37         + $signed({scaled_x_stage1[39],
38                   scaled_x_stage1});
39     assign out_valid = valid_pipe[2];
40
41     always_ff @(posedge clk or negedge rst_n) begin
42         if (!rst_n) begin
43             product_f_stage0 <= '0;
44             product_x_stage0 <= '0;
45             exponent_f_stage0 <= '0;
46             exponent_x_stage0 <= '0;
47             scaled_f_stage1 <= '0;
48             scaled_x_stage1 <= '0;
49             i_out <= '0;
50             valid_pipe <= '0;
51         end else begin
52             valid_pipe <= {valid_pipe[1:0], in_valid};
53             if (SCALE_LOCKED) begin
54                 product_f_stage0 <= {{32{i_f[7]}}}, i_f;
55                 product_x_stage0 <= {{32{i_x[7]}}}, i_x;
56                 exponent_f_stage0 <= '0;
57                 exponent_x_stage0 <= '0;

```

```

58     end else begin
59         product_f_stage0 <= $signed(i_f)
60             * $signed(m_f);
61         product_x_stage0 <= $signed(i_x)
62             * $signed(m_x);
63         exponent_f_stage0 <= e_f;
64         exponent_x_stage0 <= e_x;
65     end
66
67     scaled_f_stage1 <= product_f_stage0
68         >>> exponent_f_stage0;
69     scaled_x_stage1 <= product_x_stage0
70         >>> exponent_x_stage0;
71
72     if (sum_full > 41'sd127)
73         i_out <= 8'sd127;
74     else if (sum_full < -41'sd128)
75         i_out <= 8'sh80;
76     else
77         i_out <= sum_full[7:0];
78     end
79 end
80 endmodule

```

Listing A.2: requant_residual_add.sv: full-width products and aligned exponents

Kiểm thử tích hợp cần bao gồm $e = 0$, $e = 31$, $M = 0$, $M = 2^{31} - 1$, hai nhánh khác dấu, kết quả vượt hai biên và hệ số đổi liên tiếp. Trường hợp $I_F = 127$, $M_F = 2^{31} - 1$, $e_F = 0$, $I_X = 0$ phải kẹp về 127; nó phát hiện lỗi cắt tích về 32 bit trước khi kẹp. Kiểm tra mô hình số học bằng Python đã quét toàn bộ 65 536 cặp INT8 ở chế độ scale-locking và 100 000 trường hợp dyadic ngẫu nhiên với seed 20260914, đối chiếu phép dịch với phép chia làm tròn xuống. Đây chưa phải mô phỏng các thanh ghi hay giao thức valid của RTL.

A.3 Cây cộng pipelined: pipelined_adder_tree.sv

Trạng thái assertions. Các kiểm tra dưới `ifndef SYNTHESIS` chạy trong simulator hỗ trợ immediate assertions. Build tổng hợp phải định nghĩa macro `SYNTHESIS` để loại các monitor/assertion này khỏi datapath. Kiểm tra `$bits` phát hiện sai lệch khai báo; chứng minh không overflow dựa trên miền ở Chương 3 và assertion động của tổng mở rộng. Chưa chạy elaboration hoặc assertions HDL trong môi trường soạn thảo. Vòng `genvar` là chỉ số elaboration, không phải tín hiệu runtime.

Module nhận hai vector signed, tính dot-product không saturation và mở rộng dấu ra `OUT_WIDTH`. Điều kiện elaboration là $1 \leq N \leq 2^{30}$, $IN_WIDTH \geq 2$ và $OUT_WIDTH \geq 2 IN_WIDTH + \lceil \log_2 N \rceil$. N mặc định bằng 16, nhưng padding hỗ trợ cả số không phải lũy thừa hai. Mỗi stage có thanh ghi dữ liệu và valid; không có ready.

```

1  `timescale 1ns / 1ps
2
3  // PRE: 1 <= N <= 2^30; IN_WIDTH >= 2; no parameter overflow.
4  // PRE: OUT_WIDTH >= 2*IN_WIDTH + ceil(log2(N)).

```

```

5 // PRE: known inputs when valid; receiver accepts all outputs.
6 // POST: exact signed dot product, without saturation.
7 // INV: edge k input -> output after edge k+LEVELS, no reset.
8 // INV: LEVELS+1 register stages; bubbles advance every edge.
9 // TIMING: one vector/clock only if implementation timing closes.
10 module pipelined_adder_tree #(
11     parameter int signed N = 16,
12     parameter int signed IN_WIDTH = 8,
13     parameter int signed OUT_WIDTH = 32
14 ) (
15     input logic unsigned [0:0] clk,
16     input logic unsigned [0:0] rst_n,
17     input logic unsigned [0:0] in_valid,
18     input logic signed [IN_WIDTH-1:0] a [0:N-1],
19     input logic signed [IN_WIDTH-1:0] b [0:N-1],
20     output logic unsigned [0:0] out_valid,
21     output logic signed [OUT_WIDTH-1:0] dot_product
22 );
23     localparam int signed LEVELS = $clog2(N);
24     localparam int signed LEAVES = 1 << LEVELS;
25     localparam int signed SUM_WIDTH = 2 * IN_WIDTH + LEVELS;
26
27     `ifndef SYNTHESIS
28         initial begin
29             assert (N >= 1 && LEVELS >= 0 && LEVELS < 31)
30                 else $fatal(1, "N is outside the elaboration contract");
31             assert (IN_WIDTH >= 2 &&
32                 IN_WIDTH <= (32'sh7fffffff - LEVELS) / 2)
33                 else $fatal(1, "Width arithmetic can overflow");
34             assert (OUT_WIDTH >= SUM_WIDTH)
35                 else $fatal(1, "Output would truncate the exact sum");
36             assert ($bits(a[0]) == IN_WIDTH &&
37                 $bits(b[0]) == IN_WIDTH &&
38                 $bits(dot_product) == OUT_WIDTH)
39                 else $fatal(1, "Port-width contract mismatch");
40         end
41     `endif
42
43     generate
44         if (N < 1 || IN_WIDTH < 2 || OUT_WIDTH < SUM_WIDTH)
45             begin : invalid_parameters
46                 initial $fatal(1, "Invalid adder-tree parameters");
47             end
48
49         for (genvar stage_index = 0;
50             stage_index <= LEVELS;
51             stage_index = stage_index + 1) begin : stage
52             localparam int signed WIDTH = 2 * IN_WIDTH + stage_index;
53             localparam int signed COUNT = LEAVES >> stage_index;
54             logic signed [WIDTH-1:0] value [0:COUNT-1];
55             logic unsigned [0:0] valid;

```

```

56
57 `ifndef SYNTHESIS
58     initial begin
59         assert ($bits(value[0]) == 2*IN_WIDTH + stage_index)
60             else $fatal(1, "Stage bit-growth mismatch");
61     end
62 `endif
63
64     if (stage_index == 0) begin : input_level
65         always_ff @(posedge clk or negedge rst_n) begin
66             if (!rst_n)
67                 valid <= 1'b0;
68             else
69                 valid <= in_valid;
70         end
71         for (genvar leaf = 0; leaf < COUNT;
72             leaf = leaf + 1) begin : product
73             if (leaf < N) begin : active_leaf
74                 always_ff @(posedge clk or negedge rst_n)
75                     begin
76                         if (!rst_n)
77                             value[leaf] <= '0;
78                         else
79                             value[leaf] <= $signed(a[leaf])
80                                 * $signed(b[leaf]);
81                     end
82             end else begin : padding_leaf
83                 assign value[leaf] = '0;
84             end
85         end
86     end else begin : reduction_level
87         always_ff @(posedge clk or negedge rst_n) begin
88             if (!rst_n)
89                 valid <= 1'b0;
90             else
91                 valid <= stage[stage_index-1].valid;
92         end
93         for (genvar node_index = 0; node_index < COUNT;
94             node_index = node_index + 1) begin : adder
95             logic signed [WIDTH-1:0] left_extended;
96             logic signed [WIDTH-1:0] right_extended;
97 `ifndef SYNTHESIS
98             logic signed [WIDTH:0] checked_sum;
99             assign checked_sum =
100                 $signed({left_extended[WIDTH-1], left_extended})
101                 + $signed({right_extended[WIDTH-1], right_extended});
102             always @(posedge clk) begin
103                 if (rst_n && stage[stage_index-1].valid)
104                     assert (checked_sum[WIDTH] ==
105                         checked_sum[WIDTH-1])
106                     else $fatal(1, "Adder stage overflow");

```

```

107         end
108     'endif
109
110     assign left_extended =
111         $signed(stage[stage_index-1]
112             .value[2*node_index]);
113     assign right_extended =
114         $signed(stage[stage_index-1]
115             .value[2*node_index+1]);
116     always_ff @(posedge clk or negedge rst_n)
117     begin
118         if (!rst_n)
119             value[node_index] <= '0;
120         else
121             value[node_index] <= left_extended
122                 + right_extended;
123     end
124 end
125 end
126 endgenerate
127
128 assign dot_product = $signed(stage[LEVELS].value[0]);
129 assign out_valid = stage[LEVELS].valid;
130 endmodule

```

Listing A.3: pipelined_adder_tree.sv: signed reduction with zero-padded leaves

Ở đầu ra, phép gán có dấu mở rộng tổng chính xác, không lấy INT8. Với $N = 16$ có năm tầng thanh ghi, tương tự số tầng của ShiftGELU nhưng chức năng khác. Mô hình số học đã kiểm tra 4000 trường hợp với nhiều độ rộng và chiều vector, kể cả nhánh lẻ và cực trị. Suite RTL sau này vẫn phải thử $N = 1$, nhánh lẻ, cực trị signed, bubble và reset; kiểm tra số học độc lập không thay thế việc elaboration các cấu hình này bằng simulator.

A.4 PE DSP packing: pe_slice_dsp_packing.sv

Giao diện bổ sung in_valid/out_valid để xác định đúng giao dịch qua hai tầng. Reset của riêng module này là *đồng bộ mức thấp*. Tại cạnh k chốt sản phẩm packed; sau cạnh $k + 1$ trả hai tích. Không có ready và chưa có accumulator. Phép bù dấu thực hiện đúng phương trình (3.30).

```

1  'timescale 1ns / 1ps
2
3  // PRE: a,b,c are known signed INT8 when in_valid=1.
4  // PRE: receiver accepts every valid output; reset is synchronous.
5  // POST: result_ab=a*b and result_ac=a*c of the same input token.
6  // INV: input at edge k -> output after edge k+1 without reset.
7  // WIDTH: signed 18x27 -> 45; sign correction needs 17 bits.
8  // RESOURCE: use_dsp is a hint, not evidence of one mapped DSP.
9  module pe_slice_dsp_packing (
10     input logic unsigned [0:0] clk,
11     input logic unsigned [0:0] rst_n,

```

```

12     input logic unsigned [0:0] in_valid,
13     input logic signed [7:0] operand_a,
14     input logic signed [7:0] operand_b,
15     input logic signed [7:0] operand_c,
16     output logic unsigned [0:0] out_valid,
17     output logic signed [15:0] result_ab,
18     output logic signed [15:0] result_ac
19 );
20     logic signed [17:0] operand_a_extended;
21     logic signed [26:0] operand_b_extended;
22     logic signed [26:0] operand_c_extended;
23     logic signed [26:0] packed_operand;
24     (* use_dsp = "yes" *) logic signed [44:0] dsp_product;
25     logic signed [16:0] high_corrected;
26     logic unsigned [1:0] valid_pipe;
27
28     assign operand_a_extended = {{10{operand_a[7]}}, operand_a};
29     assign operand_b_extended = {{19{operand_b[7]}}, operand_b};
30     assign operand_c_extended = {{19{operand_c[7]}}, operand_c};
31     assign packed_operand = ($signed(operand_b_extended) <<< 18)
32                             + $signed(operand_c_extended);
33     assign high_corrected =
34         $signed({dsp_product[33], dsp_product[33:18]})
35         + $signed({16'b0, dsp_product[15]});
36     assign out_valid = valid_pipe[1];
37
38     always_ff @(posedge clk) begin
39         if (!rst_n) begin
40             dsp_product <= '0;
41             result_ab <= '0;
42             result_ac <= '0;
43             valid_pipe <= '0;
44         end else begin
45             dsp_product <= $signed(operand_a_extended)
46                             * $signed(packed_operand);
47             result_ab <= high_corrected[15:0];
48             result_ac <= dsp_product[15:0];
49             valid_pipe <= {valid_pipe[0], in_valid};
50         end
51     end
52 `ifndef SYNTHESIS
53     initial begin
54         assert ($bits(operand_a_extended) == 18 &&
55                 $bits(packed_operand) == 27 &&
56                 $bits(dsp_product) == 45 &&
57                 $bits(high_corrected) == 17)
58             else $fatal(1, "DSP packing width mismatch");
59     end
60     always @(posedge clk) begin
61         if (rst_n && valid_pipe[0]) begin
62             assert (high_corrected[16] == high_corrected[15])

```

```

63         else $fatal(1, "Corrected high product exceeds INT16");
64         assert ($signed(high_corrected) >= -17'sd16256 &&
65                $signed(high_corrected) <= 17'sd16384)
66         else $fatal(1, "High product outside INT8 product range");
67     end
68 end
69 'endif
70 endmodule

```

Listing A.4: pe_slice_dsp_packing.sv: exact signed INT8 product extraction

Mô hình số học Python đã quét toàn bộ $256^3 = 16\,777\,216$ bộ (a, b, c) và đối chiếu chính xác cả hai tích, bao gồm tích thập âm và $a = -128$. Kết quả quét mô hình số học không chứng minh số lượng DSP của Vivado. Báo cáo synthesis phải chỉ rõ số DSP48E2 thực, thanh ghi nằm trong/ngoài DSP và đường critical path của packing/bù dẫu.

A.5 Khung top-level: attention_core_top.sv

Mã dưới đây hiện thực đầy đủ *profile loopback* ở Mục 4.4; tên module không biểu thị một attention engine đã hoàn tất. Nó không tạo kết quả giả cho GEMM/Softmax. Stream vào/ra cố định 64-bit, cùng clock/reset; AWPROT/ARPROT được chấp nhận nhưng không phân quyền theo thuộc tính bảo vệ.

Một write transaction được commit sau khi đã giữ cả AW và W. Phản hồi B được giữ đến handshake; đọc có thanh ghi R độc lập. Monitor chỉ quan sát đầu ra trước cập nhật nonblocking, rồi lưu mẫu để kiểm tra cạnh sau. Reset hủy lịch sử monitor; không dùng dữ liệu trước reset để báo lỗi giao dịch mới. Nó kiểm tra hold-under-stall và một số điều kiện occupancy, không bao phủ toàn bộ AXI compliance, byte correctness hoặc liveness. Tiền điều kiện master/source tuân thủ giao thức phải được kiểm tra riêng trong testbench. Trong lúc BUSY, ghi bị từ chối để không đổi cấu hình của packet đang bay. Thanh ghi elastic một beat có thể đồng thời lấy beat cũ ra và nhận beat mới vào. Nếu beat cuối đang chờ ready, BUSY vẫn bằng một và DONE chưa được đặt.

```

1  'timescale 1ns / 1ps
2
3  // PRE: common clock/reset, source obeys AXI hold-under-stall.
4  // PRE: exclusive host ownership; valid framing uses configured NBYTES.
5  // POST: ordered byte-preserving LOOPBACK on error-free packets only.
6  // INV: accepted beats - emitted beats = elastic occupancy (0 or 1).
7  // INV: AW/W retained independently; B/R retained until handshake.
8  // DONE: asserted only after final output handshake, not input TLAST.
9  // LIVENESS: requires source progress and eventual downstream ready.
10 // CAPABILITY: no GEMM, Softmax or inference implementation here.
11 module attention_core_top (
12     input  logic unsigned [0:0] clk,
13     input  logic unsigned [0:0] rst_n,
14     input  logic unsigned [7:0] s_axi_awaddr,
15     input  logic unsigned [2:0] s_axi_awprot,
16     input  logic unsigned [0:0] s_axi_awvalid,
17     output logic unsigned [0:0] s_axi_awready,
18     input  logic unsigned [31:0] s_axi_wdata,
19     input  logic unsigned [3:0] s_axi_wstrb,

```

```

20     input  logic unsigned [0:0] s_axi_wvalid,
21     output logic unsigned [0:0] s_axi_wready,
22     output logic unsigned [1:0] s_axi_bresp,
23     output logic unsigned [0:0] s_axi_bvalid,
24     input  logic unsigned [0:0] s_axi_bready,
25     input  logic unsigned [7:0] s_axi_araddr,
26     input  logic unsigned [2:0] s_axi_arprot,
27     input  logic unsigned [0:0] s_axi_arvalid,
28     output logic unsigned [0:0] s_axi_arready,
29     output logic unsigned [31:0] s_axi_rdata,
30     output logic unsigned [1:0] s_axi_rresp,
31     output logic unsigned [0:0] s_axi_rvalid,
32     input  logic unsigned [0:0] s_axi_rready,
33     input  logic unsigned [63:0] s_axis_tdata,
34     input  logic unsigned [7:0] s_axis_tkeep,
35     input  logic unsigned [0:0] s_axis_tlast,
36     input  logic unsigned [0:0] s_axis_tvalid,
37     output logic unsigned [0:0] s_axis_tready,
38     output logic unsigned [63:0] m_axis_tdata,
39     output logic unsigned [7:0] m_axis_tkeep,
40     output logic unsigned [0:0] m_axis_tlast,
41     output logic unsigned [0:0] m_axis_tvalid,
42     input  logic unsigned [0:0] m_axis_tready
43 );
44     localparam logic unsigned [31:0] CORE_ID = 32'h56495432;
45     logic unsigned [0:0] aw_pending;
46     logic unsigned [0:0] w_pending;
47     logic unsigned [7:0] write_address;
48     logic unsigned [31:0] write_data;
49     logic unsigned [3:0] write_strobes;
50     logic unsigned [0:0] write_commit;
51     logic unsigned [31:0] reg_nbytes;
52     logic unsigned [31:0] reg_tokens;
53     logic unsigned [31:0] reg_head_dim;
54     logic unsigned [31:0] reg_heads;
55     logic unsigned [0:0] busy;
56     logic unsigned [0:0] done_flag;
57     logic unsigned [0:0] error_flag;
58     logic unsigned [0:0] ingress_done;
59     logic unsigned [31:0] remaining;
60     logic unsigned [0:0] expected_last;
61     logic unsigned [7:0] expected_keep;
62     logic unsigned [0:0] input_fire;
63     logic unsigned [0:0] output_fire;
64
65     function automatic logic unsigned [31:0] merge_bytes(
66         input logic unsigned [31:0] previous,
67         input logic unsigned [31:0] incoming,
68         input logic unsigned [3:0] strobes
69     );
70         logic unsigned [31:0] merged;

```

```

71     begin
72         merged = previous;
73         for (int signed byte_index = 0; byte_index < 4;
74             byte_index = byte_index + 1) begin
75             if (stobes[byte_index])
76                 merged[8*byte_index +: 8] =
77                     incoming[8*byte_index +: 8];
78         end
79         merge_bytes = merged;
80     end
81 endfunction
82
83 assign s_axi_awready = rst_n && !aw_pending && !s_axi_bvalid;
84 assign s_axi_wready = rst_n && !w_pending && !s_axi_bvalid;
85 assign write_commit = aw_pending && w_pending && !s_axi_bvalid;
86 assign s_axi_arready = rst_n && !s_axi_rvalid;
87 assign s_axis_tready = rst_n && busy && !ingress_done
88                     && (!m_axis_tvalid || m_axis_tready);
89 assign input_fire = s_axis_tvalid && s_axis_tready;
90 assign output_fire = m_axis_tvalid && m_axis_tready;
91 assign expected_last = remaining <= 32'd8;
92 assign expected_keep = (remaining >= 32'd8) ? 8'hff
93                     : (8'hff >> (4'd8 - remaining[3:0]));
94
95 always_ff @(posedge clk or negedge rst_n) begin
96     if (!rst_n) begin
97         aw_pending <= 1'b0;
98         w_pending <= 1'b0;
99         write_address <= '0;
100        write_data <= '0;
101        write_stobes <= '0;
102        s_axi_bvalid <= 1'b0;
103        s_axi_bresp <= 2'b00;
104        reg_nbytes <= '0;
105        reg_tokens <= '0;
106        reg_head_dim <= '0;
107        reg_heads <= '0;
108        busy <= 1'b0;
109        done_flag <= 1'b0;
110        error_flag <= 1'b0;
111        ingress_done <= 1'b0;
112        remaining <= '0;
113        m_axis_tdata <= '0;
114        m_axis_tkeep <= '0;
115        m_axis_tlast <= 1'b0;
116        m_axis_tvalid <= 1'b0;
117    end else begin
118        if (s_axi_awvalid && s_axi_awready) begin
119            write_address <= s_axi_awaddr;
120            aw_pending <= 1'b1;
121        end

```

```

122     if (s_axi_wvalid && s_axi_wready) begin
123         write_data <= s_axi_wdata;
124         write_strobes <= s_axi_wstrb;
125         w_pending <= 1'b1;
126     end
127     if (s_axi_bvalid && s_axi_bready)
128         s_axi_bvalid <= 1'b0;
129
130     if (write_commit) begin
131         aw_pending <= 1'b0;
132         w_pending <= 1'b0;
133         s_axi_bvalid <= 1'b1;
134         s_axi_bresp <= 2'b00;
135         if (busy || write_address[1:0] != 2'b00) begin
136             s_axi_bresp <= 2'b10;
137         end else begin
138             case (write_address)
139                 8'h00: begin
140                     if (write_strobes[0] && write_data[0])
141                         begin
142                             if (reg_nbytes == 0 ||
143                                 reg_nbytes > 32'h03ffffff) begin
144                                 s_axi_bresp <= 2'b10;
145                                 error_flag <= 1'b1;
146                             end else begin
147                                 busy <= 1'b1;
148                                 done_flag <= 1'b0;
149                                 error_flag <= 1'b0;
150                                 ingress_done <= 1'b0;
151                                 remaining <= reg_nbytes;
152                             end
153                         end else if (write_strobes[0] &&
154                             write_data[1]) begin
155                             done_flag <= 1'b0;
156                             error_flag <= 1'b0;
157                         end
158                     end
159                 8'h04: reg_nbytes <= merge_bytes(
160                     reg_nbytes, write_data, write_strobes);
161                 8'h10: reg_tokens <= merge_bytes(
162                     reg_tokens, write_data, write_strobes);
163                 8'h18: reg_head_dim <= merge_bytes(
164                     reg_head_dim, write_data, write_strobes);
165                 8'h1c: reg_heads <= merge_bytes(
166                     reg_heads, write_data, write_strobes);
167                 default: s_axi_bresp <= 2'b10;
168             endcase
169         end
170     end
171
172     if (output_fire) begin

```

```

173         m_axis_tvalid <= 1'b0;
174         if (m_axis_tlast) begin
175             busy <= 1'b0;
176             done_flag <= 1'b1;
177         end
178     end
179     if (input_fire) begin
180         m_axis_tdata <= s_axis_tdata;
181         m_axis_tkeep <= expected_keep;
182         m_axis_tlast <= s_axis_tlast || expected_last;
183         m_axis_tvalid <= 1'b1;
184         if (s_axis_tkeep != expected_keep ||
185             s_axis_tlast != expected_last)
186             error_flag <= 1'b1;
187         if (s_axis_tlast || expected_last) begin
188             ingress_done <= 1'b1;
189             remaining <= '0;
190         end else begin
191             remaining <= remaining - 32'd8;
192         end
193     end
194 end
195 end
196
197 always_ff @(posedge clk or negedge rst_n) begin
198     if (!rst_n) begin
199         s_axi_rvalid <= 1'b0;
200         s_axi_rdata <= '0;
201         s_axi_rresp <= 2'b00;
202     end else begin
203         if (s_axi_rvalid && s_axi_rready)
204             s_axi_rvalid <= 1'b0;
205         if (s_axi_arvalid && s_axi_arready) begin
206             s_axi_rvalid <= 1'b1;
207             s_axi_rdata <= '0;
208             s_axi_rresp <= 2'b00;
209             case (s_axi_araddr)
210                 8'h00: s_axi_rdata <=
211                     {29'b0, error_flag, done_flag, busy};
212                 8'h04: s_axi_rdata <= reg_nbytes;
213                 8'h08: s_axi_rdata <= 32'd1;
214                 8'h0c: s_axi_rdata <= CORE_ID;
215                 8'h10: s_axi_rdata <= reg_tokens;
216                 8'h18: s_axi_rdata <= reg_head_dim;
217                 8'h1c: s_axi_rdata <= reg_heads;
218                 default: s_axi_rresp <= 2'b10;
219             endcase
220         end
221     end
222 end
223 `ifndef SYNTHESIS

```

```

224 logic unsigned [0:0] stream_stalled;
225 logic unsigned [0:0] response_b_stalled;
226 logic unsigned [0:0] response_r_stalled;
227 logic unsigned [72:0] held_stream;
228 logic unsigned [1:0] held_b;
229 logic unsigned [33:0] held_r;
230
231 initial begin
232     assert ($bits(s_axis_tdata) == 64 &&
233             $bits(m_axis_tdata) == 64 &&
234             $bits(s_axis_tkeep) == 8 &&
235             $bits(m_axis_tkeep) == 8)
236         else $fatal(1, "Stream byte-lane width mismatch");
237     assert ($bits(s_axi_wdata) == 32 &&
238             $bits(s_axi_wstrb) == 4 &&
239             $bits(s_axi_rdata) == 32)
240         else $fatal(1, "AXI-Lite width mismatch");
241     assert ($bits(remaining) == 32)
242         else $fatal(1, "Byte counter width mismatch");
243 end
244
245 // Sample pre-NBA state at each edge, compare with prior edge.
246 always @(posedge clk or negedge rst_n) begin
247     if (!rst_n) begin
248         stream_stalled <= 1'b0;
249         response_b_stalled <= 1'b0;
250         response_r_stalled <= 1'b0;
251         held_stream <= '0;
252         held_b <= '0;
253         held_r <= '0;
254     end else begin
255         if (stream_stalled)
256             assert (m_axis_tvalid &&
257                     {m_axis_tdata,m_axis_tkeep,m_axis_tlast}
258                     === held_stream)
259                 else $fatal(1, "Stream changed under stall");
260         if (response_b_stalled)
261             assert (s_axi_bvalid && s_axi_bresp === held_b)
262                 else $fatal(1, "B response changed under stall");
263         if (response_r_stalled)
264             assert (s_axi_rvalid &&
265                     {s_axi_rdata,s_axi_rresp} === held_r)
266                 else $fatal(1, "R response changed under stall");
267         assert (!input_fire || (busy && remaining > 0))
268             else $fatal(1, "Input accepted outside active packet");
269         assert (!m_axis_tvalid || busy)
270             else $fatal(1, "Output exists while core is idle");
271         assert (!ingress_done || !s_axis_tready)
272             else $fatal(1, "Input accepted after packet end");
273         stream_stalled <= m_axis_tvalid && !m_axis_tready;
274         response_b_stalled <= s_axi_bvalid && !s_axi_bready;

```

```

275         response_r_stalled <= s_axi_rvalid && !s_axi_rready;
276         held_stream <= {m_axis_tdata,m_axis_tkeep,m_axis_tlast};
277         held_b <= s_axi_bresp;
278         held_r <= {s_axi_rdata,s_axi_rresp};
279     end
280 end
281 'endif
282 endmodule

```

Listing A.5: attention_core_top.sv: AXI-Lite controlled diagnostic loopback

Khi đọc và ghi cùng một thanh ghi ở cùng cạnh, đọc trả trạng thái trước cạnh đó. Không hỗ trợ burst AXI đầy đủ, clock-domain crossing, giao dịch outstanding không giới hạn hay abort qua CSR. Các kết nối PS/DMA qua miền clock khác phải có interconnect và bộ đổi miền clock thích hợp bên ngoài shell. Đây là một profile slave nhỏ để bring-up, không phải thay thế AXI protocol checker trong kiểm thử hệ thống.

Phụ lục B

Python lượng tử hóa và thu thập activation

B.1 Forward hooks: `quant_hooks.py`

B.1.1 Phạm vi và quy định về dữ liệu

Script yêu cầu PyTorch trong môi trường chạy của người dùng. Nó đăng ký forward hook trên những lớp `nn.Linear` được chọn, lưu bản sao CPU của activation gần nhất và tích lũy cực đại tuyệt đối qua các lượt calibration. Hook quan sát đầu ra module nhưng không thay đổi đầu ra của mô hình [25]. Phải gọi `model.eval()` trước khi thu thập; chế độ này không thay thế `torch.no_grad()`.

Với projection ghép QKV, người dùng truyền ánh xạ tên module sang số head. Script giả định tensor đầu ra có shape $(B, T, 3D)$ và thứ tự lưu là toàn bộ Q , rồi K , rồi V . Sau khi reshape, các tensor quan sát có shape (B, h, T, d_k) . Projection có layout khác, attention dùng Q/K normalization hoặc biến đổi vị trí sau projection cần hook bổ sung ở đúng vị trí; chỉ bắt lớp linear không bảo đảm đó là Q, K cuối cùng đưa vào phép nhân attention.

`freeze()` đóng băng scale sau calibration; `quantize_captured()` không được gọi trước bước này. Có thể truyền `fixed_scales` cho các activation cần scale phần cứng cố định, ví dụ đầu vào ShiftGELU bằng 1/16. Nếu lớp được gọi nhiều lần trong một forward, thống kê bao phủ mọi lần gọi nhưng chỉ activation của lần cuối được lưu. Đây không phải cơ chế ghi lại toàn bộ trace của mô hình.

`torch.round` dùng nearest-even [26]. Quy tắc này dành cho bước FP32 sang INT8, khác với floor trong RTL dyadic. Script từ chối dữ liệu không hữu hạn, giữ scale trên đúng device trong phép chia và trả tensor INT8 cùng scale. Phần mềm không giả lập training có fake-quant và chưa xuất tệp `.hex`; exporter và golden model toàn chuỗi thuộc batch tiếp theo.

Tiền/hậu điều kiện observer. Tensor đầu vào phải không rỗng, có kiểu floating, và hữu hạn sau chuyển FP32. Scale trả về là một số Python `float` mang đúng giá trị scale đã làm tròn FP32, không phải giá trị số thực trước chuyển kiểu. Hậu điều kiện là cùng shape/device với tensor nguồn, dtype `torch.int8`, mã nằm trong $[-128, 127]$. Hook không thay output và bản snapshot tách khỏi autograd; caller chịu trách nhiệm chọn đúng vị trí graph. Bất biến calibration là `max_abs[name]` bằng cực đại của mọi snapshot FP32 quan sát từ khi tạo observer; `freeze()` chốt scale từ thống kê đó. `clear_activations()` không xóa thống kê hay scale.

Reference đơn vị có kiểu. Các hàm `shift_gelu_q44`, `residual_dyadic_int8`, `isqrt_u32_fixed` dùng Python int không wrap và kiểm tra miền trước tính. Chúng hiện thực các phương trình Chương 2; riêng căn không đại diện cho RTL LayerNorm đã triển khai. Assertions kiểm tra hậu điều kiện của lần gọi; không thay thế chứng minh hoặc simulator. Type hints không tự kiểm tra miền runtime, vì vậy không bỏ các kiểm tra `ValueError/TypeError`. Mã vẫn yêu cầu PyTorch nếu chạy nguyên tệp; các reference số nguyên không cần tensor và được kiểm tra độc lập trong lần biên soạn này.

```

1 from __future__ import annotations
2
3 import math
4 from types import TracebackType
5 from typing import Callable, Dict, List, Mapping, Optional
6 from typing import Sequence, Tuple, Type, TypedDict
7
8 import torch
9 import torch.nn as nn
10 from torch.utils.hooks import RemovableHandle
11
12
13 class QuantizedCapture(TypedDict):
14     codes: torch.Tensor
15     scale: float
16
17
18 def require_int_range(value: int, lower: int, upper: int) -> None:
19     if type(value) is not int:
20         raise TypeError("Expected a Python int, not bool or float")
21     if not lower <= value <= upper:
22         raise ValueError(f"Integer outside [{lower},{upper}]")
23
24
25 def floor_shift(value: int, exponent: int) -> int:
26     if type(value) is not int:
27         raise TypeError("Expected a Python int")
28     require_int_range(exponent, 0, 31)
29     quotient = value >> exponent
30     assert quotient * (1 << exponent) <= value
31     assert value < (quotient + 1) * (1 << exponent)
32     return quotient
33
34
35 def int8_clip(value: int) -> int:
36     if type(value) is not int:
37         raise TypeError("Expected a Python int")
38     return min(127, max(-128, value))
39
40
41 def shift_gelu_q44(sample: int) -> int:
42     """Q4.4 reference for A.1; exact integer datapath, not exact GELU."""
43     require_int_range(sample, -128, 127)
44     power = (sample + floor_shift(sample, 1)

```

```

45         + floor_shift(sample, 3) + floor_shift(sample, 4))
46     assert -216 <= power <= 212
47     delta = -abs(power)
48     base2_power = delta + floor_shift(delta, 1) - floor_shift(delta, 4)
49     whole = (-base2_power) // 16
50     remainder = base2_power + 16 * whole
51     assert -310 <= base2_power <= 0
52     assert 0 <= whole <= 19 and -15 <= remainder <= 0
53     exponent = floor_shift(4096 + 128 * remainder, whole)
54     numerator = 4096 if power >= 0 else exponent
55     denominator = 4096 + exponent
56     gain = (256 * numerator) // denominator
57     assert 0 <= exponent <= 4096 and 0 <= gain <= 256
58     product = sample * gain
59     assert -32768 <= product <= 32512
60     return int8_clip(floor_shift(product, 8))
61
62
63 def residual_dyadic_int8(
64     branch: int, shortcut: int,
65     multiplier_f: int, exponent_f: int,
66     multiplier_x: int, exponent_x: int,
67     scale_locked: bool = False,
68 ) -> int:
69     """A.2 contract; caller aligns tokens and physical scales."""
70     require_int_range(branch, -128, 127)
71     require_int_range(shortcut, -128, 127)
72     require_int_range(multiplier_f, -(1 << 31), (1 << 31) - 1)
73     require_int_range(multiplier_x, -(1 << 31), (1 << 31) - 1)
74     require_int_range(exponent_f, 0, 31)
75     require_int_range(exponent_x, 0, 31)
76     if type(scale_locked) is not bool:
77         raise TypeError("scale_locked must be bool")
78     if scale_locked:
79         return int8_clip(branch + shortcut)
80     if multiplier_f < 0 or multiplier_x < 0:
81         raise ValueError("Unlocked scale multipliers must be nonnegative")
82     product_f = branch * multiplier_f
83     product_x = shortcut * multiplier_x
84     assert -(1 << 39) <= product_f < (1 << 39)
85     assert -(1 << 39) <= product_x < (1 << 39)
86     summed = (floor_shift(product_f, exponent_f)
87               + floor_shift(product_x, exponent_x))
88     assert -(1 << 40) <= summed < (1 << 40)
89     return int8_clip(summed)
90
91
92 def isqrt_u32_fixed(variance: int) -> int:
93     """Ten clamped Newton iterations; no hardware cycle claim."""
94     require_int_range(variance, 1, (1 << 32) - 1)
95     estimate = 1 << ((variance.bit_length() + 1) // 2)

```

```

96     for _iteration in range(10):
97         quotient = variance // estimate
98         summed = estimate + quotient
99         assert 1 <= estimate <= 65536
100        assert 0 <= quotient <= 65537 and summed < (1 << 18)
101        candidate = summed // 2
102        estimate = min(estimate, candidate)
103    assert estimate * estimate <= variance < (estimate + 1) ** 2
104    return estimate
105
106
107 class ViTQuantizerHook:
108     def __init__(
109         self, model: nn.Module,
110         layer_names: Optional[Sequence[str]] = None,
111         qkv_heads: Optional[Mapping[str, int]] = None,
112     ) -> None:
113         self.model = model
114         self.activations: Dict[str, torch.Tensor] = {}
115         self.max_abs: Dict[str, float] = {}
116         self.scales: Dict[str, float] = {}
117         self.hooks: List[RemovableHandle] = []
118         self.calibrating: bool = True
119         self.qkv_heads: Dict[str, int] = dict(qkv_heads or {})
120         modules = dict(model.named_modules())
121         if layer_names is None:
122             selected = {
123                 name for name, module in modules.items()
124                 if isinstance(module, nn.Linear)
125             }
126         else:
127             selected = set(layer_names)
128         if not selected:
129             raise ValueError("No linear layers selected")
130         for name in selected:
131             if name not in modules:
132                 raise ValueError(f"Unknown module: {name}")
133             if not isinstance(modules[name], nn.Linear):
134                 raise TypeError(f"Not a linear layer: {name}")
135         for name, heads in self.qkv_heads.items():
136             if name not in selected:
137                 raise ValueError(f"QKV layer not selected: {name}")
138             if type(heads) is not int or heads <= 0:
139                 raise ValueError("Head count must be positive")
140             if modules[name].out_features % (3 * heads):
141                 raise ValueError("QKV width is not 3 * h * d")
142         try:
143             for name in sorted(selected):
144                 handle = modules[name].register_forward_hook(
145                     self._hook_fn(name))
146                 self.hooks.append(handle)

```

```

147     except Exception:
148         self.remove_hooks()
149         raise
150
151 def _store(self, name: str, tensor: torch.Tensor) -> None:
152     if not torch.is_tensor(tensor):
153         raise TypeError("Hook output must be a tensor")
154     if not tensor.is_floating_point() or tensor.numel() == 0:
155         raise ValueError("Expected nonempty floating tensor")
156     snapshot = tensor.detach().to(
157         device="cpu", dtype=torch.float32).clone()
158     if not bool(torch.isfinite(snapshot).all().item()):
159         raise ValueError(f"Non-finite activation: {name}")
160     self.activations[name] = snapshot
161     if self.calibrating:
162         peak = float(snapshot.abs().amax().item())
163         self.max_abs[name] = max(
164             self.max_abs.get(name, 0.0), peak)
165
166 def _hook_fn(self, layer_name: str) -> Callable[
167     [nn.Module, Tuple[torch.Tensor, ...], torch.Tensor], None
168 ]:
169     def hook(
170         module: nn.Module, inputs: Tuple[torch.Tensor, ...],
171         output: torch.Tensor,
172     ) -> None:
173         if self.model.training:
174             raise RuntimeError("Call model.eval() first")
175         self._store(layer_name, output)
176         if layer_name in self.qkv_heads:
177             if output.ndim != 3:
178                 raise ValueError("QKV expects [batch,tokens,3D]")
179             batch, tokens, width = output.shape
180             heads = self.qkv_heads[layer_name]
181             head_dim = width // (3 * heads)
182             projected = output.reshape(
183                 batch, tokens, 3, heads, head_dim)
184             projected = projected.permute(2, 0, 3, 1, 4)
185             for index, suffix in enumerate(("q", "k", "v")):
186                 self._store(
187                     f"{layer_name}.{suffix}", projected[index])
188     return hook
189
190 @staticmethod
191 def quantize_symmetric_int8(
192     tensor: torch.Tensor, scale: Optional[float] = None,
193 ) -> Tuple[torch.Tensor, float]:
194     if not torch.is_tensor(tensor):
195         raise TypeError("Expected tensor")
196     if not tensor.is_floating_point() or tensor.numel() == 0:
197         raise ValueError("Expected nonempty floating tensor")

```

```

198     values = tensor.detach().to(dtype=torch.float32)
199     if not bool(torch.isfinite(values).all().item()):
200         raise ValueError("Tensor contains NaN or infinity")
201     if scale is None:
202         peak = float(values.abs().amax().item())
203         scale = max(peak / 127.0, 1e-8)
204     scale = float(scale)
205     if not math.isfinite(scale) or scale <= 0:
206         raise ValueError("Scale must be finite and positive")
207     scale_tensor = values.new_tensor(scale)
208     if not bool(torch.isfinite(scale_tensor).item()):
209         raise ValueError("Scale is not finite in FP32")
210     if float(scale_tensor.item()) <= 0:
211         raise ValueError("Scale underflows FP32")
212     quantized = torch.round(values / scale_tensor)
213     quantized = quantized.clamp(-128, 127).to(torch.int8)
214     return quantized, float(scale_tensor.item())
215
216 def freeze(
217     self, fixed_scales: Optional[Mapping[str, float]] = None,
218 ) -> Dict[str, float]:
219     if not self.max_abs:
220         raise RuntimeError("Run calibration forwards first")
221     scales = {
222         name: max(peak / 127.0, 1e-8)
223         for name, peak in self.max_abs.items()
224     }
225     for name, value in dict(fixed_scales or {}).items():
226         if name not in scales:
227             raise KeyError(f"Activation not observed: {name}")
228         value = float(value)
229         if not math.isfinite(value) or value <= 0:
230             raise ValueError("Invalid fixed scale")
231         scales[name] = value
232     normalized: Dict[str, float] = {}
233     for name, scale in scales.items():
234         _, normalized[name] = self.quantize_symmetric_int8(
235             torch.zeros(1, dtype=torch.float32), scale)
236     self.scales = normalized
237     self.calibrating = False
238     return dict(self.scales)
239
240 def quantize_captured(self) -> Dict[str, QuantizedCapture]:
241     if self.calibrating:
242         raise RuntimeError("Freeze scales before quantization")
243     result: Dict[str, QuantizedCapture] = {}
244     for name, tensor in self.activations.items():
245         if name not in self.scales:
246             raise RuntimeError(f"Missing frozen scale: {name}")
247         codes, scale = self.quantize_symmetric_int8(
248             tensor, self.scales[name])

```

```

249         result[name] = {"codes": codes, "scale": scale}
250     return result
251
252     def clear_activations(self) -> None:
253         self.activations.clear()
254
255     def remove_hooks(self) -> None:
256         for handle in self.hooks:
257             handle.remove()
258         self.hooks.clear()
259
260     def __enter__(self) -> ViTQuantizerHook:
261         return self
262
263     def __exit__(
264         self, exc_type: Optional[Type[BaseException]],
265         exc_value: Optional[BaseException],
266         traceback: Optional[TracebackType],
267     ) -> bool:
268         self.remove_hooks()
269         return False
270
271
272 class ProjectionProbe(nn.Module):
273     def __init__(self) -> None:
274         super().__init__()
275         self.qkv = nn.Linear(8, 24)
276         self.fc1 = nn.Linear(8, 32)
277         self.gelu = nn.GELU()
278         self.fc2 = nn.Linear(32, 8)
279
280     def forward(
281         self, tokens: torch.Tensor,
282     ) -> Tuple[torch.Tensor, torch.Tensor]:
283         projected = self.qkv(tokens)
284         mlp_output = self.fc2(self.gelu(self.fc1(tokens)))
285         return projected, mlp_output
286
287
288 def main() -> None:
289     torch.manual_seed(7)
290     model = ProjectionProbe().eval()
291     with ViTQuantizerHook(model, qkv_heads={"qkv": 2}) as observer:
292         with torch.no_grad():
293             for _ in range(4):
294                 model(torch.randn(2, 5, 8))
295             observer.freeze(fixed_scales={"fc1": 1.0 / 16.0})
296             observer.clear_activations()
297             with torch.no_grad():
298                 model(torch.randn(1, 5, 8))
299             captures = observer.quantize_captured()

```

```

300     for name, entry in captures.items():
301         codes = entry["codes"]
302         scale = entry["scale"]
303         print(name, tuple(codes.shape), codes.dtype, scale)
304
305
306 if __name__ == "__main__":
307     main()

```

Listing B.1: quant_hooks.py: calibration, frozen scales, and explicit QKV capture

B.1.2 Sử dụng và giới hạn

Lưu listing thành `quant_hooks.py` rồi chạy trong môi trường có PyTorch.

`ProjectionProbe` chỉ minh họa các phép chiếu và MLP để thử hook; nó không phải một encoder ViT đầy đủ và không tải checkpoint hay dữ liệu ngoài. Thay đổi tượng này bằng mô hình mục tiêu, lựa chọn đúng tên lớp và số head trước khi thu thập dữ liệu thực.

Ba scale của Q, K, V được calibration độc lập. Nếu thiết kế phần cứng yêu cầu scale chung, cần truyền cùng giá trị qua `fixed_scales` cho ba khóa tương ứng. Scale của projection ghép được lưu riêng và không tự áp vào ba tensor đã tách.

`clear_activations()` xóa snapshot nhưng giữ thống kê calibration và scale; gọi nó trước một inference mới giúp tránh dùng nhầm snapshot của nhánh không được thực thi. Nếu muốn calibration lại từ đầu, tạo observer mới.

Bản biên soạn này kiểm tra cú pháp toàn listing và thực thi riêng các reference số nguyên bằng kiểm tra hữu hạn; chưa thực thi forward bằng PyTorch vì thư viện chưa được cài. Không có simulator SystemVerilog hay Vivado trong môi trường này, nên không diễn giải kiểm tra số học ở Phụ lục A thành chứng nhận HW/SW bit-exact. Các suite mô phỏng, exporter hex và kết quả trên bo mạch sẽ được bổ sung đúng phạm vi các batch sau.

B.1.3 Kết quả kiểm tra hữu hạn khi refactoring

Ngày 14/09/2026, kiểm tra parser áp dụng cho toàn listing; các reference Python int được tách bằng AST và chạy độc lập, không giả lập import PyTorch. Kết quả kiểm tra:

- ShiftGELU: toàn bộ 256 mã INT8, đối chiếu với bộ chia chín bước so sánh-trừ và tích theo từng bit của gain. Các chỉ số sai số vô hướng trong A.1 không thay đổi.
- Residual: toàn bộ 65536 cặp INT8 scale-locked và 100000 trường hợp dyadic với seed 20260914, đối chiếu floor bằng số hữu tỷ chính xác.
- Newton: 361365 giá trị uint32 phân biệt, gồm toàn miền 1 đến 65535, các số $a^2 - 1, a^2, a^2 + 1$ hợp lệ với $1 \leq a \leq 65535$, cực đại uint32 và 100000 mẫu ngẫu nhiên bổ sung trước khử trùng. Mọi kết quả khớp `math.isqrt`; cũng kiểm tra cận sáu bước của Mục 2.4.4. Bộ sinh ngẫu nhiên được tiếp tục từ trạng thái sau kiểm tra residual.
- Các đầu vào ngoài miền, bool thay int, scale multiplier âm khi không locked, căn zero và số mũ quá 31 bị từ chối trong các trường hợp kiểm tra.

Đây là kiểm tra hữu hạn của reference và một mô hình số học độc lập, không phải chạy HDL, static type checker hay Lean kernel. Không suy ra tính đúng temporal của A.1/A.2 từ các phép đối chiếu vô hướng này.

B.2 Xuất vector có manifest: export_vectors.py

B.2 nhận NPZ cung cấp bởi người dùng: mode dot dùng a/b shape (cases,N), loopback dùng payload, qkv dùng q/k/v cùng shape. Không tự tạo dữ liệu ngẫu nhiên rồi gọi là activation của mô hình. Đầu vào nguyên giữ nguyên mã; đầu vào floating dùng float64 nearest-even và scale khai báo. Muốn giữ bit-exact mã observer FP32 B.1, truyền chính tensor INT8 đã xuất từ observer, không lượng tử hóa lại. Golden dot là tổng INT32 trước rescale với scale S^2 ; golden loopback là payload sao chép. QKV chỉ xuất input, không có golden attention giả.

Mỗi dòng INT8 là hai chữ số hex (một byte), INT32 là tám chữ số. Manifest ghi shape, scale, C-order và SHA-256, chỉ được tạo cuối sau mọi file. Thư mục đích phải chứa tồn tại; thiếu manifest nghĩa là export chưa hoàn tất. Chạy hai script B.2/B.3 trong cùng thư mục phần mềm.

```
1 from __future__ import annotations
2 import argparse
3 import hashlib
4 import json
5 from pathlib import Path
6 from typing import Dict
7 import numpy as np
8 from numpy.typing import NDArray
9 from golden_model_sim import dot_product, loopback_payload
10
11
12 def quantize(values: NDArray, scale: float) -> NDArray[np.int8]:
13     if values.size == 0 or not np.isfinite(scale) or scale <= 0:
14         raise ValueError("Empty tensor or invalid scale")
15     if np.issubdtype(values.dtype, np.integer):
16         if int(values.min()) < -128 or int(values.max()) > 127:
17             raise ValueError("Integer input must already be INT8 codes")
18         return values.astype(np.int8)
19     if not np.issubdtype(values.dtype, np.floating):
20         raise TypeError("Expected floating values or integer codes")
21     floating = values.astype(np.float64)
22     if not np.isfinite(floating).all():
23         raise ValueError("Non-finite tensor")
24     with np.errstate(over="ignore"):
25         normalized = np.clip(floating / scale, -128.0, 127.0)
26     return np rint(normalized).astype(np.int8)
27
28
29 def write_hex(path: Path, values: NDArray, width: int) -> Dict:
30     if width not in (8, 32) or values.size == 0:
31         raise ValueError("Invalid width or empty tensor")
32     if not np.issubdtype(values.dtype, np.integer):
33         raise TypeError("Expected integer codes")
34     lower, upper = -(1 << (width-1)), (1 << (width-1))-1
35     integers = [int(value) for value in values.flat]
36     if any(value < lower or value > upper for value in integers):
37         raise OverflowError("Element outside signed width")
```

```

38     text = "".join(f"{value & ((1 << width)-1):0{width//4}X}\n"
39                     for value in integers)
40     path.write_text(text, encoding="ascii")
41     return {"file": path.name, "shape": list(values.shape),
42            "count": len(integers), "signed_width": width,
43            "sha256": hashlib.sha256(text.encode("ascii")).hexdigest()}
44
45
46 def export_vectors(source: Path, destination: Path,
47                   kind: str, scale: float) -> Dict:
48     for path in (source, destination):
49         if path.is_absolute() or ".." in path.parts:
50             raise ValueError("Use paths within the working directory")
51     if not np.isfinite(scale) or scale <= 0:
52         raise ValueError("Invalid scale")
53     with np.load(source, allow_pickle=False) as archive:
54         if kind == "dot":
55             first, second = [quantize(archive[name], scale) for name in ("a", "
56 b")]
57             if first.ndim != 2 or first.shape != second.shape:
58                 raise ValueError("a/b require equal [cases,N] shapes")
59             if first.shape[1] * 16384 > (1 << 31)-1:
60                 raise OverflowError("INT32 accumulator contract exceeded")
61             tensors = {"a": first, "b": second, "golden_dot": np.array([
62                 dot_product(left.tolist(), right.tolist())
63                 for left, right in zip(first, second)], dtype=np.int32)}
64             widths = {"a": 8, "b": 8, "golden_dot": 32}
65         elif kind == "loopback":
66             payload = quantize(archive["payload"], scale).reshape(-1)
67             tensors = {"payload": payload, "golden_loopback": loopback_payload(
68                 payload)}
69             widths = {name: 8 for name in tensors}
70         elif kind == "qkv":
71             tensors = {name: quantize(archive[name], scale) for name in ("q", "
72 k", "v")}
73             shape = tensors["q"].shape
74             if len(shape) != 2 or any(value.shape != shape for value in tensors
75 .values()):
76                 raise ValueError("Q/K/V require equal [tokens,channels] shapes"
77 )
78             widths = {name: 8 for name in tensors}
79         else:
80             raise ValueError("Unknown kind")
81     golden_scale = (scale*scale if kind == "dot"
82                    else scale if kind == "loopback" else None)
83     if golden_scale is not None and (
84         not np.isfinite(golden_scale) or golden_scale <= 0):
85         raise ValueError("Output scale is not representable")
86     destination.mkdir(parents=True, exist_ok=False)
87     files = [write_hex(destination / f"{name}.hex", value, widths[name])
88              for name, value in tensors.items()]

```

```

84     manifest = {"schema": 1, "kind": kind, "order": "C", "input_scale": scale,
85                 "zero_point": 0, "floating_round": "float64 nearest-even, INT8
clip",
86                 "integer_inputs": "preserved codes, no rescaling",
87                 "golden_scale": golden_scale,
88                 "attention_output_available": False, "files": files,
89                 "source_sha256": hashlib.sha256(source.read_bytes()).hexdigest
90             })
91     (destination / "manifest.json").write_text(
92         json.dumps(manifest, indent=2, allow_nan=False) + "\n", encoding="utf8"
93     )
94     return manifest
95
96 def main() -> None:
97     parser = argparse.ArgumentParser(description="Export supplied NPZ inputs")
98     parser.add_argument("source", type=Path)
99     parser.add_argument("destination", type=Path)
100    parser.add_argument("--kind", choices=("dot", "loopback", "qkv"), required=
True)
101    parser.add_argument("--scale", type=float, default=1.0/16.0)
102    args = parser.parse_args()
103    export_vectors(args.source, args.destination, args.kind, args.scale)
104
105 if __name__ == "__main__":
106     main()

```

Listing B.2: export_vectors.py: element-wise hex and explicit metadata

B.3 Golden unit số nguyên: golden_model_sim.py

Các hàm Q4.4, residual và căn uint32 giữ nguyên định nghĩa B.1 nhưng không import PyTorch. Python int không wrap trung gian; biên cổng kiểm tra miền. Dot-product không saturate tổng; caller chọn độ rộng output đủ lớn. Reference loopback trả bản sao để không alias nguồn. Không có Softmax số thực đặt tên integer-only. Khi tích hợp attention cần định nghĩa scale score $S_Q S_K / \sqrt{d_k}$, thuật toán Softmax nguyên và packet layout trước khi thêm golden tương ứng.

```

1 from __future__ import annotations
2 from pathlib import Path
3 from typing import List, Sequence, Tuple
4 import numpy as np
5 from numpy.typing import NDArray
6
7 def require_int_range(value: int, lower: int, upper: int) -> None:
8     if type(value) is not int:
9         raise TypeError("Expected a Python int, not bool or float")
10    if not lower <= value <= upper:
11        raise ValueError(f"Integer outside [{lower},{upper}]")
12

```

```

13
14 def floor_shift(value: int, exponent: int) -> int:
15     if type(value) is not int:
16         raise TypeError("Expected a Python int")
17     require_int_range(exponent, 0, 31)
18     quotient = value >> exponent
19     assert quotient * (1 << exponent) <= value
20     assert value < (quotient + 1) * (1 << exponent)
21     return quotient
22
23
24 def int8_clip(value: int) -> int:
25     if type(value) is not int:
26         raise TypeError("Expected a Python int")
27     return min(127, max(-128, value))
28
29
30 def shift_gelu_q44(sample: int) -> int:
31     """Q4.4 reference for A.1; exact integer datapath, not exact GELU."""
32     require_int_range(sample, -128, 127)
33     power = (sample + floor_shift(sample, 1)
34             + floor_shift(sample, 3) + floor_shift(sample, 4))
35     assert -216 <= power <= 212
36     delta = -abs(power)
37     base2_power = delta + floor_shift(delta, 1) - floor_shift(delta, 4)
38     whole = (-base2_power) // 16
39     remainder = base2_power + 16 * whole
40     assert -310 <= base2_power <= 0
41     assert 0 <= whole <= 19 and -15 <= remainder <= 0
42     exponent = floor_shift(4096 + 128 * remainder, whole)
43     numerator = 4096 if power >= 0 else exponent
44     denominator = 4096 + exponent
45     gain = (256 * numerator) // denominator
46     assert 0 <= exponent <= 4096 and 0 <= gain <= 256
47     product = sample * gain
48     assert -32768 <= product <= 32512
49     return int8_clip(floor_shift(product, 8))
50
51
52 def residual_dyadic_int8(
53     branch: int, shortcut: int,
54     multiplier_f: int, exponent_f: int,
55     multiplier_x: int, exponent_x: int,
56     scale_locked: bool = False,
57 ) -> int:
58     """A.2 contract; caller aligns tokens and physical scales."""
59     require_int_range(branch, -128, 127)
60     require_int_range(shortcut, -128, 127)
61     require_int_range(multiplier_f, -(1 << 31), (1 << 31) - 1)
62     require_int_range(multiplier_x, -(1 << 31), (1 << 31) - 1)
63     require_int_range(exponent_f, 0, 31)

```

```

64     require_int_range(exponent_x, 0, 31)
65     if type(scale_locked) is not bool:
66         raise TypeError("scale_locked must be bool")
67     if scale_locked:
68         return int8_clip(branch + shortcut)
69     if multiplier_f < 0 or multiplier_x < 0:
70         raise ValueError("Unlocked scale multipliers must be nonnegative")
71     product_f = branch * multiplier_f
72     product_x = shortcut * multiplier_x
73     assert -(1 << 39) <= product_f < (1 << 39)
74     assert -(1 << 39) <= product_x < (1 << 39)
75     summed = (floor_shift(product_f, exponent_f)
76               + floor_shift(product_x, exponent_x))
77     assert -(1 << 40) <= summed < (1 << 40)
78     return int8_clip(summed)
79
80
81 def isqrt_u32_fixed(variance: int) -> int:
82     """Ten clamped Newton iterations; no hardware cycle claim."""
83     require_int_range(variance, 1, (1 << 32) - 1)
84     estimate = 1 << ((variance.bit_length() + 1) // 2)
85     for _iteration in range(10):
86         quotient = variance // estimate
87         summed = estimate + quotient
88         assert 1 <= estimate <= 65536
89         assert 0 <= quotient <= 65537 and summed < (1 << 18)
90         candidate = summed // 2
91         estimate = min(estimate, candidate)
92     assert estimate * estimate <= variance < (estimate + 1) ** 2
93     return estimate
94
95
96 def dot_product(first: Sequence[int], second: Sequence[int],
97                 width: int = 8) -> int:
98     require_int_range(width, 2, 32)
99     if not first or len(first) != len(second):
100         raise ValueError("Expected equal nonempty vectors")
101     lower, upper = -(1 << (width-1)), (1 << (width-1))-1
102     accumulator = 0
103     for left, right in zip(first, second):
104         require_int_range(left, lower, upper)
105         require_int_range(right, lower, upper)
106         accumulator += left * right
107     return accumulator
108
109
110 def packed_products(operand_a: int, operand_b: int,
111                     operand_c: int) -> Tuple[int, int]:
112     for value in (operand_a, operand_b, operand_c):
113         require_int_range(value, -128, 127)
114     product = operand_a * ((operand_b << 18) + operand_c)

```

```

115     low = product & 65535
116     low = low - 65536 if low >= 32768 else low
117     high = (product >> 18) & 65535
118     high = high - 65536 if high >= 32768 else high
119     high += int(low < 0)
120     assert high == operand_a * operand_b and low == operand_a * operand_c
121     return high, low
122
123
124 def loopback_payload(payload: NDArray[np.int8]) -> NDArray[np.int8]:
125     if payload.dtype != np.int8 or payload.ndim != 1 or payload.size == 0:
126         raise ValueError("Expected nonempty 1D INT8 payload")
127     return np.array(payload, dtype=np.int8, order="C", copy=True)
128
129
130 def read_hex(path: Path, width: int, count: int) -> List[int]:
131     if width not in (8, 32) or count <= 0:
132         raise ValueError("Invalid width/count")
133     lines = path.read_text(encoding="ascii").splitlines()
134     if len(lines) != count:
135         raise ValueError("Hex count mismatch")
136     result: List[int] = []
137     for line in lines:
138         if len(line) != width // 4 or any(
139             char not in "0123456789abcdefABCDEF" for char in line):
140             raise ValueError("Malformed hex element")
141         value = int(line, 16)
142         result.append(value - (1 << width) if value >= (1 << (width-1)) else
143             value)
144     return result
145
146 def main() -> None:
147     import math
148     for sample in range(-128, 128):
149         assert -128 <= shift_gelu_q44(sample) <= 127
150         assert packed_products(sample, -128, 127) == (sample*-128, sample*127)
151     for value in range(1, 65536):
152         assert isqrt_u32_fixed(value) == math.isqrt(value)
153     assert dot_product([-128, 127], [-128, 127]) == 32513
154     print("Integer unit self-checks passed; no HDL simulation performed")
155
156
157 if __name__ == "__main__":
158     main()

```

Listing B.3: golden_model_sim.py: integer reference for implemented units

Phụ lục C

Suite Cocotb và cấu hình simulator

C.1 Scoreboard adder tree và wrapper phẳng

Lưu RTL A.3/A.5 trong `rtl/`, golden B.3 trong `sw/`, các listing phụ lục này trong `tests/`. Wrapper chỉ nối lane j vào bit $[jW+ : W]$, không thêm clock, reset, saturation hoặc latency. Phải kiểm tra elaboration trên simulator đã chọn; port phẳng giúp giảm phụ thuộc VPI vào mảng unpacked, không cam kết mọi phiên bản Icarus hỗ trợ mọi cấu trúc SystemVerilog bên trong DUT.

```
1  `timescale 1ns/1ps
2  module adder_tree_flat #(
3      parameter int signed N = 16,
4      parameter int signed IN_WIDTH = 8,
5      parameter int signed OUT_WIDTH = 32
6  ) (
7      input logic unsigned [0:0] clk, rst_n, in_valid,
8      input logic unsigned [N*IN_WIDTH-1:0] a_flat, b_flat,
9      output logic unsigned [0:0] out_valid,
10     output logic signed [OUT_WIDTH-1:0] dot_product
11 );
12     wire signed [IN_WIDTH-1:0] vector_a [0:N-1];
13     wire signed [IN_WIDTH-1:0] vector_b [0:N-1];
14     for (genvar index = 0; index < N; index = index + 1) begin : lanes
15         assign vector_a[index] = $signed(a_flat[index*IN_WIDTH +: IN_WIDTH]);
16         assign vector_b[index] = $signed(b_flat[index*IN_WIDTH +: IN_WIDTH]);
17     end
18     pipelined_adder_tree #(.N(N), .IN_WIDTH(IN_WIDTH), .OUT_WIDTH(OUT_WIDTH))
19         core(.clk(clk), .rst_n(rst_n), .in_valid(in_valid),
20             .a(vector_a), .b(vector_b), .out_valid(out_valid),
21             .dot_product(dot_product));
22 endmodule
```

Listing C.1: `adder_tree_flat.sv`: lane-preserving simulation wrapper

Test so sánh ở mỗi cạnh, kể cả lúc phát vector, và kiểm tra đúng lịch valid. Reset giữa luồng hủy queue epoch cũ. Có thể truyền `VECTORS` trở đến export dot B.2 (tính từ thư mục `tests`) để kiểm tra thêm file hex/checksum; vector ngẫu nhiên và cực trị vẫn luôn được thêm. Test mặc định dùng $W = 8$, các cấu hình khác phải chọn `OUT_WIDTH` đủ lớn.

```

1 from __future__ import annotations
2 import hashlib
3 import json
4 import os
5 import random
6 from collections import deque
7 from pathlib import Path
8 from typing import List, Tuple
9 import cocotb
10 from cocotb.handle import HierarchyObject
11 from cocotb.triggers import ReadOnly, Timer
12 from golden_model_sim import dot_product, read_hex
13
14
15 def pack(values: List[int], width: int) -> int:
16     return sum((value & ((1 << width)-1)) << (index*width)
17               for index, value in enumerate(values))
18
19
20 async def step(dut: HierarchyObject, reset: bool, valid: bool,
21               first: List[int], second: List[int], width: int) -> Tuple[int,
22                               int]:
23     await Timer(1, unit="ns")
24     dut.clk.value = 0
25     dut.rst_n.value = int(not reset)
26     dut.in_valid.value = int(valid)
27     dut.a_flat.value = pack(first, width)
28     dut.b_flat.value = pack(second, width)
29     await Timer(4, unit="ns")
30     dut.clk.value = 1
31     await ReadOnly()
32     output_valid = int(dut.out_valid.value)
33     value = dut.dot_product.value.to_signed() if output_valid else 0
34     await Timer(5, unit="ns")
35     return output_valid, value
36
37 @cocotb.test()
38 async def test_adder(dut: HierarchyObject) -> None:
39     width = int(os.environ.get("WIDTH", "8"))
40     count = int(os.environ.get("N", "16"))
41     assert len(dut.a_flat) == count*width and len(dut.b_flat) == count*width
42     levels = (count-1).bit_length()
43     assert len(dut.dot_product) >= 2*width+levels
44     rng = random.Random(20260914)
45     zero = [0]*count
46     pending = deque([None]*levels)
47     vectors = []
48     directory = os.environ.get("VECTORS")
49     if directory:
50         assert width == 8

```

```

51     root = Path(directory)
52     manifest = json.loads((root / "manifest.json").read_text())
53     assert manifest["kind"] == "dot" and manifest["order"] == "C"
54     for entry in manifest["files"]:
55         assert hashlib.sha256((root/entry["file"]).read_bytes()).hexdigest
56         () == entry["sha256"]
57         entry = next(item for item in manifest["files"] if item["file"] == "a.
58         hex")
59         cases, lanes = entry["shape"]
60         assert lanes == count
61         first = read_hex(root/"a.hex", 8, cases*count)
62         second = read_hex(root/"b.hex", 8, cases*count)
63         expected = read_hex(root/"golden_dot.hex", 32, cases)
64         for index in range(cases):
65             left, right = first[index*count:(index+1)*count], second[index*
66             count:(index+1)*count]
67             assert dot_product(left, right) == expected[index]
68             vectors.append((left, right))
69         file_cases = len(vectors)
70         lower, upper = -(1 << (width-1)), (1 << (width-1))-1
71         vectors += [[lower]*count, [lower]*count), ([upper]*count, [lower]*count)]
72         vectors += [[rng.randint(lower, upper) for _ in range(count)],
73                     [rng.randint(lower, upper) for _ in range(count)]] for _ in
74         range(160)]
75         for _cycle in range(3):
76             actual, _value = await step(dut, True, False, zero, zero, width)
77             assert actual == 0
78             accepted = emitted = 0
79             for cycle, (left, right) in enumerate(vectors):
80                 reset = cycle == file_cases+80
81                 valid = not reset and (cycle < file_cases+2 or rng.random() > 0.25)
82                 if reset:
83                     pending = deque([None]*levels)
84                     accepted = emitted = 0
85                     expected = None
86                 else:
87                     pending.append(dot_product(left, right, width) if valid else None)
88                     expected = pending.popleft()
89                     accepted += int(valid)
90                 actual, value = await step(dut, reset, valid, left, right, width)
91                 assert actual == int(expected is not None), f"valid offset at cycle {
92                 cycle}"
93                 if expected is not None:
94                     assert value == expected, f"data mismatch at cycle {cycle}"
95                     emitted += 1
96             for _cycle in range(levels+3):
97                 pending.append(None)
98                 expected = pending.popleft()
99                 actual, value = await step(dut, False, False, zero, zero, width)
100                 assert actual == int(expected is not None)
101                 if expected is not None:

```

```

97         assert value == expected
98         emitted += 1
99     assert accepted == emitted and all(item is None for item in pending)

```

Listing C.2: test_adder_tree.py: cycle-indexed scoreboard and reset epochs

C.2 AXI-Stream và AXI4-Lite của shell loopback

Test khởi tạo mọi input, reset DUT, cấu hình NBYTES và START trước truyền. Snapshot trước cạnh dùng quyết định handshake; đọc sau cạnh chỉ dùng kiểm tra thanh ghi. Monitor giữ payload khi stall cho stream/B/R. Các packet hợp lệ được so sánh toàn beat và đếm số lượng; packet TLAST sớm được kỳ vọng ERROR, không được coi là suy luận đúng. Bounded loops thất bại rõ ràng thay vì treo. Suite này không phải chứng minh exhaustive AXI compliance và không kiểm tra GEMM/Softmax chưa tích hợp.

```

1  from __future__ import annotations
2  import random
3  from collections import deque
4  from typing import Dict, Tuple
5  import cocotb
6  from cocotb.handle import HierarchyObject
7  from cocotb.triggers import ReadOnly, Timer
8
9
10 class Bench:
11     def __init__(self, dut: HierarchyObject) -> None:
12         self.dut = dut
13         self.inputs = {name: 0 for name in (
14             "s_axi_awaddr", "s_axi_awprot", "s_axi_awvalid", "s_axi_wdata",
15             "s_axi_wstrb", "s_axi_wvalid", "s_axi_bready", "s_axi_araddr",
16             "s_axi_arprot", "s_axi_arvalid", "s_axi_rready", "s_axis_tdata",
17             "s_axis_tkeep", "s_axis_tlast", "s_axis_tvalid", "m_axis_tready"))}
18         self.inputs["rst_n"] = 0
19         self.held: Dict[str, Tuple[int, ...]] = {}
20
21     async def cycle(self, updates: Dict[str, int]) -> Dict[str, int]:
22         await Timer(1, unit="ns")
23         self.dut.clk.value = 0
24         self.inputs.update(updates)
25         for name, value in self.inputs.items():
26             setattr(self.dut, name).value = value
27         await Timer(4, unit="ns")
28         names = ("s_axi_awready", "s_axi_wready", "s_axi_bvalid", "s_axi_bresp",
29             ,
30             "s_axi_arready", "s_axi_rvalid", "s_axi_rdata", "s_axi_rresp",
31             "s_axis_tready", "m_axis_tvalid", "m_axis_tdata",
32             "m_axis_tkeep", "m_axis_tlast")
33         before = {name: int(getattr(self.dut, name).value) for name in names}
34         if not self.inputs["rst_n"]:
35             self.held.clear()
36         else:

```

```

36     channels = {
37         "stream": ("m_axis_tvalid", "m_axis_tready",
38                 ("m_axis_tdata", "m_axis_tkeep", "m_axis_tlast")),
39         "b": ("s_axi_bvalid", "s_axi_bready", ("s_axi_bresp",)),
40         "r": ("s_axi_rvalid", "s_axi_rready", ("s_axi_rdata", "
s_axi_rresp"))})
41     for channel, (valid, ready, fields) in channels.items():
42         payload = tuple(before[field] for field in fields)
43         if channel in self.held:
44             assert before[valid] == 1 and payload == self.held[channel]
45             if before[valid] and not self.inputs[ready]:
46                 self.held[channel] = payload
47             else:
48                 self.held.pop(channel, None)
49     self.dut.clk.value = 1
50     await ReadOnly()
51     await Timer(5, unit="ns")
52     return before
53
54 async def reset(self) -> None:
55     for _cycle in range(3):
56         await self.cycle({name: 0 for name in self.inputs})
57         await self.cycle({"rst_n": 1})
58         assert int(self.dut.m_axis_tvalid.value) == 0
59
60 async def write(self, address: int, data: int, strobe: int = 15,
61                aw_delay: int = 0, w_delay: int = 3,
62                response: int = 0) -> None:
63     aw_done = w_done = False
64     held_cycles = 0
65     for cycle in range(100):
66         aw_valid, w_valid = int(not aw_done and cycle >= aw_delay), int(not
w_done and cycle >= w_delay)
67         ready = int(held_cycles >= 3)
68         before = await self.cycle({"s_axi_awaddr": address, "s_axi_awvalid"
: aw_valid,
69                                   "s_axi_wdata": data, "s_axi_wstrb": strobe,
70                                   "s_axi_wvalid": w_valid, "s_axi_bready": ready})
71         aw_done |= bool(aw_valid and before["s_axi_awready"])
72         w_done |= bool(w_valid and before["s_axi_wready"])
73         if before["s_axi_bvalid"]:
74             if ready:
75                 assert aw_done and w_done and before["s_axi_bresp"] ==
response
76                 await self.cycle({"s_axi_awvalid": 0, "s_axi_wvalid": 0, "
s_axi_bready": 0})
77                 return
78             held_cycles += 1
79         raise AssertionError("AXI-Lite write timeout")
80
81 async def read(self, address: int, response: int = 0) -> int:

```

```

82     accepted = False
83     held_cycles = 0
84     for _cycle in range(100):
85         ready = int(held_cycles >= 3)
86         before = await self.cycle({"s_axi_araddr": address,
87             "s_axi_arvalid": int(not accepted), "s_axi_rready": ready})
88         accepted |= bool(before["s_axi_arready"] and not accepted)
89         if before["s_axi_rvalid"]:
90             if ready:
91                 assert accepted and before["s_axi_rresp"] == response
92                 await self.cycle({"s_axi_arvalid": 0, "s_axi_rready": 0})
93                 return before["s_axi_rdata"]
94             held_cycles += 1
95     raise AssertionError("AXI-Lite read timeout")
96
97 async def packet(self, payload: bytes, early_last: bool = False) -> None:
98     await self.write(4, len(payload))
99     await self.write(0, 1)
100     beats = []
101     for offset in range(0, len(payload), 8):
102         fragment = payload[offset:offset+8]
103         beats.append((int.from_bytes(fragment.ljust(8, b"\0"), "little"),
104             (1 << len(fragment))-1, int(offset+8 >= len(payload))
105 ))
106     if early_last:
107         beats = [(beats[0][0], beats[0][1], 1)]
108     queue = deque()
109     index = received = 0
110     active = False
111     rng = random.Random(20260914 + len(payload))
112     for cycle in range(2000):
113         if not active and index < len(beats) and rng.random() > 0.25:
114             active = True
115         data, keep, last = beats[index] if active else (0, 0, 0)
116         ready = int(cycle >= 10 and rng.random() > 0.35)
117         before = await self.cycle({"s_axis_tvalid": int(active),
118             "s_axis_tdata": data, "s_axis_tkeep": keep,
119             "s_axis_tlast": last, "m_axis_tready": ready})
120         if active and before["s_axis_tready"]:
121             queue.append((data, keep, last))
122             index += 1
123             active = False
124         if before["m_axis_tvalid"] and ready:
125             assert queue, "Unexpected output beat"
126             assert (before["m_axis_tdata"], before["m_axis_tkeep"], before[
127 "m_axis_tlast"]) == queue.popleft()
128             received += 1
129             if index == len(beats) and not queue:
130                 break
131     else:
132         raise AssertionError("Stream transfer timeout")

```

```

131     assert received == len(beats)
132     for _cycle in range(3):
133         before = await self.cycle({"s_axis_tvalid": 0, "m_axis_tready": 1})
134         assert before["m_axis_tvalid"] == 0, "Trailing duplicate output"
135     status = await self.read(0)
136     assert status & 3 == 2
137     assert bool(status & 4) == early_last
138
139
140 @cocotb.test()
141 async def test_axis_and_control(dut: HierarchyObject) -> None:
142     bench = Bench(dut)
143     await bench.reset()
144     assert await bench.read(12) == 0x56495432
145     assert await bench.read(8) == 1
146     await bench.write(0, 1, response=2)
147     assert (await bench.read(0)) & 4
148     await bench.write(0, 2)
149     await bench.write(16, 0x11223344, aw_delay=4, w_delay=0)
150     await bench.write(16, 0xAABBCCDD, strobe=5)
151     assert await bench.read(16) == 0x11BB33DD
152     await bench.read(2, response=2)
153     await bench.write(12, 0, response=2)
154     for length in (1, 7, 8, 9, 17, 64, 129):
155         await bench.packet(bytes((index*37)&255 for index in range(length)))
156     await bench.packet(bytes(range(17)), early_last=True)
157     await bench.reset()
158     await bench.write(4, 16)
159     await bench.write(0, 1)
160     await bench.write(16, 99, response=2)
161     before = await bench.cycle({"s_axis_tvalid": 1, "s_axis_tdata": 123,
162                                "s_axis_tkeep": 255, "s_axis_tlast": 0, "m_axis_tready": 0})
163     assert before["s_axis_tready"] == 1
164     await bench.cycle({"s_axis_tvalid": 0})
165     assert int(dut.m_axis_tvalid.value) == 1
166     await bench.reset()
167     assert await bench.read(0) == 0

```

Listing C.3: test_attention_axis.py: backpressure, control channels and error recovery

C.3 Makefile cho hai simulator

Chạy từ tests/. Ví dụ thứ nhất: make CASE=addder SIM=icarus N=5. Ví dụ thứ hai: make CASE=axis SIM=verilator. Chạy thêm $N = 1, 3, 16, 17$ để bao phủ trường hợp không có tầng cộng và padding. Bản Makefile Cocotb 2.0.1 yêu cầu Verilator từ 5.036 trở lên [27]. Các tham số build nằm trong tên thư mục cache. Không truyền cờ Verilator cho Icarus và không dùng -Wno-fatal để che mọi warning. Không định nghĩa SYNTHESIS trong simulation vì cần assertions. File kết quả phải có số test thực chạy và không có failure/error; việc Makefile được parse không có nghĩa simulation đã đạt.

```
1 SIM ?= icarus
```

```

2 CASE ?= adder
3 N ?= 16
4 WIDTH ?= 8
5 OUT_WIDTH ?= 32
6 TOPLEVEL_LANG := verilog
7 export N WIDTH
8 export PYTHONPATH := ../sw:${PYTHONPATH}
9 SIM_BUILD := sim_build/${SIM}_${CASE}_${N}_${WIDTH}_${OUT_WIDTH}
10 COCOTB_RESULTS_FILE := ${SIM_BUILD}/results.xml
11 ifeq ($(CASE),adder)
12 COCOTB_TOPLEVEL := adder_tree_flat
13 COCOTB_TEST_MODULES := test_adder_tree
14 VERILOG_SOURCES := ../rtl/pipelined_adder_tree.sv adder_tree_flat.sv
15 ifeq ($(SIM),icarus)
16 COMPILE_ARGS += -g2012 -Padder_tree_flat.N=$(N)
17 COMPILE_ARGS += -Padder_tree_flat.IN_WIDTH=$(WIDTH) -Padder_tree_flat.OUT_WIDTH
    =$(OUT_WIDTH)
18 else ifeq ($(SIM),verilator)
19 COMPILE_ARGS += --assert -GN=$(N) -GIN_WIDTH=$(WIDTH) -GOUT_WIDTH=$(OUT_WIDTH)
20 endif
21 else ifeq ($(CASE),axis)
22 COCOTB_TOPLEVEL := attention_core_top
23 COCOTB_TEST_MODULES := test_attention_axis
24 VERILOG_SOURCES := ../rtl/attention_core_top.sv
25 ifeq ($(SIM),icarus)
26 COMPILE_ARGS += -g2012
27 else ifeq ($(SIM),verilator)
28 COMPILE_ARGS += --assert
29 endif
30 else
31 $(error CASE must be adder or axis)
32 endif
33 ifeq ($(filter $(SIM),icarus verilator),)
34 $(error SIM must be icarus or verilator)
35 endif
36 COCOTB_MAKEFILES ?= $(shell cocotb-config --makefiles)
37 ifeq ($(strip $(COCOTB_MAKEFILES)),)
38 $(error cocotb-config is unavailable; use the documented Cocotb environment)
39 endif
40 include $(COCOTB_MAKEFILES)/Makefile.sim

```

Listing C.4: Makefile: simulator-specific flags and isolated build directories

Phụ lục D

Trình điều khiển PYNQ và webcam HIL

D.1 Host webcam: realtime_vit_inference.py

D.1.1 Phạm vi thực thi và cấu hình bắt buộc

Tên tệp được giữ theo đề cương, nhưng chương trình chỉ chạy **HIL loopback có đối chiếu dữ liệu**, không chạy inference ViT. Nó không tải checkpoint, không tạo QKV ngẫu nhiên và không hiển thị nhãn phân loại. Ảnh nhận được giải mã từ buffer trả về của FPGA sau khi kiểm tra từng byte.

Môi trường đích cần PYNQ, NumPy, OpenCV, camera và overlay đã build bên ngoài tài liệu. Tệp .hwh phải cùng tên gốc với .bit; tên hai IP phải khớp tham số dòng lệnh. Hai stream phải rộng 64 bit. DMA dùng simple mode, không dùng scatter-gather hoặc micro-DMA. Với ảnh $224 \times 224 \times 3$, LENGTH phải chứa được 150528 byte; script từ chối cấu hình có giới hạn nhỏ hơn thay vì đợi DMA treo.

Các helper MMIO trong listing phục vụ duy nhất chương trình này, chưa thay thế module driver D.2 dự kiến của Batch 3. Không gọi đồng thời driver PYNQ `transfer()` và `wait()` trên cùng DMA: mã này sở hữu các thanh ghi điều khiển của cả hai kênh. Các offset và cờ reset/halted/error/IOC được dùng theo AXI DMA PG021 [5].

D.1.2 Tiền/hậu điều kiện và quyền sở hữu tài nguyên

Các type hints xác định `PynqBuffer` cho vùng DMA, `NDArray[np.uint8]` cho ảnh BGR và `NDArray[np.int8]` cho payload; không thay thế kiểm tra shape, miền, địa chỉ hoặc quyền sở hữu runtime. `HostOptions` là cấu hình có kiểu, không phải descriptor attention. Giữ tham chiếu TX/RX từ khi allocate đến khi DMA quiescent; không dùng context manager tự động free buffer trong lúc timeout chưa xác nhận dừng [9, 16].

Invariant của vòng xử lý. Đầu mỗi vòng, cả hai buffer còn được cấp phát, shell idle, frame trước đã hoàn tất hoặc chưa có giao dịch. Sau prepare/flush, PS nhường quyền truy cập cho DMA; sau `wait_job`, kiểm tra LENGTH và invalidate, PS mới được so sánh/đọc RX. Nếu một bước lỗi, không đi tiếp vòng sau; chuyển cleanup. `try...finally` bao cả hai lần cấp phát: nếu cấp phát RX thất bại sau TX thì TX vẫn được thu hồi qua danh sách đã giữ. Cleanup camera và cửa sổ được lồng finally để một lỗi đóng camera không bỏ qua dọn UI/DMA.

Điều kiện giải phóng bộ nhớ. Việc giải phóng có tiền điều kiện DMA đã dừng.

Chỉ gọi `freebuffer()` sau reset hoàn tất hoặc xác nhận HALTED cả hai kênh, theo giả thiết PG021 và MMIO còn phản hồi. Nếu một lần free báo lỗi khi đã quiescent, vẫn thử các buffer còn lại rồi báo lỗi; không tuyên bố tất cả đã giải phóng. Nếu DMA không quiescent, chương trình giữ tham chiếu CMA và chờ phục hồi vô hạn. Safety và khả năng luôn thoát hữu hạn không đồng thời được bảo đảm trong tình huống phần cứng không thể dừng. Host phải đọc quyền điều khiển DMA; tác nhân khác khởi động DMA sau kiểm tra HALTED sẽ phá tiền điều kiện.

D.1.3 Mã nguồn hoàn chỉnh

```
1 from __future__ import annotations
2
3 import argparse
4 from dataclasses import dataclass
5 import math
6 from pathlib import Path
7 import sys
8 import time
9 from typing import Callable, List, Mapping, Optional, Tuple, Union
10
11 import cv2
12 import numpy as np
13 from numpy.typing import NDArray
14 from pynq import MMIO, Overlay, allocate
15 from pynq.buffer import PynqBuffer
16
17
18 CORE_ID: int = 0x56495432
19 DMA_ERRORS: int = 0x70
20 DMA_COMPLETE: int = 0x1002
21
22
23 @dataclass(frozen=True)
24 class HostOptions:
25     bitstream: str = "attention_core.bit"
26     dma_name: str = "axi_dma_0"
27     core_name: str = "attention_core_0"
28     camera: int = 0
29     width: int = 224
30     height: int = 224
31     max_frames: int = 0
32     timeout: float = 2.0
33     no_display: bool = False
34
35
36 def wait_until(
37     predicate: Callable[[], bool], timeout: float, message: str,
38 ) -> None:
39     if not math.isfinite(timeout) or timeout <= 0:
40         raise ValueError("Timeout must be finite and positive")
41     deadline = time.monotonic() + timeout
```

```

42     while not predicate():
43         if time.monotonic() >= deadline:
44             raise TimeoutError(message)
45         time.sleep(0.0001)
46
47
48 def dma_halted(dma: MMIO) -> bool:
49     return bool((dma.read(0x04) & 1) and
50                 (dma.read(0x34) & 1))
51
52
53 def reset_dma(dma: MMIO, timeout: float) -> None:
54     dma.write(0x00, 0x04)
55     wait_until(
56         lambda: not (dma.read(0x00) & 4)
57         and not (dma.read(0x30) & 4) and dma_halted(dma),
58         timeout, "DMA reset did not complete")
59
60
61 def start_channel(
62     dma: MMIO, base: int, buffer: PynqBuffer,
63     nbytes: int, timeout: float,
64 ) -> None:
65     if base not in (0x00, 0x30):
66         raise ValueError("Unknown DMA channel base")
67     if nbytes <= 0 or nbytes > buffer.nbytes:
68         raise ValueError("Transfer exceeds allocated buffer")
69     dma.write(base, 0x01)
70     wait_until(lambda: not (dma.read(base + 4) & 1),
71               timeout, "DMA channel did not start")
72     dma.write(base + 4, 0x7000)
73     address = int(buffer.device_address)
74     dma.write(base + 0x18, address & 0xffffffff)
75     dma.write(base + 0x1c, (address >> 32) & 0xffffffff)
76     dma.write(base + 0x28, nbytes)
77
78
79 def wait_job(dma: MMIO, core: MMIO, timeout: float) -> None:
80     deadline = time.monotonic() + timeout
81     while True:
82         tx_status = dma.read(0x04)
83         rx_status = dma.read(0x34)
84         core_status = core.read(0x00)
85         if (tx_status | rx_status) & DMA_ERRORS:
86             raise RuntimeError(
87                 f"DMA error: TX=0x{tx_status:08x}, "
88                 f"RX=0x{rx_status:08x}")
89         if (tx_status | rx_status) & 1:
90             raise RuntimeError("DMA halted during transfer")
91         if core_status & 4:
92             raise RuntimeError("Core reports packet framing error")

```

```

93     tx_done = (tx_status & DMA_COMPLETE) == DMA_COMPLETE
94     rx_done = (rx_status & DMA_COMPLETE) == DMA_COMPLETE
95     if tx_done and rx_done and (core_status & 3) == 2:
96         return
97     if time.monotonic() >= deadline:
98         raise TimeoutError("HIL transfer timed out")
99     time.sleep(0.0001)
100
101
102 def release_buffers_safely(
103     dma: MMIO, buffers: List[PynqBuffer], timeout: float,
104 ) -> None:
105     """Keep CMA alive until both DMA channels are quiescent."""
106     if not buffers:
107         return
108     try:
109         reset_dma(dma, timeout)
110     except (Exception, KeyboardInterrupt):
111         print("DMA is not quiescent; retaining CMA buffers. "
112             "Reset the board if hardware cannot halt.",
113             file=sys.stderr, flush=True)
114         while True:
115             try:
116                 if dma_halted(dma):
117                     break
118                 time.sleep(0.1)
119             except (Exception, KeyboardInterrupt):
120                 continue
121     failures: List[Exception] = []
122     for buffer in tuple(buffers):
123         try:
124             buffer.freebuffer()
125         except Exception as error:
126             failures.append(error)
127     if failures:
128         raise RuntimeError("CMA release failed after DMA halted") from failures
129     buffers.clear()
130
131
132 def validate_dma(
133     parameters: Mapping[str, Union[str, int]], nbytes: int,
134 ) -> Tuple[int, int, int]:
135     params = {key.lower(): value
136               for key, value in parameters.items()}
137     for name in ("c_include_mm2s", "c_include_s2mm"):
138         if int(params[name]) != 1:
139             raise ValueError("Both DMA channels are required")
140     for name in ("c_include_sg", "c_micro_dma"):
141         if int(params.get(name, 0)) != 0:
142             raise ValueError("Only non-micro simple DMA is supported")

```

```

143     for name in ("c_m_axis_mm2s_tdata_width",
144                 "c_s_axis_s2mm_tdata_width"):
145         if int(params[name]) != 64:
146             raise ValueError("DMA stream widths must be 64 bits")
147     length_bits = int(params["c_sg_length_width"])
148     if not 1 <= length_bits <= 32:
149         raise ValueError("Invalid LENGTH field width")
150     maximum = min((1 << length_bits) - 1, 0x03ffffff)
151     if nbytes <= 0 or nbytes > maximum:
152         raise ValueError(f"Payload {nbytes} exceeds limit {maximum}")
153     address_bits = int(params.get("c_addr_width", 32))
154     if not 1 <= address_bits <= 64:
155         raise ValueError("Unsupported DMA address width")
156     tx_width = int(params["c_m_axi_mm2s_data_width"])
157     rx_width = int(params["c_m_axi_s2mm_data_width"])
158     if tx_width <= 0 or rx_width <= 0 or tx_width % 8 or rx_width % 8:
159         raise ValueError("DMA memory widths must contain whole bytes")
160     tx_alignment = tx_width // 8
161     rx_alignment = rx_width // 8
162     return address_bits, tx_alignment, rx_alignment
163
164
165 def prepare_frame(
166     frame: NDArray[np.uint8], width: int, height: int,
167 ) -> NDArray[np.int8]:
168     if width <= 0 or height <= 0:
169         raise ValueError("Target image dimensions must be positive")
170     if (not isinstance(frame, np.ndarray) or frame.dtype != np.uint8
171         or frame.ndim != 3 or frame.shape[2] != 3
172         or frame.shape[0] == 0 or frame.shape[1] == 0):
173         raise ValueError("Expected a nonempty HxWx3 uint8 BGR frame")
174     resized = cv2.resize(frame, (width, height),
175                          interpolation=cv2.INTER_AREA)
176     rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)
177     signed_pixels = (rgb.astype(np.int16) - 128).astype(np.int8)
178     return np.ascontiguousarray(signed_pixels).reshape(-1)
179
180
181 def run_realtime_vit_demo(args: HostOptions) -> None:
182     bitstream = Path(args.bitstream)
183     if bitstream.is_absolute():
184         raise ValueError("Use a project-relative bitstream path")
185     if not bitstream.is_file() or not bitstream.with_suffix(".hwh").is_file():
186         raise FileNotFoundError("Matching .bit and .hwh files required")
187     if args.width <= 0 or args.height <= 0 or args.max_frames < 0:
188         raise ValueError("Invalid image size or frame limit")
189     if not math.isfinite(args.timeout) or args.timeout <= 0:
190         raise ValueError("Timeout must be finite and positive")
191
192     overlay = Overlay(str(bitstream))
193     dma_info = overlay.ip_dict[args.dma_name]

```

```

194 core_info = overlay.ip_dict[args.core_name]
195 nbytes = args.width * args.height * 3
196 address_bits, tx_align, rx_align = validate_dma(
197     dma_info["parameters"], nbytes)
198 dma = MMIO(dma_info["phys_addr"], dma_info["addr_range"])
199 core = MMIO(core_info["phys_addr"], core_info["addr_range"])
200 if core.read(0x0c) != CORE_ID or core.read(0x08) != 1:
201     raise RuntimeError("Overlay is not the Batch-2 loopback shell")
202 if core.read(0x00) & 1:
203     raise RuntimeError("Core is busy; reload/reset the overlay")
204
205 buffers: List[PynqBuffer] = []
206 camera: Optional[cv2.VideoCapture] = None
207 tx_buffer: PynqBuffer
208 rx_buffer: PynqBuffer
209 completed = 0
210 loop_fps = 0.0
211 try:
212     reset_dma(dma, args.timeout)
213     buffers.append(allocate(shape=(nbytes,), dtype=np.int8))
214     buffers.append(allocate(shape=(nbytes,), dtype=np.int8))
215     tx_buffer, rx_buffer = buffers
216     for buffer in buffers:
217         if (buffer.shape != (nbytes,) or buffer.dtype != np.int8
218             or not buffer.flags.c_contiguous):
219             raise ValueError("CMA shape/dtype/contiguity mismatch")
220     for buffer, alignment in zip(buffers, (tx_align, rx_align)):
221         address = int(buffer.device_address)
222         if alignment <= 0 or address % alignment:
223             raise ValueError("DMA buffer alignment mismatch")
224         if address < 0 or address + nbytes > (1 << address_bits):
225             raise ValueError("DMA buffer exceeds address width")
226     tx_address = int(tx_buffer.device_address)
227     rx_address = int(rx_buffer.device_address)
228     if max(tx_address, rx_address) < min(
229         tx_address + nbytes, rx_address + nbytes):
230         raise ValueError("DMA buffers must not overlap")
231
232     core.write(0x00, 2)
233     core.write(0x04, nbytes)
234     for offset in (0x10, 0x18, 0x1c):
235         core.write(offset, 0)
236     camera = cv2.VideoCapture(args.camera)
237     if not camera.isOpened():
238         raise RuntimeError("Cannot open camera")
239     camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
240     camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
241     print("HIL LOOPBACK ONLY: no ViT inference or class labels")
242     run_start = time.perf_counter()
243
244     while args.max_frames == 0 or completed < args.max_frames:

```

```

245     success, frame = camera.read()
246     if not success:
247         raise RuntimeError("Camera frame capture failed")
248     np.copyto(tx_buffer, prepare_frame(
249         frame, args.width, args.height))
250     rx_buffer.fill(0)
251     tx_buffer.flush()
252     rx_buffer.flush()
253     if core.read(0x00) & 1:
254         raise RuntimeError("Core did not return to idle")
255
256     transfer_start = time.perf_counter()
257     start_channel(dma, 0x30, rx_buffer, nbytes, args.timeout)
258     core.write(0x00, 1)
259     start_channel(dma, 0x00, tx_buffer, nbytes, args.timeout)
260     wait_job(dma, core, args.timeout)
261     transfer_ms = 1000 * (time.perf_counter() - transfer_start)
262     if dma.read(0x58) != nbytes:
263         raise RuntimeError("S2MM received an unexpected byte count")
264     rx_buffer.invalidate()
265     if not np.array_equal(np.asarray(tx_buffer),
266                          np.asarray(rx_buffer)):
267         raise RuntimeError("Loopback payload mismatch")
268
269     key = -1
270     if not args.no_display:
271         rgb = (np.asarray(rx_buffer).reshape(
272             args.height, args.width, 3).astype(np.int16)
273             + 128).astype(np.uint8)
274         display = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
275         label = f"LOOPBACK | {transfer_ms:.2f} ms"
276         cv2.putText(display, label, (5, 20),
277                     cv2.FONT_HERSHEY_SIMPLEX, 0.4,
278                     (0, 255, 0), 1)
279         cv2.putText(display, f"Loop FPS: {loop_fps:.1f}",
280                     (5, 40), cv2.FONT_HERSHEY_SIMPLEX,
281                     0.4, (0, 255, 0), 1)
282         cv2.imshow("HIL loopback - not ViT inference", display)
283         key = cv2.waitKey(1) & 0xff
284     completed += 1
285     elapsed = time.perf_counter() - run_start
286     loop_fps = completed / elapsed
287     if key == ord("q"):
288         break
289     print(f"Verified {completed} frames; loop FPS={loop_fps:.2f}")
290 finally:
291     try:
292         try:
293             if camera is not None:
294                 camera.release()
295         finally:

```

```

296         if not args.no_display:
297             cv2.destroyAllWindows()
298     finally:
299         release_buffers_safely(dma, buffers, args.timeout)
300
301
302 def main() -> None:
303     parser = argparse.ArgumentParser(
304         description="Webcam DMA loopback validation, not ViT inference")
305     parser.add_argument("--bitstream", default="attention_core.bit")
306     parser.add_argument("--dma-name", default="axi_dma_0")
307     parser.add_argument("--core-name", default="attention_core_0")
308     parser.add_argument("--camera", type=int, default=0)
309     parser.add_argument("--width", type=int, default=224)
310     parser.add_argument("--height", type=int, default=224)
311     parser.add_argument("--max-frames", type=int, default=0)
312     parser.add_argument("--timeout", type=float, default=2.0)
313     parser.add_argument("--no-display", action="store_true")
314     run_realtime_vit_demo(HostOptions(**vars(parser.parse_args()))))
315
316
317 if __name__ == "__main__":
318     main()

```

Listing D.1: realtime_vit_inference.py: bounded-transfer webcam loopback with byte verification

D.1.4 An toàn buffer và giới hạn timeout

Vòng chờ giao dịch và khởi động/reset thông thường có timeout. Nhánh cleanup chỉ trả CMA sau khi reset hoàn tất hoặc đã xác nhận cả hai kênh HALTED. Nếu phần cứng không thể dừng, chương trình cố ý giữ buffer và chờ phục hồi; nó không tuyên bố timeout đồng nghĩa DMA đã ngừng. Nhánh fail-stop này có thể chờ vô hạn và bỏ qua Ctrl-C để tránh giải phóng vùng nhớ còn bị DMA truy cập. Khi clock/interconnect hỏng, cần reset board theo quy trình vận hành; không kill tiến trình rồi tái sử dụng CMA khi DMA vẫn hoạt động.

Mã giả định MMIO còn phản hồi và địa chỉ/clock/reset trong block design là đúng. Một truy cập MMIO bị treo ở mức bus không thể được xử lý chỉ bằng bộ đếm thời gian Python. Sau lỗi framing hoặc timeout giữa packet, phải reset/ nạp lại cả shell lẫn DMA trước phiên mới; không tiếp tục từ trạng thái BUSY cũ.

Tốc độ hiển thị là FPS trung bình của các frame đã hoàn tất; dòng trên ảnh hiển thị giá trị tính ở vòng trước để bao gồm cả thời gian UI. Thời gian truyền gồm thao tác điều khiển/đợi ở PS và DMA, không phải latency của attention. Script không đo điện năng, không cung cấp GOPS/W và không chứng minh một thiết kế đạt thời gian thực.

D.1.5 Trạng thái kiểm tra của bản biên soạn

Trong môi trường soạn thảo hiện tại không có PYNQ, OpenCV, simulator SystemVerilog hoặc Vivado. Listing Python đã được kiểm tra cú pháp và chạy luồng điều khiển với camera, buffer, MMIO giả. Các ca kiểm tra gồm hai frame liên tiếp, payload sai, cờ lỗi

DMA, timeout và cấu hình HWH không hỗ trợ; chúng xác nhận thứ tự điều khiển và cleanup trong mô hình giả, không thay thế phép thử board hoặc thư viện PYNQ/OpenCV thật. Các module RTL cần được elaboration, kiểm tra AXI với backpressure và chạy synthesis ở môi trường đích trước khi dùng cho thực nghiệm.

D.1.6 Kiểm tra của lần rigor refactoring

Ngày 14/09/2026, toàn listing Python được kiểm tra AST, bao gồm annotation của tham số và kiểu trả về. Luồng thực thi được chạy với camera, MMIO, overlay và buffer mô phỏng bằng NumPy, không dùng PYNQ/OpenCV thật. Bẫy tình huống được kiểm tra: phiên loopback thành công; byte không khớp; camera không trả frame; cấp phát RX thất bại sau TX; DMA báo lỗi; một buffer báo lỗi khi free; đóng cửa sổ UI báo lỗi. Các buffer đã cấp phát đều được đưa vào nhánh cleanup; khi một free thất bại, vẫn thử buffer còn lại và báo lỗi, không coi lần free thất bại là giải phóng thành công.

Kiểm tra riêng xác nhận flush trước transfer, arm RX trước START và TX, invalidate trước đọc RX, và không gọi free trước DMA HALTED. Với reset timeout được tiêm lỗi, hai lần poll đầu vẫn giữ CMA; chỉ lần poll phục hồi xác nhận HALTED mới cho free. Thử thêm timeout vòng đợi và từ chối frame sai shape/dtype, metadata sai độ rộng, LENGTH ngoài giới hạn. Thời gian/FPS in ra bởi mock không phải benchmark bo mạch và không được dùng làm số liệu thực nghiệm.

Phần số học kiểm tra lại toàn bộ 256^3 bộ ba DSP packing, 4000 trường hợp adder tree nhiều chiều/độ rộng, và 1000 GEMM tiled ba chiều có tile biên. Mô hình thanh ghi trừu tượng kiểm tra thứ tự valid/bubble/reset; mô hình AXI-Lite kiểm tra các bất biến đếm giao dịch với AW/W đến độc lập và B/R bị stall. Các mô hình này không thực thi listing HDL. Assertions trong A.3–A.5 vẫn chưa được chạy bằng simulator; chưa có kiểm tra Lean kernel, static type checker, synthesis/STA hoặc phép đo KV260.

D.2 Driver CMA/DMA tái sử dụng

Tệp `pynq_dma_driver.py`. D.2 cung cấp context manager cho chính shell loopback, không phải driver attention chưa hiện thực. Nó dùng cùng register map và helper D.1 nhưng không import OpenCV. Driver sở hữu độc quyền hai kênh DMA, một cặp TX/RX không chồng lấp, không thay đổi tượng allocate bằng ndarray thường. Mỗi lần run trả bản sao RX chỉ sau khi invalidate và đổi chiều; có thể tái sử dụng cặp buffer tuần tự. Khóa thread bảo vệ run/close trong một instance, không bảo vệ trước tiến trình khác hay instance khác điều khiển cùng IP.

Bất biến: trước run, buffer còn sống và shell idle; trong DMA, không free hay sửa buffer; sau lỗi, driver bị poisoned và không nhận job mới. `close()` chỉ trả CMA sau quiescence, có thể chờ vô hạn theo quy tắc xử lý tại D.1 nếu DMA không dừng. Phải dùng context manager hoặc gọi `close()` trong `finally`; không dựa vào garbage collector để dừng DMA. Sau lỗi giữa packet, reset/ nạp lại cả shell và DMA trước phiên mới. Không có ping-pong DMA bất đồng bộ trong lớp này: muốn hai job chồng lấp phải thêm descriptor và chứng minh quyền sở hữu khác.

```
1 from __future__ import annotations
2 import math
3 import sys
4 import time
5 from threading import Lock
```

```

6 from types import TracebackType
7 from typing import Callable, List, Mapping, Optional, Tuple, Type, Union
8 import numpy as np
9 from numpy.typing import NDArray
10 from pynq import MMIO, Overlay, allocate
11 from pynq.buffer import PynqBuffer
12
13 CORE_ID = 0x56495432
14 DMA_ERRORS = 0x70
15 DMA_COMPLETE = 0x1002
16
17 def wait_until(
18     predicate: Callable[[], bool], timeout: float, message: str,
19 ) -> None:
20     if not math.isfinite(timeout) or timeout <= 0:
21         raise ValueError("Timeout must be finite and positive")
22     deadline = time.monotonic() + timeout
23     while not predicate():
24         if time.monotonic() >= deadline:
25             raise TimeoutError(message)
26         time.sleep(0.0001)
27
28
29 def dma_halted(dma: MMIO) -> bool:
30     return bool((dma.read(0x04) & 1) and
31                (dma.read(0x34) & 1))
32
33
34 def reset_dma(dma: MMIO, timeout: float) -> None:
35     dma.write(0x00, 0x04)
36     wait_until(
37         lambda: not (dma.read(0x00) & 4)
38         and not (dma.read(0x30) & 4) and dma_halted(dma),
39         timeout, "DMA reset did not complete")
40
41
42 def start_channel(
43     dma: MMIO, base: int, buffer: PynqBuffer,
44     nbytes: int, timeout: float,
45 ) -> None:
46     if base not in (0x00, 0x30):
47         raise ValueError("Unknown DMA channel base")
48     if nbytes <= 0 or nbytes > buffer.nbytes:
49         raise ValueError("Transfer exceeds allocated buffer")
50     dma.write(base, 0x01)
51     wait_until(lambda: not (dma.read(base + 4) & 1),
52               timeout, "DMA channel did not start")
53     dma.write(base + 4, 0x7000)
54     address = int(buffer.device_address)
55     dma.write(base + 0x18, address & 0xffffffff)
56     dma.write(base + 0x1c, (address >> 32) & 0xffffffff)

```

```

57     dma.write(base + 0x28, nbytes)
58
59
60 def wait_job(dma: MMIO, core: MMIO, timeout: float) -> None:
61     deadline = time.monotonic() + timeout
62     while True:
63         tx_status = dma.read(0x04)
64         rx_status = dma.read(0x34)
65         core_status = core.read(0x00)
66         if (tx_status | rx_status) & DMA_ERRORS:
67             raise RuntimeError(
68                 f"DMA error: TX=0x{tx_status:08x}, "
69                 f"RX=0x{rx_status:08x}")
70         if (tx_status | rx_status) & 1:
71             raise RuntimeError("DMA halted during transfer")
72         if core_status & 4:
73             raise RuntimeError("Core reports packet framing error")
74         tx_done = (tx_status & DMA_COMPLETE) == DMA_COMPLETE
75         rx_done = (rx_status & DMA_COMPLETE) == DMA_COMPLETE
76         if tx_done and rx_done and (core_status & 3) == 2:
77             return
78         if time.monotonic() >= deadline:
79             raise TimeoutError("HIL transfer timed out")
80         time.sleep(0.0001)
81
82
83 def release_buffers_safely(
84     dma: MMIO, buffers: List[PynqBuffer], timeout: float,
85 ) -> None:
86     """Keep CMA alive until both DMA channels are quiescent."""
87     if not buffers:
88         return
89     try:
90         reset_dma(dma, timeout)
91     except (Exception, KeyboardInterrupt):
92         print("DMA is not quiescent; retaining CMA buffers. "
93             "Reset the board if hardware cannot halt.",
94             file=sys.stderr, flush=True)
95     while True:
96         try:
97             if dma_halted(dma):
98                 break
99             time.sleep(0.1)
100         except (Exception, KeyboardInterrupt):
101             continue
102     failures: List[Exception] = []
103     for buffer in tuple(buffers):
104         try:
105             buffer.freebuffer()
106         except Exception as error:
107             failures.append(error)

```

```

108     if failures:
109         raise RuntimeError("CMA release failed after DMA halted") from failures
110     buffers.clear()
111
112
113 def validate_dma(
114     parameters: Mapping[str, Union[str, int]], nbytes: int,
115 ) -> Tuple[int, int, int]:
116     params = {key.lower(): value
117               for key, value in parameters.items()}
118     for name in ("c_include_mm2s", "c_include_s2mm"):
119         if int(params[name]) != 1:
120             raise ValueError("Both DMA channels are required")
121     for name in ("c_include_sg", "c_micro_dma"):
122         if int(params.get(name, 0)) != 0:
123             raise ValueError("Only non-micro simple DMA is supported")
124     for name in ("c_m_axis_mm2s_tdata_width",
125                 "c_s_axis_s2mm_tdata_width"):
126         if int(params[name]) != 64:
127             raise ValueError("DMA stream widths must be 64 bits")
128     length_bits = int(params["c_sg_length_width"])
129     if not 1 <= length_bits <= 32:
130         raise ValueError("Invalid LENGTH field width")
131     maximum = min((1 << length_bits) - 1, 0x03ffffff)
132     if nbytes <= 0 or nbytes > maximum:
133         raise ValueError(f"Payload {nbytes} exceeds limit {maximum}")
134     address_bits = int(params.get("c_addr_width", 32))
135     if not 1 <= address_bits <= 64:
136         raise ValueError("Unsupported DMA address width")
137     tx_width = int(params["c_m_axi_mm2s_data_width"])
138     rx_width = int(params["c_m_axi_s2mm_data_width"])
139     if tx_width <= 0 or rx_width <= 0 or tx_width % 8 or rx_width % 8:
140         raise ValueError("DMA memory widths must contain whole bytes")
141     tx_alignment = tx_width // 8
142     rx_alignment = rx_width // 8
143     return address_bits, tx_alignment, rx_alignment
144
145
146 class LoopbackDMA:
147     def __init__(self, overlay: Overlay, nbytes: int, timeout: float = 2.0,
148                 dma_name: str = "axi_dma_0", core_name: str = "
149 attention_core_0") -> None:
150         if type(nbytes) is not int or not math.isfinite(timeout) or timeout <=
151 0:
152             raise ValueError("Invalid byte count or timeout")
153         dma_info, core_info = overlay.ip_dict[dma_name], overlay.ip_dict[
154 core_name]
155         address_bits, tx_alignment, rx_alignment = validate_dma(dma_info["
156 parameters"], nbytes)
157         self.overlay = overlay

```

```

154     self.dma = MMIO(dma_info["phys_addr"], dma_info["addr_range"])
155     self.core = MMIO(core_info["phys_addr"], core_info["addr_range"])
156     self.nbytes, self.timeout = nbytes, timeout
157     self.buffers: List[PynqBuffer] = []
158     self.closed = False
159     self.failed = False
160     self.lock = Lock()
161     if self.core.read(12) != CORE_ID or self.core.read(8) != 1:
162         raise RuntimeError("Wrong shell ID/capability")
163     if self.core.read(0) & 1:
164         raise RuntimeError("Core busy; reset/reload overlay before
constructing driver")
165     try:
166         reset_dma(self.dma, timeout)
167         self.buffers.append(allocate(shape=(nbytes,), dtype=np.int8))
168         self.buffers.append(allocate(shape=(nbytes,), dtype=np.int8))
169         self.tx: PynqBuffer = self.buffers[0]
170         self.rx: PynqBuffer = self.buffers[1]
171         addresses = []
172         for buffer, alignment in zip(self.buffers, (tx_alignment,
rx_alignment)):
173             address = int(buffer.device_address)
174             if buffer.shape != (nbytes,) or buffer.dtype != np.int8 or not
buffer.flags.c_contiguous:
175                 raise ValueError("CMA shape/dtype/contiguity mismatch")
176             if address < 0 or address % alignment or address+nbytes > (1 <<
address_bits):
177                 raise ValueError("CMA address/alignment mismatch")
178             addresses.append(address)
179             if max(addresses) < min(address+nbytes for address in addresses):
180                 raise ValueError("DMA buffers overlap")
181             self.core.write(0, 2)
182             self.core.write(4, nbytes)
183             for offset in (16, 24, 28):
184                 self.core.write(offset, 0)
185         except BaseException:
186             self.close()
187             raise
188
189     def run(self, payload: NDArray[np.int8]) -> NDArray[np.int8]:
190         with self.lock:
191             if self.closed or self.failed:
192                 raise RuntimeError("Driver closed or poisoned; reset overlay
after a failed job")
193             if payload.dtype != np.int8 or payload.shape != (self.nbytes,):
194                 raise ValueError("Payload must match fixed INT8 buffer shape")
195             try:
196                 if self.core.read(0) & 1:
197                     raise RuntimeError("Unexpected BUSY")
198                 np.copyto(self.tx, payload)
199                 self.rx.fill(0)

```

```

200         self.tx.flush()
201         self.rx.flush()
202         start_channel(self.dma, 0x30, self.rx, self.nbytes, self.
timeout)
203         self.core.write(0, 1)
204         start_channel(self.dma, 0x00, self.tx, self.nbytes, self.
timeout)
205         wait_job(self.dma, self.core, self.timeout)
206         if self.dma.read(0x58) != self.nbytes:
207             raise RuntimeError("Unexpected received length")
208         self.rx.invalidate()
209         result = np.array(self.rx, dtype=np.int8, copy=True)
210         if not np.array_equal(result, self.tx):
211             raise RuntimeError("Loopback byte mismatch")
212         return result
213     except BaseException:
214         self.failed = True
215         raise
216
217     def close(self) -> None:
218         with self.lock:
219             if self.closed:
220                 return
221             self.failed = True
222             release_buffers_safely(self.dma, self.buffers, self.timeout)
223             self.closed = True
224
225     def __enter__(self) -> LoopbackDMA:
226         if self.closed:
227             raise RuntimeError("Driver already closed")
228         return self
229
230     def __exit__(self, exc_type: Optional[Type[BaseException]],
231                 exc_value: Optional[BaseException],
232                 traceback: Optional[TracebackType]) -> bool:
233         self.close()
234         return False

```

Listing D.2: pynq_dma_driver.py: exclusive, typed and fail-stop DMA ownership

Phụ lục E

Vivado build, ràng buộc timing và nghiệm thu

E.1 Build có tiền điều kiện: build_bitstream.tcl

Lưu script trong `scripts/`, RTL A.5 trong `rtl/`, XDC E.2 trong `constraints/`; chạy Vivado từ gốc cây mã. Script có hai chế độ. Lệnh `vivado -mode batch -source scripts/build_bitstream.tcl -tclargs ooc PART build/ooc` yêu cầu thay PART bằng part có trong cài đặt, tạo synthesis OOC và báo cáo, *không* tạo bitstream bo mạch. Không đoán part KV260 từ một chuỗi tên trong bản nháp.

Chế độ `overlay project/overlay.xpr build/overlay system` yêu cầu project SoC đã tạo và review, BD tên system, PS/DDR, DMA simple mode hai chiều, stream 64-bit, clock/reset, address map, wrapper top, pin/board constraints và các run `synth_1/impl_1` hợp lệ. Tên IP là `axi_dma_0` và `attention_core_0`. Người tích hợp phải xác nhận kết nối PS-DMA-shell, reset đồng bộ khi nhả và ánh xạ địa chỉ; script kiểm tra tên/cấu hình cơ bản không chứng minh toàn graph nối đúng. Source shell project phải trùng bản RTL tham chiếu trong cây mã.

Script reset run rồi build lại, vì vậy chỉ chạy trên project đã lưu và cho phép thay đổi run state. Không ghi đè thư mục kết quả. Gate trước bước bitstream yêu cầu run hoàn tất và có đường setup/hold với slack không âm. `check_timing`, DRC và timing summary được giữ để review [22]. Gate không thay thế kiểm tra unconstrained paths, CDC, recovery/removal, pulse width, cảnh báo critical hay kiểm thử bo mạch. Không vô hiệu hóa DRC để ép ghi bitstream.

HWH được lấy từ output của chính BD, không tự viết metadata giả; bit/HWH có cùng basename cho PYNQ [23]. Script chủ động dừng nếu layout artifact không khớp hai layout project thông dụng hoặc có nhiều ứng viên. XSA chỉ xuất sau run `write_bitstream` hoàn tất. Mã chưa được chạy bằng Vivado trong môi trường soạn thảo; không gọi đây là quy trình đã đạt timing trên KV260.

```
1 proc within_workspace {path} {
2     if {[file pathtype $path] ne "relative" || ".." in [file split $path]} {
3         error "Paths must stay within the launch directory"
4     }
5 }
6
7 proc reports {out} {
8     report_timing_summary -delay_type min_max -report_unconstrained \
```

```

9      -file [file join $out timing_summary.rpt]
10     report_utilization -file [file join $out utilization.rpt]
11     check_timing -verbose -file [file join $out check_timing.rpt]
12 }
13
14 proc require_slack {} {
15     foreach delay_type {max min} {
16         set paths [get_timing_paths -quiet -delay_type $delay_type -max_paths
17 1]
18         if {[llength $paths] == 0} {error "No constrained $delay_type path"}
19         set slack [get_property SLACK [lindex $paths 0]]
20         if {[string is double -strict $slack] || $slack < 0.0} {
21             error "Timing gate failed: $delay_type slack=$slack"
22         }
23     }
24 }
25
26 proc complete_run {name} {
27     wait_on_run $name
28     set run [get_runs $name]
29     if {[get_property PROGRESS $run] ne "100%" ||
30         ![string match "*Complete*" [get_property STATUS $run]]} {
31         error "Run $name did not complete: [get_property STATUS $run]"
32     }
33 }
34
35 proc build {arguments} {
36     if {[llength $arguments] < 3} {
37         error "Usage: ooc PART NEW_OUT | overlay PROJECT.xpr NEW_OUT BD_NAME"
38     }
39     lassign $arguments mode target out bd_name
40     within_workspace $out
41     if {[file exists $out]} {error "Output directory must be new"}
42     if {$mode eq "ooc"} {
43         if {[llength $arguments] != 3 || [llength [get_parts -quiet $target]] !
44 = 1} {
45             error "Specify one installed FPGA part for OOC"
46         }
47         foreach path {rtl/attention_core_top.sv constraints/
48 timing_constraints.xdc} {
49             if {[file isfile $path]} {error "Missing $path"}
50         }
51         file mkdir $out
52         create_project ooc [file join $out project] -part $target
53         add_files rtl/attention_core_top.sv
54         set_property verilog_define {SYNTHESIS} [current_fileset]
55         synth_design -top attention_core_top -part $target -mode out_of_context
56         read_xdc constraints/timing_constraints.xdc
57         reports $out
58         write_checkpoint [file join $out shell_ooc.dcp]
59         puts {OOC reports only: no board bitstream, no HWH, no routed timing

```

```

claim.}
57     close_project
58     return
59 }
60 if {$mode ne "overlay" || [llength $arguments] != 4} {
61     error "Invalid mode/argument count"
62 }
63 within_workspace $target
64 if {[!file isfile $target] || [file extension $target] ne ".xpr"} {
65     error "A reviewed board-level Vivado project is required"
66 }
67 if {[!regexp {[A-Za-z_][A-Za-z0-9_]*} $bd_name]} {error "Invalid BD name"}
68 }
69 open_project $target
70 set bd_files [get_files -quiet *${bd_name}.bd]
71 if {[llength $bd_files] != 1} {error "Expected exactly one named block
design"}
72 open_bd_design [lindex $bd_files 0]
73 foreach cell {axi_dma_0 attention_core_0} {
74     if {[llength [get_bd_cells -quiet $cell]] != 1} {error "Missing $cell"}
75 }
76 set dma [get_bd_cells axi_dma_0]
77 foreach {property expected} {
78     CONFIG.c_include_sg 0 CONFIG.c_micro_dma 0
79     CONFIG.c_include_mm2s 1 CONFIG.c_include_s2mm 1
80     CONFIG.c_m_axis_mm2s_tdata_width 64 CONFIG.c_s_axis_s2mm_tdata_width 64
81 } {
82     if {[get_property $property $dma] != $expected} {
83         error "DMA contract mismatch: $property"
84     }
85 }
86 if {[get_property CONFIG.c_sg_length_width $dma] < 18} {
87     error "Default webcam payload requires at least 18 LENGTH bits"
88 }
89 set rtl [get_files -all -quiet *attention_core_top.sv]
90 if {[llength $rtl] != 1 || ![file isfile rtl/attention_core_top.sv]} {
91     error "Expected one project shell source and one local reference source"
92 }
93 set channel [open [lindex $rtl 0] r]
94 set project_source [read $channel]
95 close $channel
96 set channel [open rtl/attention_core_top.sv r]
97 set local_source [read $channel]
98 close $channel
99 if {$project_source ne $local_source} {error "Project RTL differs from
reference"}
100 validate_bd_design
101 save_bd_design
102 generate_target all $bd_files
103 set defines [get_property verilog_define [current_fileset]]

```

```

103     set_property verilog_define [lsort -unique [concat $defines SYNTHESIS]] [
current_fileset]
104     update_compile_order -fileset sources_1
105     file mkdir $out
106     set version_file [open [file join $out tool_version.txt] w]
107     puts $version_file [version -short]
108     puts $version_file [get_property PART [current_project]]
109     close $version_file
110     reset_run synth_1
111     launch_runs synth_1 -jobs 4
112     complete_run synth_1
113     launch_runs impl_1 -to_step route_design -jobs 4
114     complete_run impl_1
115     open_run impl_1
116     reports $out
117     require_slack
118     report_drc -file [file join $out drc.rpt]
119     write_checkpoint [file join $out routed.dcp]
120     close_design
121     launch_runs impl_1 -to_step write_bitstream -jobs 4
122     complete_run impl_1
123     open_run impl_1
124     set run_directory [get_property DIRECTORY [get_runs impl_1]]
125     set bit_files [glob -nocomplain [file join $run_directory *.bit]]
126     if {[llength $bit_files] != 1} {error "Expected exactly one generated
bitstream"}
127     set project_directory [get_property DIRECTORY [current_project]]
128     set project_name [get_property NAME [current_project]]
129     set hwh_files {}
130     foreach suffix {gen srcs} {
131         set candidate [file join $project_directory ${project_name}.${suffix} \
sources_1 bd $bd_name hw_handoff ${bd_name}.hwh]
132         if {[file isfile $candidate]} {lappend hwh_files $candidate}
133     }
134     if {[llength $hwh_files] != 1} {error "Expected one generated HWH; inspect
project layout"}
135     file copy [lindex $bit_files 0] [file join $out attention_core.bit]
136     file copy [lindex $hwh_files 0] [file join $out attention_core.hwh]
137     write_hw_platform -fixed -include_bit -file [file join $out
attention_core.xsa]
138     puts {Build artifacts exported; review timing/CDC/DRC reports before board
deployment.}
139     close_project
140 }
141
142
143 if {[catch {build $argv} message]} {
144     puts stderr $message
145     exit 1
146 }
147 exit 0

```

E.2 Ngân sách timing OOC: timing_constraints.xdc

Clock $T = 5$ ns tương ứng mục tiêu $f = 1/T = 200$ MHz. Delay input/output max 1 ns, min 0.2 ns và uncertainty setup/hold 0.1/0.05 ns là *giả thiết môi trường OOC minh họa*, không phải số đo PCB hoặc clock PS. Mọi input trừ clk, kể cả reset, đều có delay; mọi output có delay, không chỉ TDATA. Reset ngoài môi trường này phải đáp ứng yêu cầu nhả reset đồng bộ; không đặt false path toàn reset để che recovery/removal.

Không nạp XDC này vào overlay một cách mù quáng: clock PS/Clocking Wizard và các đường AXI nội chip trong BD không phải external ports OOC. Project bo mạch cần constraints do thiết kế thực và board files cung cấp. Không có PACKAGE_PIN và IOSTANDARD giả cho các cổng AXI nội chip.

```

1 # OOC-only environment budget, NOT KV260 package-pin constraints.
2 set clock_port [get_ports -quiet clk]
3 if {[llength $clock_port] != 1} {error "Expected one OOC clk port"}
4 create_clock -name shell_clk -period 5.000 $clock_port
5 set_clock_uncertainty -setup 0.100 [get_clocks shell_clk]
6 set_clock_uncertainty -hold 0.050 [get_clocks shell_clk]
7 set input_ports [get_ports -filter {DIRECTION == IN && NAME != clk}]
8 set output_ports [get_ports -filter {DIRECTION == OUT}]
9 if {[llength $input_ports] == 0 || [llength $output_ports] == 0} {
10     error "Missing OOC interface ports"
11 }
12 set_input_delay -clock shell_clk -max 1.000 $input_ports
13 set_input_delay -clock shell_clk -min 0.200 $input_ports
14 set_output_delay -clock shell_clk -max 1.000 $output_ports
15 set_output_delay -clock shell_clk -min 0.200 $output_ports

```

Listing E.2: timing_constraints.xdc: complete OOC port budget, no board-pin claim

E.3 Hồ sơ nghiệm thu bắt buộc

Lưu commit/hash RTL, mã golden, manifest vector, kết quả Cocotb/JUnit, phiên bản công cụ, part, constraints, checkpoint routed, timing summary, check_timing, DRC, utilization, bit/HWH và log bo mạch. Chỉ đánh dấu nghiệm thu khi không có lỗi chưa giải thích, giao thức/thuật toán đạt bộ test đã khai báo và timing/CDC được review. Nếu không có công cụ, ghi “chưa chạy”, không thay bằng một bảng WNS/WHIS, tài nguyên hoặc FPS giả định.

Tài liệu tham khảo

- [1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. arXiv:2010.11929.
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. arXiv:1607.06450.
- [3] Zhikai Li and Qingyi Gu. I-ViT: Integer-only quantization for efficient vision transformer inference. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023. arXiv:2207.01405.
- [4] AMD. *UltraScale Architecture DSP Slice User Guide*, 2021. UG579, version 1.11, August 30.
- [5] AMD. *AXI DMA LogiCORE IP Product Guide*, 2025. PG021, version 7.1, June 24. Direct register mode and register address map.
- [6] Arm. *AMBA AXI-Stream Protocol Specification*. IHI 0051B. Handshake and byte qualifiers.
- [7] Arm. *AMBA AXI and ACE Protocol Specification*. IHI 0022H. Part B: AXI4-Lite.
- [8] AMD. *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, 2026. UG906, version 2026.1. Timing analysis and clock-pair table columns.
- [9] PYNQ Project. *Allocate: Physically Contiguous Buffers and Cache Synchronization*. PYNQ 3.0.0 documentation. Accessed 2026-09-14. https://pynq.readthedocs.io/en/v3.0.0/pynq_libraries/allocate.html.
- [10] cocotb contributors. *cocotb 2.0.1: Quickstart Guide*. Accessed 2026-09-14. <https://docs.cocotb.org/en/v2.0.1/quickstart.html>.
- [11] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs), 2016. arXiv:1606.08415.
- [12] PyTorch. *Dropout: Official API Documentation*. Accessed 2026-09-14. <https://docs.pytorch.org/docs/main/generated/torch.nn.Dropout.html>.
- [13] T. Wang, L. Gong, C. Wang, Y. Yang, Y. Gao, X. Zhou, and H. Chen. ViA: A novel vision-transformer accelerator based on FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(11):4088–4099, 2022. doi:10.1109/TCAD.2022.3197489.

- [14] Kyle Marino, Pengmiao Zhang, and Viktor Prasanna. ME-ViT: A single-load memory-efficient FPGA accelerator for vision transformers, 2024. arXiv:2402.09709.
- [15] AMD. *Vivado Design Suite Tcl Command Reference Guide*, 2026. UG835, version 2026.1. Command: `set_clock_uncertainty`.
- [16] PYNQ Project. *PynqBuffer: Device Addresses, Synchronization, and Buffer Lifetime*. Source code, tag v3.0.1, `pynq/buffer.py`. Accessed 2026-09-14.
- [17] OpenCV. *Color Space Conversions*. OpenCV 4.13.0 documentation. Accessed 2026-09-14. Official API reference.
- [18] PYNQ Project. *DMA Driver: Simple Transfers and Completion Handling*. Source code, tag v3.0.1, `pynq/lib/dma.py`. Accessed 2026-09-14.
- [19] cocotb contributors. *cocotb 2.0.1: The Timing Model*. Accessed 2026-09-14. https://docs.cocotb.org/en/v2.0.1/timing_model.html.
- [20] cocotb contributors. *cocotb 2.0.1: Makefile-based Build System and Test Runner*. Accessed 2026-09-14. <https://docs.cocotb.org/en/v2.0.1/building.html>.
- [21] Verilator contributors. *Verilator User Guide: Overview*. Accessed 2026-09-14. <https://verilator.org/guide/latest/overview.html>.
- [22] AMD. *Vivado Design Suite Tcl Command Reference Guide*, 2024. UG835, version 2024.1. Commands: `get_timing_paths`, `check_timing`, `report_timing_summary`, `write_hw_platform`.
- [23] PYNQ Project. *PYNQ 3.0.0: Loading an Overlay*. Accessed 2026-09-14. https://pynq.readthedocs.io/en/v3.0.0/pynq_overlays/loading_an_overlay.html.
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. arXiv:1706.03762.
- [25] PyTorch. *Module: Forward Hooks and Evaluation Mode*. Accessed 2026-09-14. <https://docs.pytorch.org/docs/main/generated/torch.nn.Module.html>.
- [26] PyTorch. *torch.round: Rounding to Nearest Even*. Accessed 2026-09-14. <https://docs.pytorch.org/docs/main/generated/torch.round.html>.
- [27] cocotb contributors. *cocotb 2.0.1: Verilator Simulator Makefile*. Source code, tag v2.0.1. Accessed 2026-09-14. https://github.com/cocotb/cocotb/blob/v2.0.1/src/cocotb_tools/makefiles/simulators/Makefile.verilator.